

机器人控制与仿真原理

(1.0)

作者 乔大大 Qisheng

说明

本文旨在围绕通用机器人的发展，分析机器人控制与仿真相关原理。

作者水平所限，文理可能存在偏误，欢迎读者指正。

注，本文多用网络图例，以便概念说明。若及版权或不妥，请联系移去！

信箱， ThinkingLearning@163.com

2026 年 1 月

目录

总体概要篇.....	1
一、 机器人控制.....	2
1. 控制方法.....	3
2. 大模型.....	3
3. 控制框架.....	4
二、 机器人仿真.....	5
三、 定位建图(SLAM).....	6
四、 三维重建.....	6
五、 强化学习.....	7
1. 基于价值的方法.....	10
2. 基于策略的方法.....	12
3. 基于 Actor-Critic 的方法.....	13
4. 蒙特卡洛树搜索(MCTS , Monte Carlo Tree Search).....	18
机器人控制篇.....	23
第一章. 机械臂.....	24
一、 机械臂类型.....	24
二、 运动学.....	25
1. 正向运动学.....	25
2. 逆向运动学.....	26
三、 动力学.....	28
1. 牛顿-欧拉法(Newton-Euler).....	29
2. 欧拉-拉格朗日法(Euler-Lagrange).....	30
3. 正向动力学.....	34
4. 逆向动力学.....	35
四、 路径规划.....	35
五、 轨迹规划.....	35
第二章. 传统机器人.....	36
一、 平衡准则.....	36
1. 零力矩点.....	36
2. 捕获点(Capture Point).....	37
3. 质心运动散量.....	39
4. 扭力和.....	40
二、 线性倒立摆模型(LIPM, Linear Inverted Pendulum Model).....	41
1. 模型图示例.....	41
2. 零力矩方程.....	41
三、 整体模型(Whole Body Model).....	42
1. 运动方程.....	43

2.	方程推算.....	44
四、	运动控制.....	46
1.	比例积分控制器(PID, Proportional Integral Derivative)	47
2.	线性二次调整器(LQR, Linear Quadratic Regulator)	47
3.	模型预测控制(MPC, Model Predictive Control).....	48
4.	整体控制(WBC, Whole Body Control).....	52
5.	混合零动力(HZD, Hybrid Zero Dynamics)控制	58
第三章.	机器人运动控制-VLA	59
一、	总体框架.....	59
二、	未来隐表征对齐(FLARE)	59
1.	损失函数.....	60
2.	视觉语言模型.....	61
3.	查询变换器(Q-Former, Querying Transformer)	62
三、	DreamGen	63
1.	总体框架.....	63
2.	视频世界模型.....	64
3.	还原动作模型.....	64
第四章.	机器人运动控制-RL.....	66
第五章.	机器人手操作-RL.....	67
一、	任务定义.....	67
二、	环境定义.....	68
三、	策略学习.....	68
四、	奖励定义.....	70
五、	仿真.....	71
机器人仿真篇.....		72
第一章.	动力学.....	73
第二章.	笛卡尔坐标系物理引擎.....	78
第三章.	广义坐标系物理引擎.....	79
第四章.	仿真引擎-MuJoCo	81
一、	MuJoCo 特性.....	81
二、	动力学解算.....	82
三、	连接仿真解算算法.....	85
1.	隐式互补解算器.....	85
2.	凸解算器.....	85
3.	对角解算器.....	86
四、	MuJoCo 实践.....	86
1.	实例一	87
2.	实例二.....	90
第五章.	仿真引擎-Newton.....	94
第六章.	训练场-MuJoCo Playground	95
传统 SLAM 篇		103
第一章.	光束平差(BA, Bundle Adjustment)	106
一、	算法思路.....	106
二、	优化过程.....	107

三、	SLAM 应用	110
第二章.	卡尔曼滤波(KF, Kalman Filter)	111
一、	算法思路.....	112
二、	数学定义.....	112
三、	举例推导.....	113
四、	预测校准.....	115
五、	推导方法.....	116
六、	SLAM 应用	116
第三章.	扩展卡尔曼滤波(EKF, Extend Kalman Filter).....	117
一、	线性推导.....	117
二、	误差方程.....	119
三、	预测校准.....	121
四、	SLAM 应用	122
第四章.	图优化(Graph Optimization).....	123
一、	优化原理.....	123
二、	数学定义.....	124
三、	优化算法.....	126
四、	SLAM 应用	126
深度学习 SLAM 篇		128
第一章.	DROID-SLAM	129
一、	总体框架.....	129
1.	基本定义.....	130
2.	重投影.....	131
3.	修正投影.....	132
4.	预测位姿及深度变化.....	135
5.	更新预测.....	136
二、	训练.....	137
1.	计量自由(Gauge Freedom).....	137
2.	训练视频.....	137
3.	监督.....	137
三、	推理.....	138
1.	初始化.....	139
2.	前端.....	139
3.	后端.....	139
四、	优化计算.....	140
1.	投影函数.....	140
2.	投影及逆投影函数的雅可比.....	142
3.	三维点变换的雅可比.....	142
五、	RAFT(Recurrent All-Pairs Field Transformers)	143
第二章.	NERF-SLAM.....	144
一、	总体框架.....	144
二、	表征.....	145
三、	追踪.....	146
1.	光流更新.....	146

2.	更新计算.....	146
3.	置信计算.....	148
四、	建图.....	148
五、	训练.....	150
第三章.	3D Gauss Splatting SLAM	151
一、	三维重建-Gaussian Splatting.....	151
1.	总体框架.....	151
2.	栅格化(Rasterization)算法	151
3.	像素颜色合成.....	152
二、	3D Gaussian Splatting SLAM	152
三、	总体框架.....	153
1.	投影变换(Projective Transformation).....	153
2.	相机位姿与三维高斯位姿导.....	154
四、	追踪.....	156
1.	光学残差.....	156
2.	几何残差.....	156
3.	目标函数.....	157
五、	关键帧(Keyframing)	157
六、	建图.....	158
1.	各向同性残差.....	158
2.	目标函数.....	158
3.	RGB-D	159
三维重建篇.....		160
第一章.	截取符号距离(TSDF)	161
第二章.	占据网络(Occupancy Networks)	164
一、	模型框架.....	164
1.	调控模型.....	165
2.	生成模型(Unconditional Mesh Generation Model)	166
二、	模型定义.....	167
1.	可控模型.....	167
2.	生成模型.....	167
三、	模型训练.....	167
1.	可控模型.....	168
2.	生成模型.....	168
四、	模型推理.....	169
1.	体素划分.....	169
2.	网面提取.....	170
3.	网面简化.....	170
4.	网面优化.....	170
第三章.	神经辐射场(NeRF).....	172
一、	模型框架.....	173
1.	NeRF 框架	173
2.	多层感知器框架.....	173
二、	体渲染.....	174

1.	相机光线定义.....	174
2.	相机光线颜色定义.....	174
3.	相机光线颜色估计.....	175
三、	优化计算.....	176
1.	位置编码(Positional Encoding).....	176
2.	层级采样(Hierarchical Volume Sampling)	176
3.	损失函数.....	177
第四章.	三维高斯泼溅(3D Gaussian Splatting)	178
一、	模型框架.....	178
二、	模型定义.....	179
1.	三维高斯定义.....	179
2.	高斯投影.....	179
3.	像素颜色定义.....	180
4.	损失函数定义	180
三、	优化控制.....	181
1.	优化控制.....	181
2.	栅格渲染.....	182
3.	稠密控制.....	183
世界模型篇.....		184
机器人操作系统篇-ROS2.....		187
参考文献.....		190

总体概要篇

机器人控制实现，第一步，确定机器人模型；第二步，预测机器人动作。

传统机器人控制方法，从内向外，从后向前。第一步，假定机器人本体刚体模型，比方，倒立摆；第二步，已知特定任务，比方，行走轨迹，基于刚体模型及环境连接力(Contact)假设等，倒推计算机器人每步的电机控制量。

机器学习控制方法，从外向内，从前向后。第一步，基于外部数据拟合机器人思维控制模型，比方，看到当前图像、听到当前指令，应作何动作反应；第二步，基于拟合的模型，在应对新任务指令时，预测未来每步的电机控制量。

传统机械臂基于运动学(Kinematics)及动力学(Dynamics)计算关节力矩及终端执行器位置、速度等，适用于有限特定场景及任务。

传统机器人基于倒立摆、整体模型等，利用线性二次调整(LQR, Linear Quadratic Regulator)、模型预测控制(MPC, Model Preview Control)及整体控制(WBC, Whole Body Control)等优化计算、控制机器人运动。

通用机器人，基于神经网络拟合人类思维函数。不同于传统机器人基于物理力学、动能分析定义变换函数。

注，传统机械臂、机器人、SLAM 等与神经网络方法表象似都是基于目标函数、优化算法进行计算，实质迥异。传统方法以**机器人本体物理模型为中心**，简化模型的同时，简化状态，基于任务优化**计算物理模型控制量**，无法主动适用多样化环境；神经网络方法以**状态变换数学模型为中心**，不限制状态，基于大规模数据**优化拟合变换函数**，预测控制量，可具备主动及通用性。

基于机器人运动控制过程来看，神经网络方法是基于整体运动变换、更本质的机制。跳离对机器人本体模型的定义，模仿人类思维，直接计算状态变换函数。

当前，通用机器人基于视觉语言动作模型进行运动控制，模型有 RT-1、RT-2、Gemini-Robotic、Phi、SmolVLA、GR00T 等。

RT-1，基于 FiLM EfficientNet 及 标准 Transformer Decoder-only 的视觉语言动作模型。基于

Transformer 框架 不过不是 Diffusion Transformer.

RT-2，基于 PaLI-X 或 PaLM-E 视觉语言模型，直接生成动作的视觉语言动作模型(RT-2-PaLI-X and RT-2-PaLM-E)。

注，当前通用机器人的控制挑战应在于步态平衡及灵活手控制。

机器人仿真是机器人模型训练及应用到物理现实场景的重要前提。

常用的仿真平台，Gazebo、Nvidia Omniverse、Mujoco 等。在物理引擎、场景合成、渲染水平、传感器支持度、硬件需求、机器人操作系统 ROS 结合、强化学习支持、社区支持、开源支持等方面不同。

CRITERIA	GAZEBO	NVIDIA OMNIVERSE ISAAC SIM™	MUJOCO	O3DE	MVSIM
Physics	☆☆☆	☆☆☆☆☆	☆☆☆☆☆☆	☆☆☆☆☆	☆☆
Rendering	☆☆☆	☆☆☆☆☆	☆☆	☆☆☆☆☆	☆
ROS 2 integration	☆☆☆☆☆	☆☆☆☆☆	☆	☆☆☆☆☆	☆☆☆☆☆
Hardware Requirements	☆☆☆☆☆	☆☆	☆☆☆☆☆	☆☆☆☆☆	☆☆☆☆☆
Sensors support	☆☆☆☆☆	☆☆☆☆☆	☆☆	☆☆☆☆☆	☆☆☆☆
Community	☆☆☆☆☆	☆☆☆☆☆	☆☆☆☆☆	☆☆	☆☆

机器人场景重建、场景合成等涉及 SLAM、三维重建及世界模型。

一、 机器人控制

1. 控制方法

机器人控制基本可分作运动控制(Motion Control)及操作控制(Manipulation Control)。

控制方法基本可分作基于模型的运动学(Kinematics)、动力学(Dynamics)方法及不基于模型的机器学习方法。

比方，基于倒立摆模型的机器人步态控制(Gait Control)、基于中枢模式生成器(CPG, Central Pattern Generator)-藏本模型(Kuramoto Model)的机器人四肢节律运动(Rhythmic Behavior)；基于视觉语言动作大模型的机器人手操作控制等。

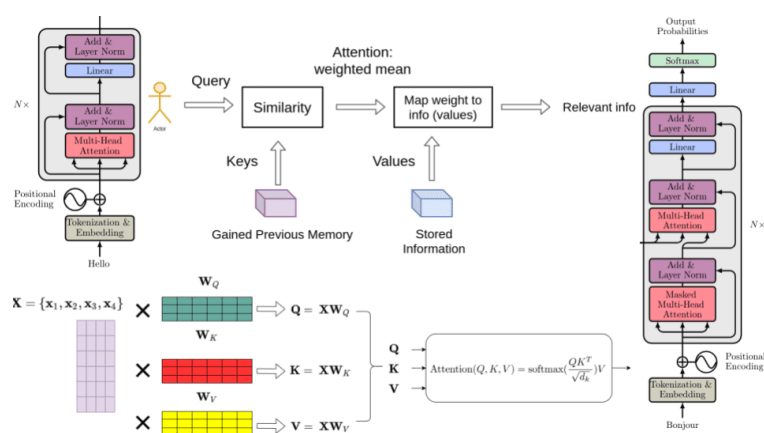
2. 大模型

传统机器学习利用有限的数据训练拟合形式基本确定的函数，容易过拟合或缺乏泛化能力。

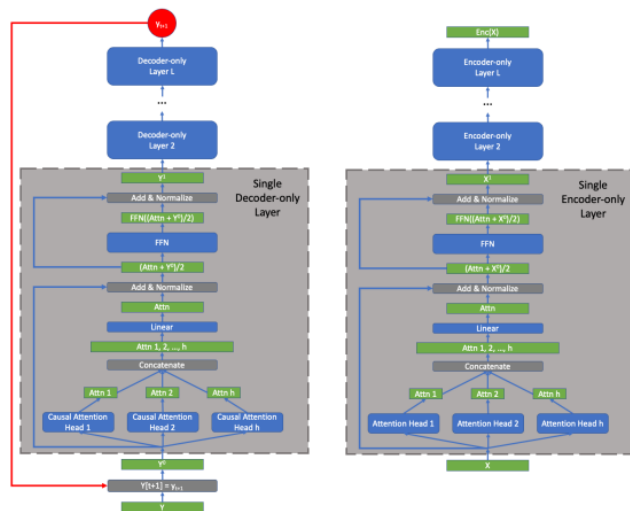
深度学习基于对神经网络的模仿，利用海量数据拟合具有泛化能力的函数模型。

基于 Transformer 及规模定律(Scaling Law)的大模型，利用多模态数据表征、高维张量注意力机制、专家混合路由模式、扩散或流匹配生成、梯度优化等算法，结合模仿学习、强化学习、GPU/NPU 集群、分布训练拟合可与现实对齐(Ground)、同时具有泛化能力的函数模型。

Transformer 框架示例。



Transformer Decoder-Only 及 Encoder-Only 框架。



注，Transformer 核心实质类似多个支持向量机(SVM)的测量加权。输入嵌量基于不同的基函数变换到相同维度空间，进行注意力测量计算(Attention)；多个测量结果加权计算综合注意力(Multi-Head Attention)；注意力基于前馈网络进行激活表征。

当前，存在着模仿人类情感价值观作奖励函数以及新的框架实现世界模型的趋势。

3. 控制框架

机器人控制框架基本可分作传统的模型预测控制(MPC, Model Predictive Control)、整体控制(WBC, Whole Body Control)，基于深度学习的模仿学习、强化学习训练框架等。

基于不同的控制框架及控制方法，可产生不同的机器人控制设计。

比方，基于 MPC+ 倒立摆模型、MPC+CPG、RL+CPG、MPC+WBC+PD、RL+HZD+PD、DeepLearning+CPG 等。

二、 机器人仿真

机器人仿真器，Gazebo、MuJoCo、Pybullet、Webots、V-REP、Nvidia Isaac Sim、Agility Robotics AR-Sim 等。

仿真器主要由渲染引擎、物理引擎组成，基于物理引擎进行物理建模及运动仿真，通过渲染引擎进行二维、三维显示，界面可基于 Qt 等实现。

渲染引擎，OpenGL、DirectX、Vulkan、Nvidia OptiX 光线追踪器(Ray Tracer)、路线追踪器(Path Tracer)等。

物理引擎，可看作基于当前位置、速度、控制力(τ)及惯性特性(μ)计算下一时刻位置及速度的函数，动力学状态可基于广义坐标系(Generalized Coordinate；最小坐标系，Minimal Coordinates；关节坐标系，Joint Coordinates)，或笛卡尔坐标系(Cartesian Coordinate；最大坐标系，Maximal Coordinates，六自由度；XYZ 坐标系)。

$$P(\mathbf{q}_t, \dot{\mathbf{q}}_t, \boldsymbol{\tau}, \boldsymbol{\mu}) = [\mathbf{q}_{t+1}, \dot{\mathbf{q}}_{t+1}]$$

物理引擎类型可分为不可微分引擎(Non-Differentiable Physics Engines；基于梯度的，Gradient-Based)及可微分引擎(Differentiable Physics Engines；非梯度的，Gradient-Free)类型，ODE、Bullet、PhysX(Unreal、Unity3D)、MuJoCo、Havok、DART、Simbody、Vortex 等。

Engine	Application	Integrator	State	Contact Type	Contact Solver	Gradients
MuJoCo	robotics	RK4	minimal	soft	Newton	finite difference
Drake	robotics	implicit Euler	minimal	soft/hard	LCP	randomized smoothing
Bullet	graphics	implicit Euler	minimal	soft/hard	LCP	subgradient
DART	robotics	implicit Euler	minimal	hard	LCP	subgradient
PhysX	graphics	explicit	minimal	soft	iterative	finite difference
Brax	graphics	explicit	maximal	soft	iterative	subgradient

学院及工业界常用， Gazebo。

自动交通工具行业， Carla ， SUMO。

强化学习仿真器， Pybullet ， Isaac Sim ， Mujoco 。

三、 定位建图(SLAM)

传统 SLAM 主要在于定位，所建地图是稀疏的。

比方，光束平差、卡尔曼滤波、图优化等方法。

深度学习及高斯泼溅等的发展促进了 SLAM 建图的光学及几何准确度。

比方， DROID-SLAM、Nerf-SLAM、3D Gaussian Splatting-SLAM 等。

四、 三维重建

传统三维重建基于符号距离、占据网络等。

当前，基于深度学习及高斯泼溅等的发展，形成神经辐射场(Nerf)、3D Gaussian Splatting 等方法。

五、 强化学习

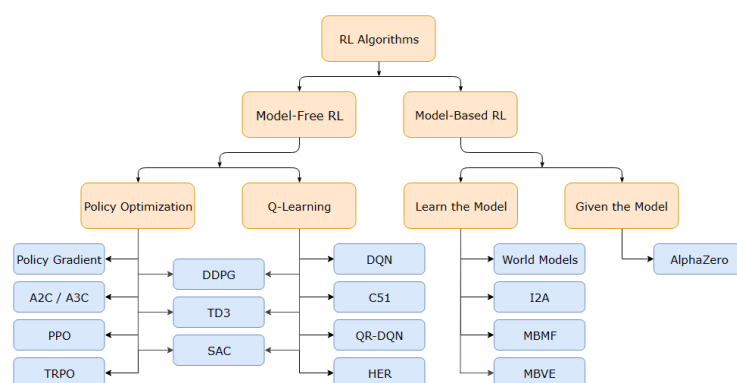
强化学习(RL, Reinforcement Learning), 代理(Agent)在每一时步, 基于当前状态选择最优动作, 使整个决策过程的累积奖励(Cumulative Reward)或回报(Return)最大。因此, 强化学习过程对应马尔科夫决策过程。

强化学习基于环境模型情况, 可分作无模型(Model-Free)、基于模型(Model-Based)两类。

无模型, 不关心环境模型。比方, 机器人运动控制, 在每一时步, 机器人可以存在无限个动作选项。

基于模型, 环境模型已知, 或者需要学习。比方, 围棋, 代理在每一时步基于固定的规则选择动作。

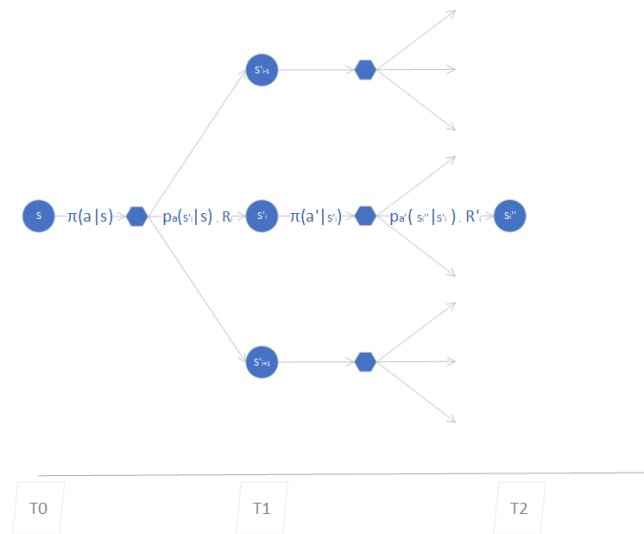
无模型学习, 可分作基于价值(Value-Based)、基于策略(Policy-Based)的方法。实际常基于两者, 形成表演-评价(Actor-Critic)方法。基于模型学习, 可分作已知模型、模型可学习。



强化学习中, 基于状态预测动作, 称策略函数; 对动作优劣进行立即反馈(Immediate Feedback), 称奖励函数。基于当前动作, 在特定动作时间轴或轨迹线上, 对动作进行折扣(Discount)加权, 称回报(Return); 基于当前状态, 全部动作回报的期望, 称状态价值函数; 基于特定状态-动作对, 相应的动作奖励+下一状态价值的折扣, 称动作价值或 Q(uality)函数。

注, 奖励, 特定动作的立即反馈; 回报, 特定动作轨迹的加权反馈; 价值函数基于动作或动作轨迹概率, 是动作回报期望或均值的反映。回报、价值都可定义作基于折扣加权的贝尔曼递归方程。

状态、动作、状态迁移、奖励示意图。



多臂老虎机(Multi-Armed Bandit)，强化学习的基本示例，主要阐释状态、动作、奖励、利用及探索(Exploit-Explore)的平衡问题。

赌博机。比方，老虎机有三个玻璃框，里面有不同的图案，投币之后拉下拉杆，就会开始转，当显示图像相同等特定形式时就会奖励，相同图型越多奖金则越高。

多臂或多台老虎机，每台老虎机有一个类似机械臂的连杆，拉一下机械臂，老虎机随机提供一定金币作奖励。通过多次尝试，获得最大奖益。

老虎机拉杆动作不改变状态(State)，因此是无状态(Stateless)强化问题。

多臂老虎机常用小概率-贪婪算法(ϵ -Greedy)，基于小概率进行探索，大多数情况利用已知经验执行。

假定，在 k 台老虎机中，基于经验已知一台奖励高，别的老虎机奖励概率不知；现在对这 k 台机器选 10 次。这 10 次中，只 1 次，或者，以概率 $\epsilon = 1/10$ ，在所有 k 台老虎机中随机进行选择，以探索(Explore)可能更大的奖励；大多数情况，选已知奖励的老虎机，以利用已知信息(Exploit)获利。

强化学习可基于状态、动作空间规模分作小空间及大空间类型。相应的解法对应表式(Tabular)及近似解法。[R. Sutton, etc. 2015]

表式强化学习问题(Tabular Reinforcement Learning), 状态、动作空间充分小, 可表示作数组或表格(Table)形式。这类情况, 一般可找到准确解或真正的最优解。

这类强化学习问题的三类基本解法。

- 1、动态规划。需要知道环境模型。当前全部轨迹下一步的奖励均值作目标值, 迭代更新。
- 2、蒙特卡洛方法。无法逐步累计计算(Incremental Computation)。基于当前状态探索完整轨迹的价值作目标值, 迭代更新。
- 3、时序差分方法(TD, Time Difference)。计算分析繁琐。基于当前状态动作最大值作目标值, 迭代更新。

蒙特卡洛法、时序差分法可与模型学习、规划方法结合, 形成 Tabular 类型强化学习问题的统一解法。

注, 蒙特卡洛法估计器估计所有步, 偏差 0, 方差大, 容易过拟合; 时序差分法只利用一步估计, 方差小, 偏差打, 易欠拟合。

基于蒙特卡洛方法, 意味着要遍历该奖励之前所有的动作来找到确定的轨迹(patterns), 这样的轨迹不一定适用于未来的回合局(Episodes), 因此是过拟合的。

基于时序差分方法, 只看该奖励之前的一个动作, 但是却忽略了此前别的动作的累积才导致了当前的奖励, 因此找不到轨迹(Pattern), 即是欠拟合的。

可通过 TD(λ) 方法统一动态规划、蒙特卡洛及时序差分法。

相反于 TD(λ) 基于未来价值更新策略, 资格迹(Eligibility Traces)可基于历史轨迹价值更新策略。因此, TD(λ) 可称前向视野法(Forward View), 资格迹可称后向视野法(Backward View)。

资格迹递归方程。

$$\begin{aligned} e_0(s) &\leftarrow 0 \quad \forall s \in S \\ e_t(s) &\leftarrow \lambda \gamma e_{t-1}(s) + \mathbf{I}(S_t = s) \quad \forall s \in S \end{aligned}$$

s_t , 状态。

I_t , 单位函数。当前状态的历史轨迹中的状态对应 1。

方程逻辑意义，当前状态历史轨迹的状态影响当前状态，离当前状态越近的历史状态影响越大。

深度强化学习，基于深度神经网络拟合强化学习中的策略函数或价值函数。策略函数训练基于价值最大化；价值函数训练基于动作价值对齐，训练的价值函数可作状态动作决策。

1. 基于价值的方法

1) SARSA

SARSA(State Action Reward State Action)，预测与目标价值网络基于同一网络；状态动作样本基于相邻状态动作。

动作价值更新方程。

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$

价值网络基于环境交互产生 $\{(s, a, r, s', a')\}$ 样本集。

基于预测网络 $Q(s, a)$ 预测动作价值。

基于目标网络 $Q'(s', a')$ 计算下一状态动作价值。

基于预测误差优化更新价值网络。

循环更新，完成训练局(Episode)。

2) Q-Learning

Q-Learning，基于目标网络计算状态最大动作价值，学习最优价值函数。

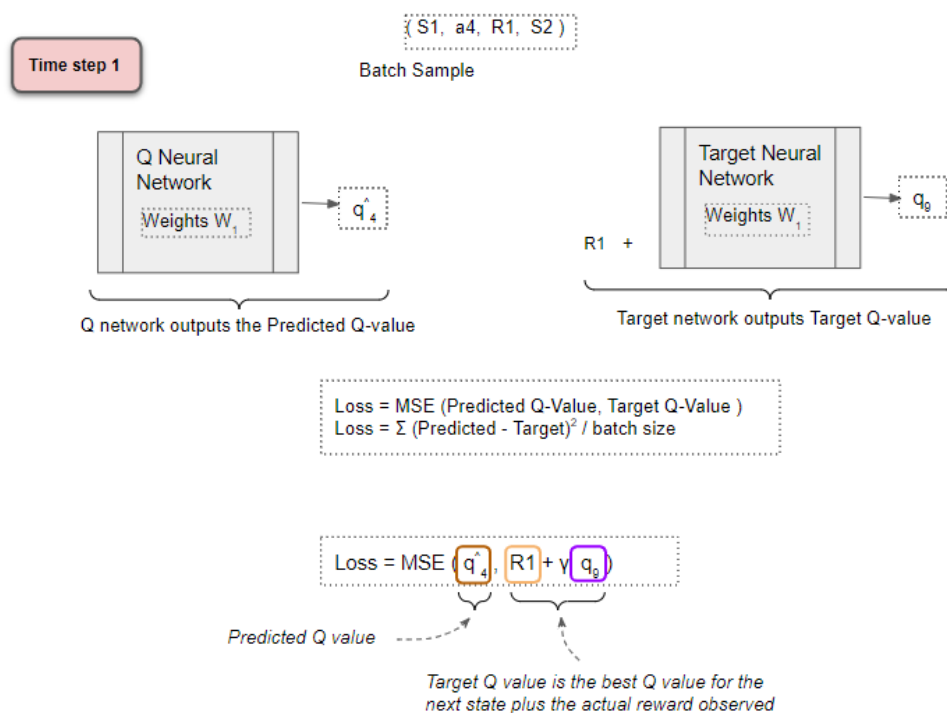
动作价值更新方程。

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[\underbrace{R_{t+1} + \gamma \max_a Q(S_{t+1}, a)}_{\text{TD Target (uses maximum Q-value over next actions)}} - Q(S_t, A_t) \right]$$

注，SARSA，基于目标网络计算状态动作价值 $Q'(s', a')$ ，学习当前策略的价值函数。动作价值更新方程。

3) DQN

深度 Q 网络(DQN, Deep Q Networks)，核心在于，基于动作价值网络预测的价值与该动作相应的递归方程计算的价值进行对齐。



框架图，选择样本($S1, a4, R1, S2$)；Q 网络预测状态 $S1$ 的动作价值中，选 $q(S1, a4)$ 作预测值；目标网络(Target Neural Network)计算下一状态 $S2$ 的动作价值中，选最大值，再加奖励 $R1$ 作目标 Q 值；基于批样本的预测值与目标 Q 值的均方差损失函数，更新 Q 网络权重。

经历多次更新后，Q 网络权重拷贝到目标网络，开始再一次的 Q 网络训练。

2. 基于策略的方法

策略网络参数更新方程。

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

当学习率 α 过大时，新策略可能完全不同于此前策略，预测的动作相对更新前策略动作可能变化很大，带来极大不确定。形成策略坍塌(Policy Collapse)。

3. 基于 Actor-Critic 的方法

1) DDPG

深度确定策略梯度(DDPG, Deep Deterministic Policy Gradient), 核心在于, 策略网络(Actor)以动作价值网络(Critic)函数作目标函数; 同时, 动作价值网络是策略网络的函数。因此, 目标函数的梯度等于动作价值网络对动作的梯度与策略网络对参数梯度的乘积。动作价值网络参数更新方式与 DQN 相似, 区别在于, DQN 的目标动作价值网络(Target Q Network)基于下一状态预测动作价值分布, DDPG 的目标动作价值网络基于下一状态及目标动作网络(Target Actor Network)确定的下一动作预测动作价值。

策略网络与动作价值网络。



策略网络目标函数定义。

$$J(\theta^\mu) = E_{s_t \sim \rho^\theta} [Q(s_t, \mu(s_t | \theta^\mu))]$$

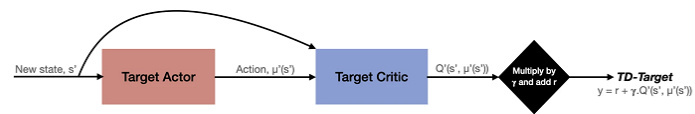
目标函数关于策略网络参数梯度。

$$\nabla_{\theta^\mu} J = E_{s_t \sim \rho^\theta} [\nabla_a Q(s, a | \theta^Q) |_{a=\mu(s)} \nabla_{\theta^\mu} \mu(s | \theta^\mu)]$$

动作价值网络损失函数定义。

$$L(\theta^Q) = E_{(s, a, r, s') \sim D} [(y_t - Q(s, a | \theta^Q))^2]$$

目标策略网络与目标动作价值网络。



损失函数对齐真值。

$$y_t = r + \gamma Q'(s', \mu'(s' | \theta^{\mu'}))$$

Q' ，目标动作价值网络。

μ' ，目标动作网络。

r ，样本中当前状态及动作的奖励。

s' ，样本中下一状态。

注，基于策略网络预测动作时，通过添加噪声以加强探索(Exploration)能力；目标网络基于策略网络或动作价值网络参数线性加权进行软更新(Soft Update)。

算法过程。

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q, \theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
 Initialize a random process $\mathcal{N}$ for action exploration
 Receive initial observation state s_1
 for $t = 1, T$ **do**
 Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
 Execute action a_t and observe reward r_t and observe new state s_{t+1}
 Store transition (s_t, a_t, r_t, s_{t+1}) in R
 Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
 Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'}))|\theta^{Q'}$
 Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
 Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

 Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

 end for
end for

算法步骤。

第一步，基于策略(Actor)及价值网络(Critic)克隆相应目标网络。

第二步，基于策略网络预测动作，加噪。

第三步，基于加噪动作与环境进行交互，记录状态、动作、奖励样本(s, a, r, s')。

第四步，基于小批量样本，利用目标动作价值网络及目标策略网络，通过递归加权方程计算价值网络对齐真值。

第五步，基于动作价值网络损失函数优化，更新价值网络。

第六步，基于策略网络目标函数梯度优化，更新策略网络。

第七步，基于 Polyak 方法更新目标网络。

2) TD3

孪生延迟深度确定策略梯度网络(TD3, Twin Delayed Deep Deterministic Policy Gradient)，基于 DDPG 进行改进。通过孪生动作价值网络分别预测动作价值，选最小值加权计算目标 Q 值，以最小化过估计偏差(Overestimation Bias)；在目标策略网络(Target Policy)生成动作时，添加

噪声(Small Clipped Noise)以防止动作价值网络过拟合；每 **N** 步更新策略网络参数。

动作价值网络损失函数中的对齐真值。

$$y = r + \gamma \times \min(Q'_1, Q'_2)$$

Q'_1 ，孪生目标动作价值网络之一。

Q'_2 ，孪生目标动作价值网络之一。

3) SAC

相对于 TD3，SAC(Soft Actor-Critic)策略网络预测的是动作分布，同时，目标函数基于动作价值及熵正则项(Entropy Regularization)和的最大化。

4) PPO

近端策略最优(PPO , Proximal Policy Optimization), 基于 Actor-Critic 框架实现的策略优化算法。

核心在于，使近端策略，或者说相近于原策略动作概率的新策略动作的价值优势函数 (Advantage Function)最大。

注，动作价值优势函数，动作价值与状态价值差。

裁剪替代目标函数(Clipped Surrogate Objective)。

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$$

r_t ，新旧策略动作概率比例(Ratio)。

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

ϵ , 限制动作概率比例取值区域的超参。

优势函数定义。

$$\hat{A}_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

PPO 整体目标函数方程。

$$L_{\text{total}}(\theta) = L_{\text{CLIP}}(\theta) - c_1 L_{\text{VF}}(\phi) + c_2 H[\pi_\theta]$$

方程逻辑意义，第一项，使相近于原策略动作概率的新策略动作的价值优势最大；第二项，价值网络预测误差最小；第三项，选小概率动作以加强策略网络探索能力。

L_{VF} , 价值网络损失函数。

$$L_{\text{VF}}(\phi) = \mathbb{E}_t \left[(V_\phi(s_t) - V_t^{\text{target}})^2 \right]$$

$H[\pi_\theta]$, 熵正则项。

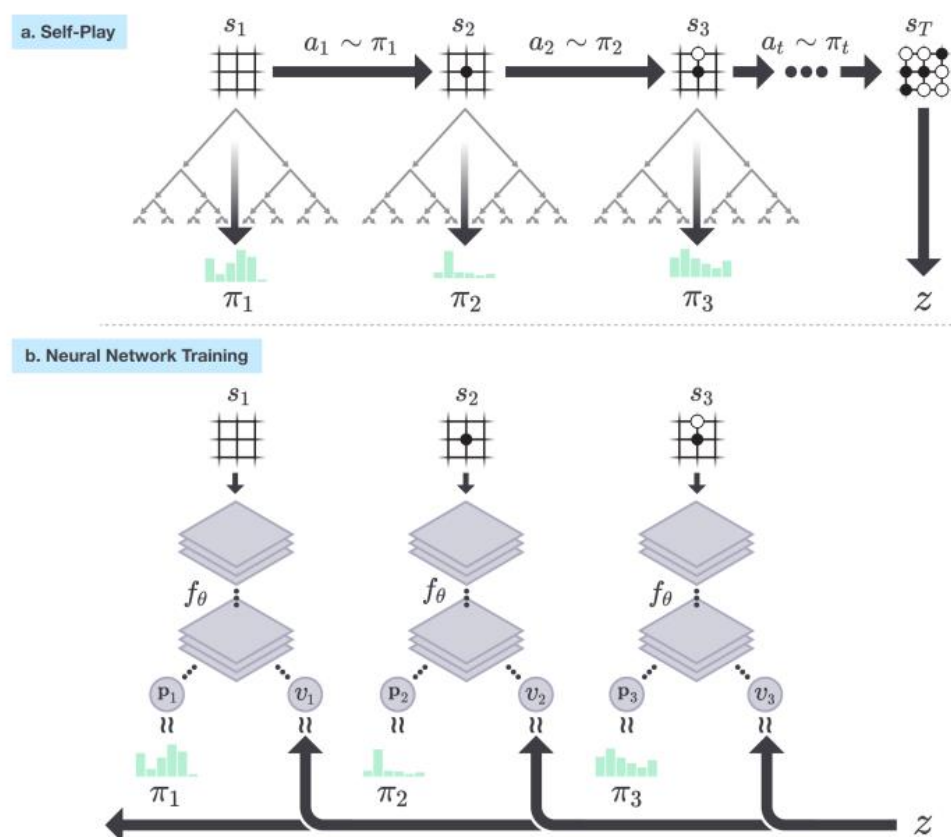
$$H[\pi_\theta] = \mathbb{E}_t [-\log \pi_\theta(a_t | s_t)]$$

4. 蒙特卡洛树搜索(MCTS , Monte Carlo Tree Search)

AlphaZero 等基于模型的强化学习应用中，常利用蒙特卡洛树搜索方法提供对弈步骤中的动作分布。

注，AlphaZero，基于 AlphaGo Zero 算法改进，更具通用性(Generic)。适用于国际象棋(Cheess)、将棋(Shogi From Chaturanga)及围棋(Go From Weiqi)等棋盘类游戏(Board Games)。

AlphaGo Zero 自博弈(Self-Play)强化学习框架。



框架图，顶部(Self-Play)，整体训练过程。基于图像样本集定义棋局不同时刻棋子状态，每个图像对应不同棋子位置， $\{s_1, s_2, s_3, \dots, s_T\}$ ；每个位置状态 s_t ，通过执行蒙特卡洛树搜索(MCTS) α_0 计算搜索概率(Search Probabilities)或动作分布 π_t ，采样得到动作 a_t ；在棋局结束状态 s_T ，基于对弈规则计算得分(Scored)，判断赢方 z 。

底部(Neural Network Training)，蒙特卡洛搜索树仿真步骤中的神经网络 f_θ 的训练。神经网络预测的动作概率分布 p_t 及动作价值 v_t ，分别与前一时刻预测的搜索概率 π_t 及棋局结束态赢

方 z 进行对齐。

神经网络函数 f_θ 及损失函数 l 定义。

$$(\mathbf{p}, v) = f_\theta(s), \quad l = (z - v)^2 - \boldsymbol{\pi}^\top \log \mathbf{p} + c \|\boldsymbol{\theta}\|^2$$

$\mathbf{p}$, 神经网络预测的动作概率分布。向量。

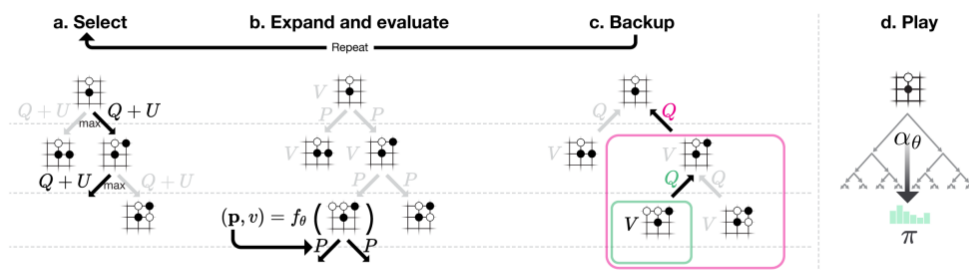
v , 神经网络预测的动作价值或赢方。标量。

z , 棋局结束状态的赢方。

$\boldsymbol{\pi}$, 前一时刻蒙特卡洛树搜索计算的动作概率分布。

c , L2 正则项权重。

AlphaGo Zero 中的蒙特卡洛树搜索框架。



框架图，Select 步骤，基于最大动作价值 Q 及上置信边界 U 选择动作；Expand 及 Evaluate 步骤，基于神经网络 f_θ 预测动作分布及价值；Backup 步骤，更新动作轨迹中的动作价值 Q ；Play 步骤，搜索结束，得到搜索概率 π 。

蒙特卡洛树搜索，实质在于用树形结构定义全部状态动作轨迹，用状态动作策略模型预测动作，用状态动作轨迹结束时的奖励更新轨迹中的状态、动作价值。

算法基本步骤，可分作选择(Selection)、扩展(Expansion)、仿真(Simulation)及反向传播(Backpropagation)。

第一步，选择。从起始节点开始，基于上置信边界(UCB, Upper Confidence Bound)算法等选择待探索的状态子节点，直到叶结点。

注，上置信边界算法实质，基于当前状态节点的上级节点视角，当前状态节点相对别的子节点具备更高的平均奖励，同时当前状态节点访问或探索次数相对少，则选当前状态节点进行探索。

UCB1 方程。

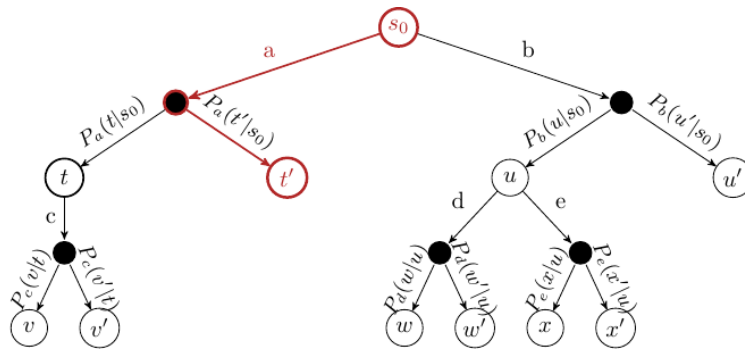
$$UCB1 = \text{average reward} + C \times \sqrt{\frac{\ln N}{n}}$$

上置信树(UCT, Upper Confidence Tree)，UCB1 算法的变体。

$$\operatorname{argmax}_{a \in A(s)} Q(s, a) + 2C_p \sqrt{\frac{2 \ln N(s)}{N(s, a)}}$$

注，选择过程传统多臂老虎机算法。

选择过程示例。

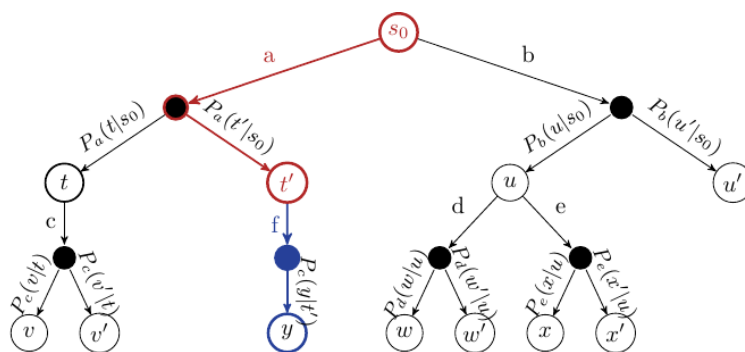


注，蒙特卡洛搜索树中，状态 $s_0 \rightarrow t$ 的概率 $P_a(t|s_0)$ ，状态 $s_0 \rightarrow t'$ 的概率 $P_a(t'|s_0)$ ，可表示为状态 s_0 执行动作 a 后，分别迁移到状态 t 或状态 t' 的概率， $P(t|s_0, a)$ 或 $P(t'|s_0, a)$ 。状态 s_0 执行动作 a 时，隐含着策略 $\pi_a(a|s_0)$ 。

第二步，扩展。从待探索的叶结点开始，选一动作，以添加新状态子节点(New Child Nodes)。

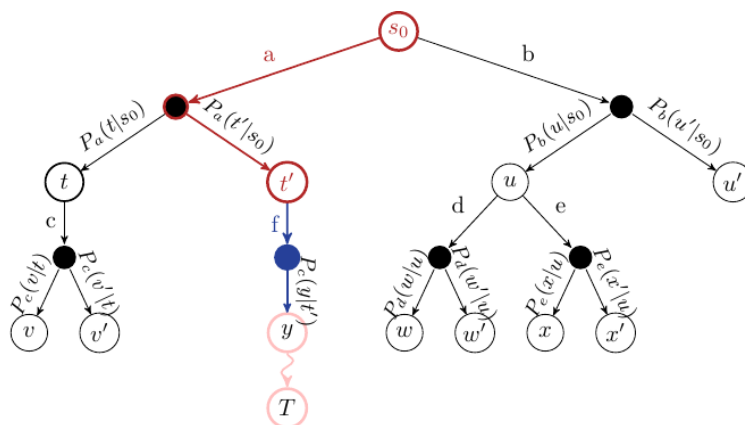
注，基于当前状态，选一动作，可迁移到不同的状态。

扩展过程示例。



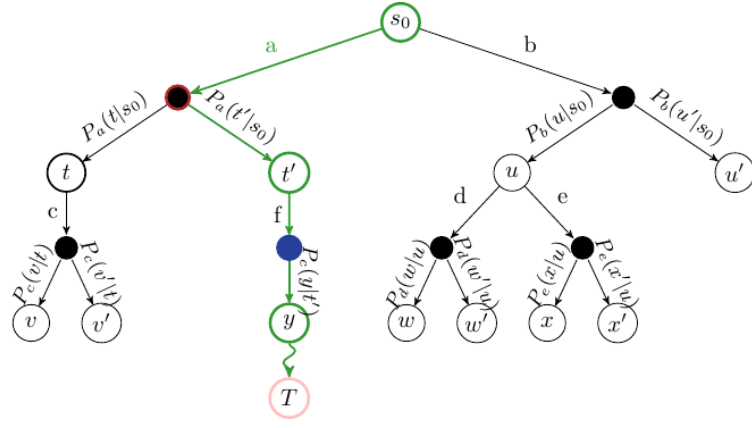
第三步，仿真。选一新状态子节点，利用状态动作策略模型预测动作，迁移状态，直到终止状态(Terminating State)。

仿真过程示例。



第四步，反向传播。基于终止状态的奖励，从终止状态开始沿着状态动作轨迹反方向，更新轨迹中状态的价值 $V(s)$ 。

反向传播过程示例。



机器人控制篇

传统机械臂基于运动学(Kinematics)及动力学(Dynamics)计算关节力矩及终端执行器位置、速度等，适用于有限特定场景及任务。

传统机器人基于倒立摆、整体模型等，利用线性二次调整(LQR, Linear Quadratic Regulator)、模型预测控制(MPC, Model Preview Control)及整体控制(WBC, Whole Body Control)等优化计算、控制机器人运动。

通用机器人，基于神经网络拟合人类思维函数。不同于传统机器人基于物理力学、动能分析定义变换函数。

注，传统机械臂、机器人、SLAM 等与神经网络方法表象似都是基于目标函数、优化算法进行计算，实质迥异。传统方法以机器人**本体物理模型为中心**，简化模型的同时，简化状态，基于任务优化**计算物理模型控制量**，无法主动适用多样化环境；神经网络方法以**状态变换数学模型为中心**，不限制状态，基于大规模数据**优化拟合变换函数**，预测控制量，可具备主动及通用性。

基于机器人运动控制过程来看，神经网络方法是基于整体运动变换、更本质的机制。跳离对机器人本体模型的定义，模仿人类思维，直接计算状态变换函数。

当前，通用机器人基于视觉语言动作模型进行运动控制，模型有 RT-1、RT-2、Gemini-Robotic、Phi、SmoIVLA、GR00T 等。

RT-1，基于 FiLM EfficientNet 及 标准 Transformer Decoder-only 的视觉语言动作模型。基于 Transformer 框架 不过不是 Diffusion Transformer。

RT-2，基于 PaLI-X 或 PaLM-E 视觉语言模型，直接生成动作的视觉语言动作模型(RT-2-PaLI-X and RT-2-PaLM-E)。

注，当前通用机器人的控制挑战应在于步态平衡及灵活手控制。

第一章. 机械臂

机械臂一般基于基座、连杆(Link)、关节(Joint)及终端执行器(End Effector)组成。关节可以分为 Linear、Rotational 等类型。不同类型的机械臂用不同类型的机械关节(Mechanical Joints)。在工业制造中,机械臂用于将终端执行器移动到目标位置进行相应的操作。这一移动过程主要涉及路径与轨迹规划、运动学(Kinematics)、动力学(Dynamics)与控制。

基于路径规划算法确定终端执行器从原始位置移动到目标位置的路径,形成 Waypoints 序列;基于时间约束,利用多项式、样条函数等轨迹函数实现对整个路径的轨迹优化;基于 Waypoints 进行逆向运动学计算,获得相邻 Waypoints 机械臂各关节应该调整的角度/偏移等信息;将相邻 Waypoints 的时间划分为多个时刻,基于轨迹函数计算每一时刻,机械臂各关节的角度/角加速度等信息;基于逆向动力学计算每一时刻机械臂各关节应该设置的力/力矩等信息;用计算得到的力/力矩等信息驱动电机使机械臂移动;基于反馈控制控制电机使机械臂运动轨迹尽可能符合期望轨迹;测量机械臂实际运行轨迹与期望轨迹的区别,进行调整。

动力学的必要性在于,尽管对于 Pick and Place 等简单应用场景,机械臂可以基于路径规划与运动学控制机械臂实现终端执行器从原始位置到目标点的运动,但是在大多数情况下,这不仅要求在路径规划的每个 Waypoint 机械臂需要进行逆向运动学计算,导致计算量较大,而且容易出现卡顿,能量消耗大,对于某些曲线轨迹出现难以控制的问题,更重要的是计算获得的关节位置可能不符合关节空间,同时在大功率情况下,不可控的机械臂会带来很大的危险。

基于轨迹规划与动力学则可以在符合时间约束、关节约束、能耗限制等约束条件的情况下,控制机械臂关节依照轨迹拟合函数更平滑、更准确地运动,并且可以控制机械臂与外界交互时的作用力。

一、 机械臂类型

串联机械臂。比方,多关节型机械臂(Articulated), 扭关节连接基座,别的关节为旋转关节。

笛卡尔机械臂(Cartesian), 连杆之间线性关节连接,终端执行器可以用旋转关节。

圆柱形机械臂(Cylindrical), 旋转关节连接基座,连杆之间线性关节连接。

极坐标机械臂(Polar), 扭关节连接基座,连杆之间用旋转关节及线性关节。

并联机械臂。

一个基座，多个机械臂；连杆之间用旋转关节及线性关节连接。

二、 运动学

运动学(kinematics)可分为正向运动学，逆向运动学。

正向运动学，基于关节的角度、角速度等信息计算终端执行器的位置、速度等信息；逆向运动学，基于终端执行器位置计算各关节的角度信息。

正逆向运动学的实质，在于刚体假设下，关节与终端执行器是联动的。关节角度与执行器位置的变换关系是几何直观的；关节角速度在刚体的约束下，与终端执行器的速度亦存在变换。

假定，机械臂是单连杆，长度 l ，角速度 a 。连杆末端的运动，可视作基座坐标系下，原点或圆心在基座，半径是 l ，角速度是 a 的圆周运动。基于圆周运动切线速度角速度方程， $v = r * \omega$ ，可知连杆末端线速度。

当单连杆末端通过关节再连接第二个连杆时。在单连杆末端坐标系下，第二个连杆末端的速度计算方式相似。同时，基于坐标系变换，可知，在基座坐标系下，第二个连杆末端的速度。

因此，多连杆关节角速度与终端执行器速度关系，可视作坐标系变换与圆周运动方程的叠加效应。

1. 正向运动学

常用计算方法，DH 法。

在相邻关节 z 轴不平行的前提下，可基于 DH 法快速计算机械臂基座到终端执行器位置的变换矩阵。DH 法的本质是获取相邻关节坐标系的 x 轴、 z 轴的旋转、平移信息，然后基于关节坐标变换矩阵的乘积关系计算终端执行器的旋转与位置信息。

DH 参数: d , X 轴平移; θ , X 轴的旋转; $a(r)$, Z 轴的平移; α , Z 轴的旋转。

计算得到每个关节的 DH 参数后,可计算之前关节到该关节的变换矩阵,将所有关节的变换矩阵相乘,得到的矩阵为终端执行器位置及旋转信息。得到的 T 矩阵中, T 向量为终端执行器的位置信息, R 向量为旋转角度信息。

$$T = \left[\begin{array}{ccc|c} R & & & T \\ \hline 0 & 0 & 0 & 1 \end{array} \right] = \left[\begin{array}{ccc|c} \cos \theta & -\sin \theta \cos \alpha & \sin \theta \sin \alpha & r \cos \theta \\ \sin \theta & \cos \theta \cos \alpha & -\cos \theta \sin \alpha & r \sin \theta \\ 0 & \sin \alpha & \cos \alpha & d \\ \hline 0 & 0 & 0 & 1 \end{array} \right]$$

2. 逆向运动学

逆向运动学的传统计算方法分为解析法(Analytical)与数值法(Numerical)。

解析法有代数法、几何法等;传统数值法有迭代法、优化法等。迭代法则有雅可比求逆法、牛顿迭代法等。解析法运算更快,数值法更适合高自由度机械臂计算。

代数法(Algebraic)

对于二维平面,三个旋转关节的机械臂的逆向运动学计算,就是基于正向运动学基座到目标点的矩阵变换关系公式形成目标点位置 x, y 的方程组;解该方程组可知前两个关节的旋转角度,目标点关节即第三个关节旋转角度等于基准点与目标点角度减去前两个关节旋转角度的和。

几何法(Geometrical)

基于机械臂的俯视图、侧视图的三角形,利用三角形定理计算关节角度。

雅可比(Jacobian)求逆法

直观逻辑，假定基于刚体联动，终端执行器速度与各关节速度变化存在着一定的比例关系，则机械臂各关节速度的加权等于终端执行器速度。关节速度与执行器速度之间的映射关系可视为多变量方程组，可基于函数的雅可比矩阵定义权重系数。通过执行器速度向量与雅可比矩阵逆可计算关节速度向量。

不同于代数法基于各关节坐标系的旋转、平移的变换进行解方程组的方法，雅可比求逆法基于雅可比矩阵计算关节角度信息。雅可比矩阵描述的是机械臂各关节速度与终端执行器速度之间的关系。

$$\dot{X} = J\dot{q}$$

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ \dot{\alpha} \\ \dot{\beta} \\ \dot{\gamma} \end{bmatrix}_{6 \times 1} = \begin{bmatrix} J_{11} & J_{12} & \cdot & \cdot & \cdot & J_{1n} \\ J_{21} & J_{22} & \cdot & \cdot & \cdot & J_{2n} \\ \vdots & \ddots & \vdots & \vdots & \vdots & \vdots \\ \cdot & \cdots & \cdot & \cdot & \cdot & \cdot \\ J_{61} & \cdots & \cdot & \cdot & \cdot & J_{6n} \end{bmatrix}_{6 \times n} * \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \dot{q}_3 \\ \vdots \\ \vdots \\ \dot{q}_n \end{bmatrix}_{n \times 1}$$

方程，左侧，终端执行器线速度及角速度向量；右侧，J，雅可比权重矩阵；q 向量，各关节速度。

雅可比矩阵的列数目等于机械臂关节数目，每一列表示每一个关节的速度对机械臂的终端执行器的速度的影响。雅可比矩阵的前三行与机械臂终端执行器的线速度相关，后三行与终端执行器的角速度相关。

雅可比矩阵的角速度成员形式逻辑意义是各关节轴相对于机械臂基座坐标系的旋转变换，实际可由正向运动学中机械臂基座到各关节及终端执行器的变换矩阵的前三列的某一系列成员获得。

机械臂从原始位置移动到目标位置计算过程。

1、通过电机编码器获取机械臂各关节角度，基于正向运动学计算 Jacobian 矩阵；

2、以目前位置到目标位置的距离的百分比作为趋近距离，以 Jacobian 矩阵的逆乘以该趋近距离，计算得到各关节应设置的角度；

3、依据计算获得的关节角度驱动机械臂关节电机；

4、获取机械臂终端执行器位置，与目标位置不一致，则循环步骤 1-3；

注: 为确保机械臂各关节在有效范围移动，比方，Jacobian 矩阵非方形矩阵或是行列式等于零的奇异矩阵，不可逆。一般用 Jacobian Pseudo Inverse 矩阵替代 Jacobian 逆矩阵。

三、 动力学

动力学的目的是建立刚体的力与运动的数学模型。机械臂的动力学是建立机械臂的力/力矩与加速度/角速度的数学模型。动力学可分为正向动力学与逆向动力学。

正向动力学，基于关节的力(线性移动)/力矩(旋转)预测连杆的运动；逆向动力学，基于物体的运动及惯性特性计算力/力矩。

正逆向动力学的实质，与运动学相似，在于刚体假设下，关节与终端执行器是联动的。

假定，机械臂是单连杆，长度 l ，角速度 a 。连杆末端的运动，可视作基座坐标系下，原点或圆心在基座，半径是 l ，角速度是 a 的圆周运动。基于圆周运动向心力(Centripetal Force)方程，可计算关节力与连杆末端向心加速度(Centripetal Acceleration)关系。

$$F = m * a = m * \frac{v^2}{r} = m * \omega^2 * r$$

F ，向心力。

a ，向心加速度。

v ，切线速度(Tangential Velocity)。

ω ，角速度。

r , 半径长。

当多连杆时，叠加关节坐标系变换，可计算基座坐标系下，终端执行器速度与力的关系。

动力学的传统计算方法可基于牛顿-欧拉方法、欧拉-拉格朗日方法等，近年则出现了基于深度强化学习的方法。

牛顿-欧拉方法从牛顿第二定律发展而来。牛顿第二定律基于粒子动力学，分析的是力与加速度的关系。牛顿-欧拉方法则建立了适用于由粒子所组成的刚体的动力学的力矩与角加速度的模型。

欧拉-拉格朗日方法更多是从刚体能量的角度分析动力关系。欧拉-拉格朗日方法不仅建立了力矩与角加速度的模型，而且使力矩的计算更简单。

1. 牛顿-欧拉法(Newton-Euler)

基于牛顿第二运动定律。

$$\Sigma F_x = ma_x$$

$$\Sigma F_y = ma_y$$

$$\Sigma F_z = ma_z$$

$$\Sigma M = I\alpha$$

M 是力矩， I 是转动惯量， α 是角加速度。

2. 欧拉-拉格朗日法(Euler-Lagrange)

系统运动分析方法，牛顿力学、哈密顿力学、拉格朗日力学。

拉格朗日力学核心在于，拉格朗日函数(Lagrange Function)。

比方， $L = T - U$ 。

拉格朗日函数(Lagrangian) L ，等于动能 T 与势能 U 的差。

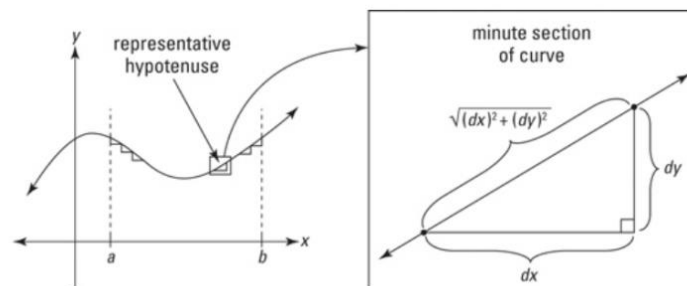
该形式未明确定义物理意义，更多是基于最小作用量原理(PLA , Principle Of Least Action)推得的。

最小作用量定义。

$$S = \int L dt$$

S ，作用量(Action)。

比方，计算两点之间最短距离 S 。



基于局部极小值毕氏定理，定义 S 计算方程。

$$S = \sum_{i=0}^n \sqrt{dx_i^2 + dy_i^2} = \sum_{i=0}^n \sqrt{1 + \left(\frac{dy_i}{dx_i}\right)^2} dx_i$$

取极限积分定义。

$$S = \int_a^b \sqrt{1 + (y')^2} dx$$

作用量 S 的定义形式。

$$S = \int_a^b L(x, y(x), y') dx$$

最小作用量相应的欧拉-拉格朗日方程。

$$\frac{\partial L}{\partial y} = \frac{d}{dx} \frac{\partial L}{\partial y'}$$

当该方程成立时，作用量存在最小值。

确定系统运动方程的步骤。

一、确定拉格朗日函数 L 。

二、基于欧拉-拉格朗日方程推算运动方程。

当系统在 n 维空间中时，基于广义坐标的拉格朗日函数定义。

$$L(q_1, \dots, q_n, \dot{q}_1, \dots, \dot{q}_n)$$

广义的欧拉-拉格朗日方程。

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}_i} \right) - \frac{\partial L}{\partial q_i} = Q_i = \tau_i$$

Q_i ，相对于广义坐标 q_i 的广义力向量元素。

注，当拉格朗日函数 L 是坐标 q 高阶导的函数时，比方，二阶导的函数， $L = L(q, \dot{q}, \ddot{q}, t)$ ，可推算相应的二阶欧拉-拉格朗日方程。

基于广义坐标 q 的广义力 F 的定义。

$$F = \frac{\partial L}{\partial q}$$

欧拉-拉格朗日法实质，基于拉格朗日方程推算机器人运动方程。

运动方程推算过程。

广义拉格朗日方程。

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}_i} \right) - \frac{\partial L}{\partial q_i} = Q_i = \tau_i$$

基于拉格朗日函数进行变量替代。

$$\mathcal{L}(q_i, \dot{q}_i) = T - U$$

计算机器人动能 T 及势能 U 。

计算第 i 个广义坐标对应的拉格朗日方程。

基于全部广义坐标的拉格朗日方程组，转换成矩阵多项式方程。

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \boldsymbol{\tau}_g(\mathbf{q}) + \mathbf{B}\mathbf{u}$$

多项式方程，左侧，第一项，惯性力， $\mathbf{M}$ ，惯性或质量矩阵；第二项，科氏力(Coriolis Force)， $\mathbf{C}$ ，科氏力矩阵；右侧，第一项，重力；第二项，广义力， $\mathbf{B}$ ，控制量 $\mathbf{u}$ 到广义力的映射矩阵。

注，科氏力，旋转坐标系下的偏转力。垂直于旋转轴与对象运动速度方向的平面。

计算方程， $\mathbf{F}_c = -2\mathbf{m}(\boldsymbol{\Omega} \times \mathbf{v})$ 。

$\boldsymbol{\Omega}$ ，旋转速度。

$\mathbf{v}$ ，线速度。

当存在外力时，可基于瑞利耗散函数(Rayleigh's Dissipation Function)扩展拉格朗日方程。

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}_j} \right) - \frac{\partial L}{\partial q_j} = Q_j - \frac{\partial R}{\partial \dot{q}_j}$$

$$R = \frac{1}{2} \sum_j b_j \dot{q}_j^2$$

相应的运动方程。

$$M\ddot{q} + C\dot{q} + h - J_e^T F_e = S^T \tau$$

h , 重力。

F_e , 外力。

J_e , 外力应用点的雅可比矩阵。

S , 选择或映射矩阵。

τ , 关节力矩或控制量。

3. 正向动力学

两连杆旋转机械臂，基于基座原点与第 2 个关节的力与力矩等式计算关节线加速度、角加速度。

4. 逆向动力学

两连杆旋转机械臂。

- 1、基于正向运动学可知机械臂每个连杆的线速度、角速度对应的 Jacobian 矩阵。
- 2、基于动能等式计算所有连杆的动能。
- 3、基于获取的矩阵项 H 的元素项计算 h 。
- 4、基于 Jacobian 矩阵列项计算重力项 G 。

四、 路径规划

基于 PRM(Probabilistic Road Map)、RRT(Rapidly Exploring Random Tree)采样等路径规划算法确定机械臂终端执行器的运动路径。终端执行器的运动路径由一系列 Waypoints 组成。

五、 轨迹规划

轨迹规划是在符合机械臂移动时间限制、关节运动范围限制等条件下，对规划的机械臂路径进行优化，实现更平滑、更准确的轨迹。传统的轨迹规划可基于直线、抛物线、多项式曲线、样条曲线等方法来拟合位置轨迹，基于球面线性插值(Slerp, Spherical Linear Interpolation)等方法来拟合终端执行器旋转轨迹。

基于终端执行器的笛卡儿空间进行轨迹规划，直观，但是耗费能量，可能出现 Singularity 点，并需大计算量，因为每一个路径点都需要进行逆运动学计算。基于各关节的关节空间进行轨迹规划，便于基于关节的约束实现机械臂的移动，省能量。

第二章. 传统机器人

传统机器人运动控制，核心步骤，1、确立机器人平衡准则；2、确立机器人刚体模型；3、确定机器人运动控制框架。

机器人平衡准则，使机器人保持平衡(Dynamic Stability)的约束条件。比方，零力矩点(ZMP, Zero Moment Point)、捕获点(CP, Capture Point)、质心散量(DCM, Divergent Component of CoM Motion)、环境连接点扭转力和(CWS, Contact Wrench Sum)等。

机器人刚体模型，基本可视作机械臂的扩展形式。比方，线性倒立摆模型(LIPM, Linear Inverted Pendulum Model)、机器人整体动力学模型(Whole Body Model)等。倒立摆等价于基座在地面的可旋转单连杆机械臂；人形机器人整体模型等价于基座在机器人腰部的树状多连杆机械臂。

机器人运动控制框架，基本基于反馈回路。比方，指定任务运动轨迹，通过运动方程估测实现该运动轨迹的关节控制量；观测机器人关节输出的控制量，反馈修改运动估测。框架主要包括基于线性二次调整器的预览控制(LQR Preview Control)、模型预测控制(MPC, Model Predictive Control)、整体控制(WBC, Whole Body Control)或混合零动态(HZD, Hybrid Zero Dynamics)等。

运动估测的代价函数，可基于机器人质心与平衡点约束关系；或机器人端与连接点力矩约束关系。

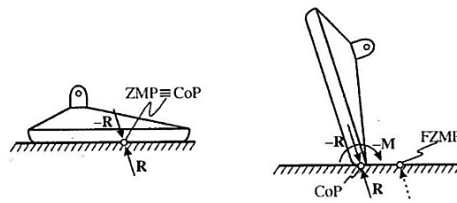
一、 平衡准则

1. 零力矩点

机器人简化作质量集中于顶部的连杆。当机器人重力及惯性力(Fictitious Force)，或总的力关于该连杆的虚拟着地点的力矩等于零时，此时该点称零力矩点。

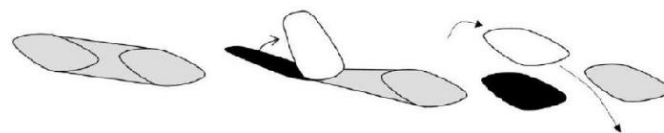
零力矩点实质，作用于机器人水平向、垂直向的力，使机器人绕着地点旋转的叉乘力矩等于零，机器人不具备绕着地点旋转的额外力。

注，零力矩点是虚拟着地点，可能同或不同于压力中心点(CoP , Center Of Pressure)实着地点。



当零力矩点在机器人双足形成的支撑多边形区域(Supporting Polygon)时，机器人可保持平衡。

支撑多边形，可基于机器人双足着地情况分作三类。



多边形区域图(灰色)，双足全着地；单足脚尖着地、单足全着地；单足着地。

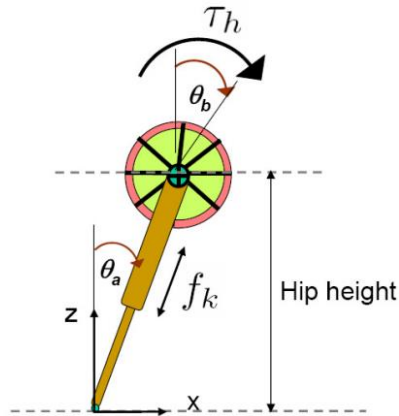
当机器人慢速行走、忽略惯性力的情况下，机器人质心(Center Of Mass)在支撑多边形区域的投影点等同于零力矩点。

2. 捕获点(Capture Point)

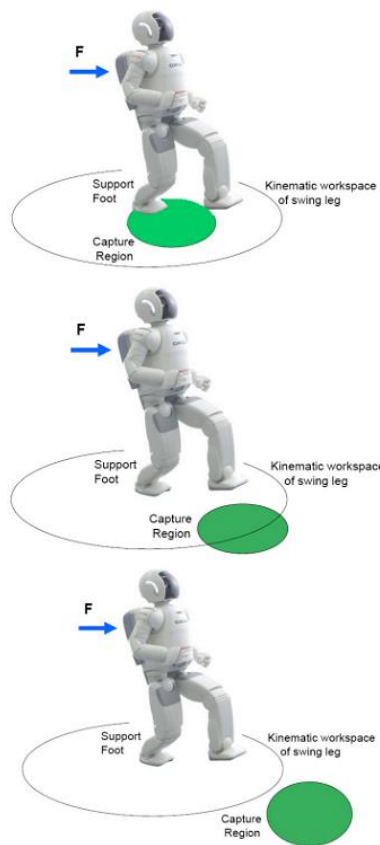
机器人遇到外力时，通过迈步到着地点，以保持平衡、静止。该点称捕获点。

实质在于，使机器人双足形成的支撑多边形区域辖括捕获点，以保持平衡。

当机器人视作顶部是飞轮、质心在顶部的倒立摆时，可存在多个捕获点。捕获点集合称捕获区(Capture Region)。



机器人平衡情况，可基于捕获点(区域)与支撑多边形区域情况分作三类。



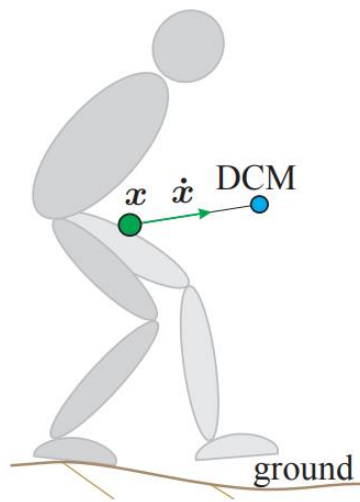
机器人平衡图，顶部，捕获区域落在多边形区域内，无需迈步可平衡；中部，捕获区域落在多边形区域外，机器人需迈步使多边形区域辖括或交于捕获区域，以保持平衡；底部，捕获区域落在多边形区域外，机器人迈步无法使多边形区域辖括或交于捕获区域，无法保持平衡。

当捕获区域落在或交于支撑多边形区域，同时，压力中心点(CoP)落在捕获区域或捕获点，机器人可保持平衡。

3. 质心运动散量

质心运动散量(DCM)，是捕获点在三维空间的扩展。

质心散量，定义作质心位置与质心速度的线性组合。



$$\xi = x + b \dot{x}$$

$$b = \sqrt{\frac{\Delta z}{g}}$$

b ，时间常量。

Δz ，质心平均高度。

g ，重力常量。

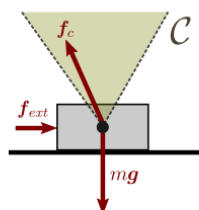
当机器人质心收敛于(Converges To)质心运动散量时，可保持平衡。

4. 扭力和

扭力和(CWS, Contact Wrench Sum), 在机器人与环境连接位置, 支撑机器人的作用力的和。

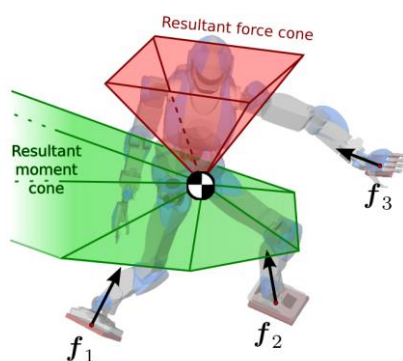
扭力锥体(CWC, Contact Wrench Cone), 在机器人与环境连接位置, 支撑机器人的作用力(力矩)在向量坐标空间所可能形成的锥体范围。

扭力锥体示例。



图示, 地面上一个质量方块, 只要作用于方块的与重力及外力的合力大小相等的扭力(Contact Wrench)在库仑摩擦锥体($\mathcal{C}$)范围内, 则这个方块保持固定; 反之, 方块移动。

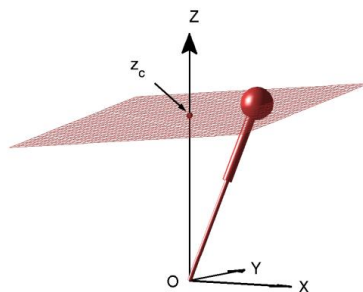
当机器人重力及外力反的向力在扭力锥体内时, 可保持平衡。



二、 线性倒立摆模型(LIPM, Linear Inverted Pendulum Model)

机器人线性倒立摆模型，假定机器人质量集中于质心，并且在腿部移动时，质心的高度始终符合约束平面，机器人的移动就像是移动中的倒立的钟摆。

1. 模型图示例



2. 零力矩方程

基于力矩方程， $\tau = l * F * \sin\omega$ ；重力、惯性力及倒立摆在相应轴的投影，可得线性倒立摆绕 x 轴、y 轴力矩方程。

$$\ddot{y} = \frac{g}{z_c}y - \frac{1}{mz_c}\tau_x$$
$$\ddot{x} = \frac{g}{z_c}x + \frac{1}{mz_c}\tau_y$$

方程逻辑，重力使倒立摆向地面旋转，水平力作用于倒立摆产生加速度，形成相反方向的虚拟惯性力，使倒立摆向直立方向旋转，以保持平衡。

基于零力矩点定义，惯性力作用于倒立摆的力矩与重力力矩相等，可得零力矩点位置方程。

$$\begin{aligned} p_x &= -\frac{\tau_y}{mg} \\ p_y &= \frac{\tau_x}{mg} \end{aligned}$$

方程组联立，可得零力矩点位置与质心位置及加速度方程。

$$\begin{aligned} p_x &= x - \frac{z_c}{g} \ddot{x} \\ p_y &= y - \frac{z_c}{g} \ddot{y} \end{aligned}$$

通过线性倒立摆模型(LIPM)可以建立零力矩点(ZMP)与质心(CoM)的关系，并使基于零力矩点轨迹计算获得的质心(CoM)轨迹符合零力矩点稳定性判断准则。

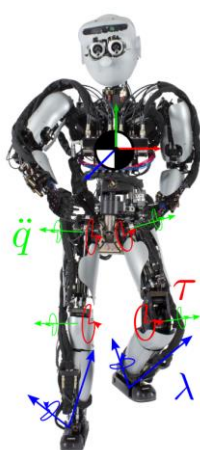
线性倒立摆模型(LIPM, Linear Inverted Pendulum Model)，主要通过忽略肢体质量、集中质量于质心、降低机器人状态维度来简化动力学计算，适合平整地面的移动，但是存在要求机器人质心(CoM)轨迹在同一平面、低速移动、躯干无旋转等限制。

三、 整体模型(Whole Body Model)

机器人视作一个树形结构的机械臂，在腰部是机械臂的基座，这样机器人可以看作是浮动基座机器人。由于仿人机器人质心(CoM)存在 6 个自由度(线性/角度)是没有电机直接控制实现的，因此仿人机器人又是欠驱动的。

1. 运动方程

整体模型可以认为是分析这个浮动的机械臂整体各部分及与外部环境的作用力的模型。基于机器人连杆、关节、质心及与环境作用部分的(线/角)速度/力矩可以建立机器人整体模型的动力学等式。有时为了简化复杂性，只分析机器人质心与腿足的动力学。



基于机械臂动力学欧拉-拉格朗日方程计算力矩的等式，可以将机器人力矩分为状态(关节)空间的力矩及任务(扭力)空间的力矩。

$$M\ddot{q} + h\dot{q} + g = S^T\tau + J_c\lambda$$

M ，惯性矩阵。

h ，科氏力矩阵。

g ，重力向量。

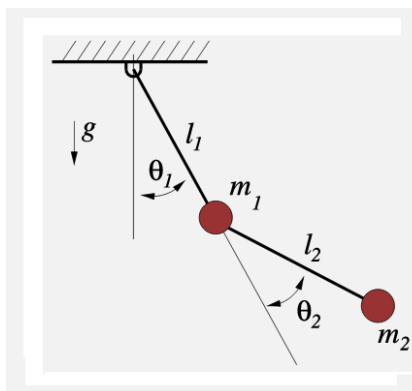
τ ，关节力矩向量。

λ ，外部力。

机器人单只脚连接地面时，添加 6 个约束；双脚都连接地面，则会形成过驱动，使得计算逆向动力学方程时，方程相对于未知量存在冗余。因此，浮动基座的动力学计算是随着机器人与地面的接触情况在变化的。

2. 方程推算

基于两连杆双摆推算运动方程。



基于连杆长度及 sin 三角函数计算，可得质心 m1、m2 位置方程。

$$\mathbf{p}_1 = l_1 \begin{bmatrix} s_1 \\ -c_1 \end{bmatrix}, \mathbf{p}_2 = \mathbf{p}_1 + l_2 \begin{bmatrix} s_{1+2} \\ -c_{1+2} \end{bmatrix}$$

s_1 ，质心 m_1 的 $\sin\theta_1$ 。

s_{1+2} ，质心 m_2 的 $\sin(\theta_1 + \theta_2)$ 。

c_1 ，质心 m_1 的 $\cos\theta_1$ 。

c_{1+2} ，质心 m_2 的 $\cos(\theta_1 + \theta_2)$ 。

基于质心位置方程导数计算，可得质心速度方程。

$$\dot{\mathbf{p}}_1 = l_1 \dot{q}_1 \begin{bmatrix} c_1 \\ s_1 \end{bmatrix}, \dot{\mathbf{p}}_2 = \dot{\mathbf{p}}_1 + l_2 (\dot{q}_1 + \dot{q}_2) \begin{bmatrix} c_{1+2} \\ s_{1+2} \end{bmatrix}$$

基于位置及速度方程，可得动能、势能(Kinetic And Potential Energy)方程。

$$\begin{aligned}
T &= \frac{1}{2} m_1 \dot{\mathbf{P}}_1^T \dot{\mathbf{P}}_1 + \frac{1}{2} m_2 \dot{\mathbf{P}}_2^T \dot{\mathbf{P}}_2 \\
&= \frac{1}{2} (m_1 + m_2) l_1^2 \dot{q}_1^2 + \frac{1}{2} m_2 l_2^2 (\dot{q}_1 + \dot{q}_2)^2 + m_2 l_1 l_2 \dot{q}_1 (\dot{q}_1 + \dot{q}_2) c_2 \\
U &= m_1 g y_1 + m_2 g y_2 = -(m_1 + m_2) g l_1 c_1 - m_2 g l_2 c_{1+2}
\end{aligned}$$

基于动能、势能方程分别计算关于广义坐标量的偏导。

$$\begin{aligned}
\frac{\partial T}{\partial q_1} &= 0, \\
\frac{\partial U}{\partial q_1} &= (m_1 + m_2) g l_1 s_1 + m_2 g l_2 s_{1+2}, \\
\frac{\partial T}{\partial q_2} &= -m_2 l_1 l_2 \dot{q}_1 (\dot{q}_1 + \dot{q}_2) s_2, \\
\frac{\partial U}{\partial q_2} &= m_2 g l_2 s_{1+2}, \\
\frac{\partial T}{\partial \dot{q}_1} &= (m_1 + m_2) l_1^2 \dot{q}_1 + m_2 l_2^2 (\dot{q}_1 + \dot{q}_2) + m_2 l_1 l_2 (2\dot{q}_1 + \dot{q}_2) c_2, \\
\frac{\partial T}{\partial \dot{q}_2} &= m_2 l_2^2 (\dot{q}_1 + \dot{q}_2) + m_2 l_1 l_2 \dot{q}_1 c_2, \\
\frac{d}{dt} \frac{\partial T}{\partial \dot{q}_1} &= (m_1 + m_2) l_1^2 \ddot{q}_1 + m_2 l_2^2 (\ddot{q}_1 + \ddot{q}_2) + m_2 l_1 l_2 (2\ddot{q}_1 + \ddot{q}_2) c_2 - m_2 l_1 l_2 (2\dot{q}_1 + \dot{q}_2) \dot{q}_2 s_2, \\
\frac{d}{dt} \frac{\partial T}{\partial \dot{q}_2} &= m_2 l_2^2 (\ddot{q}_1 + \ddot{q}_2) + m_2 l_1 l_2 \ddot{q}_1 c_2 - m_2 l_1 l_2 \dot{q}_1 \dot{q}_2 s_2,
\end{aligned}$$

偏导代到欧拉-拉格朗日方程，可得，

$$\begin{aligned}
&(m_1 + m_2) l_1^2 \ddot{q}_1 + m_2 l_2^2 (\ddot{q}_1 + \ddot{q}_2) + m_2 l_1 l_2 (2\ddot{q}_1 + \ddot{q}_2) c_2 \\
&\quad - m_2 l_1 l_2 (2\dot{q}_1 + \dot{q}_2) \dot{q}_2 s_2 + (m_1 + m_2) l_1 g s_1 + m_2 g l_2 s_{1+2} = \tau_1 \\
&m_2 l_2^2 (\ddot{q}_1 + \ddot{q}_2) + m_2 l_1 l_2 \ddot{q}_1 c_2 + m_2 l_1 l_2 \dot{q}_1^2 s_2 + m_2 g l_2 s_{1+2} = \tau_2
\end{aligned}$$

整理可得运动方程。

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \boldsymbol{\tau}_g(\mathbf{q}) + \mathbf{B}\mathbf{u}$$

惯性矩阵、科氏力矩阵、关节力矩及映射矩阵定义。

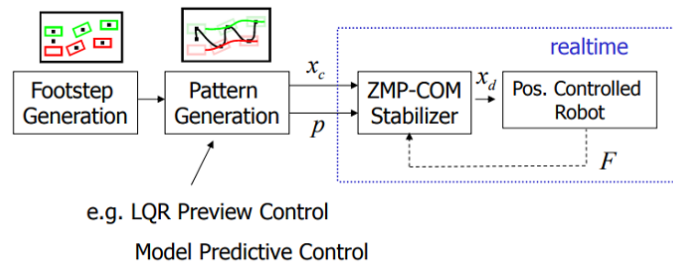
$$\begin{aligned} \mathbf{M}(\mathbf{q}) &= \begin{bmatrix} (m_1 + m_2)l_1^2 + m_2l_2^2 + 2m_2l_1l_2c_2 & m_2l_2^2 + m_2l_1l_2c_2 \\ m_2l_2^2 + m_2l_1l_2c_2 & m_2l_2^2 \end{bmatrix} \\ \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) &= \begin{bmatrix} 0 & -m_2l_1l_2(2\dot{q}_1 + \dot{q}_2)s_2 \\ \frac{1}{2}m_2l_1l_2(2\dot{q}_1 + \dot{q}_2)s_2 & -\frac{1}{2}m_2l_1l_2\dot{q}_1s_2 \end{bmatrix} \\ \tau_g(\mathbf{q}) &= -g \begin{bmatrix} (m_1 + m_2)l_1s_1 + m_2l_2s_{1+2} \\ m_2l_2s_{1+2} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \end{aligned}$$

为实现多个不同的任务，力矩可通过任务零空间(Null-Space)投影方式、加权方式或层级(Hierarchical)方式实现对不同目标任务力矩进行优先级控制。

机器人整体模型(Whole Body Model)，主要通过分析机器人各连杆、关节以及与环境的动力学，来实现在不平整地面上的移动、通过上肢摆动实现平衡、行走的同时实现多个任务等需求。

四、 运动控制

基于线性二次调整器(LQR, Linear Quadratic Regulator)的预览控制(Preview Control)是基于固定时间窗，通过黎卡提方程(Riccati Equation)进行优化计算的控制框架；模型预测控制(Model Prediction Control, MPC)是基于回退时域控制(Receding Horizon Control)适合线性/非线性模型及多约束，可通过二次规划(Quadratic Programming)进行优化计算、实时计算非周期步态的控制框架；混合零动态(HZD, Hybrid Zero Dynamics)是基于传递函数判断机器人周期步态稳定性的适用于离线计算的控制框架。



1. 比例积分控制器(PID, Proportional Integral Derivative)

基于预期量与实际测量量的误差，利用比例器、积分器及微分器调整系统控制量使系统状态趋近预期目标。

2. 线性二次调整器(LQR, Linear Quadratic Regulator)

线性二次调整器，Linear 指线性时不变系统 + Quadratic 指代价函数(cost-to-go function)是二次凸函数(即存在最优解)，即用二次凸代价函数来调整线性时不变系统。

线性时不变系统。

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$$

线性二次调整器的代价函数，实质是对目标状态量 $\mathbf{x}$ 与输入控制量 $\mathbf{u}$ 进行加权，找到两者的平衡点，以便使状态量(偏离目标的变化)最小，同时控制量不过度。

注，代价函数中，状态量是隐含的误差量，控制量相当于正则项(Regularization)。

比方，恒温器(Thermostat)调温时，使温度与设定温度最接近，同时，加热器的输入功率不过大。移动小车上的倒立摆平衡时，应使倒立摆与垂直向的偏移角及角度变化速度尽量小，同时，小车速度不过快。

线性二次调整器基于无限时窗(Infinite Time Horizon)的代价函数。

$$J = \int_0^{\infty} (\mathbf{x}(t)^T \mathbf{Q} \mathbf{x}(t) + \mathbf{u}(t)^T \mathbf{R} \mathbf{u}(t)) dt$$

$\mathbf{Q}$, 状态向量 $\mathbf{x}$ 的权重对角矩阵。

R ，控制向量 u 的权重对角矩阵。

假定, x 及 u 是单变量，则 Q 、 R 对应标量权重值。

代价函数逻辑意义，计算未来全部时刻状态量与控制量的加权值。

基于对代价函数进行优化计算，可得，

$$u(t) = -Kx(t)$$

K ，基于 Q 及 R 的最优增益矩阵(Optimal Gain Matrix)。

可基于黎卡提方程(Algebraic Riccati Equation)解算该方程。

代价函数最优解。

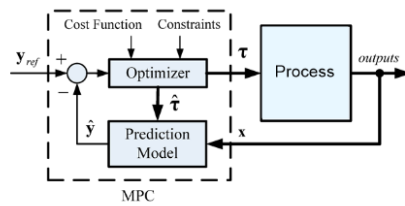
$$J^* = \mathbf{x}_0^T P \mathbf{x}_0$$

注，线性控制器(Linear Controller)与线性估计器(Linear Estimator)。

比方，卡尔曼滤波器是一个线性二次估计器(Linear Quadratic Estimator)，未来轨迹不确定，基于历史数据估测未来状态；线性二次调整器(LQR)是线性控制器，基于任务轨迹，要求特定控制量以达成该任务。

3. 模型预测控制(MPC, Model Predictive Control)

MPC，可视作有限时窗(Finite Horizon)的线性二次调整器。特别是，每个时步计算的控制序列中，只选第一个控制量执行。



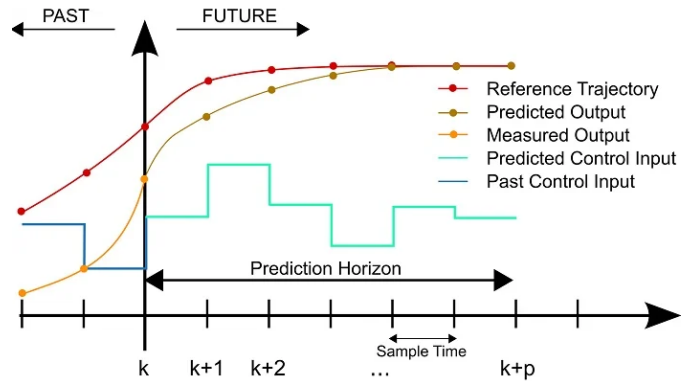
相对于 LQR,MPC 基于机器人状态预测模型及反馈信息计算机器人运动控制量，控制机器人运动更趋近任务轨迹。

MPC 运行的三个要素：机器人动力学模型，以预测机器人在未来一段时间内的状态信息；代价函数，对模型预测的轨迹与规划轨迹的最小误差、控制量的最小变加速度(Jerk)等误差信息的定义；优化算法，用于计算代价函数。

代价函数的逻辑意义在于找到一系列控制器操作量使未来一段时间内预测的轨迹与规划的轨迹尽可能拟合，并且这个控制量序列相邻的变量之间变化尽可能的小。

$$\begin{aligned}
 & \min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \|y_k - r(t)\|_2^2 + \rho \|u_k - u_r(t)\|_2^2 \\
 \text{s.t.} \quad & x_{k+1} = f(x_k, u_k) \quad \text{prediction model} \\
 & y_k = g(x_k) \\
 & u_{\min} \leq u_k \leq u_{\max} \quad \text{constraints} \\
 & y_{\min} \leq y_k \leq y_{\max} \\
 & x_0 = x(t) \quad \text{state feedback}
 \end{aligned}$$

模型轨迹预测时窗。



基于 MPC 进行线性倒立摆模型轨迹控制。

模型状态运动方程。

$$\hat{x}_{k+1} = A \hat{x}_k + B \ddot{x}_k$$

方程左侧，第 $k+1$ 时刻质心状态估计；右侧，第一项，第 k 时刻质心状态估计；第二项，质心变加速度。

基于 T 周期进行状态采样，方程形式。

$$\hat{x}_{k+1} = \begin{bmatrix} 1 & T & T^2/2 \\ 0 & 1 & T \\ 0 & 0 & 1 \end{bmatrix} \hat{x}_k + \begin{bmatrix} T^3/6 \\ T^2/2 \\ T \end{bmatrix} \ddot{x}_k$$

质心状态估计向量。

$$\hat{x}_k = \begin{bmatrix} x(kT) \\ \dot{x}(kT) \\ \ddot{x}(kT) \end{bmatrix}$$

零力矩点与质心状态关系方程。

$$z_k = \begin{bmatrix} 1 & 0 & h_{CoM}/g \end{bmatrix} \hat{x}_k$$

$$z_k^{min} \leq z_k \leq z_k^{max}$$

方程联立，可得未来 N 步时窗零力矩点方程。

$$\begin{bmatrix} z_{k+1} \\ \vdots \\ z_{k+N} \end{bmatrix} = \begin{bmatrix} 1 & T & T^2/2 - h_{CoM}/g \\ \vdots & \vdots & \vdots \\ 1 & NT & N^2T^2/2 - h_{CoM}/g \end{bmatrix} \hat{x}_k + \begin{bmatrix} T^3/6 - Th_{CoM}/g & 0 & 0 \\ \vdots & \ddots & 0 \\ (1+3N+3N^2)T^3/6 - Th_{CoM}/g & \dots & T^3/6 - Th_{CoM}/g \end{bmatrix} \times \begin{bmatrix} \ddot{x}_k \\ \vdots \\ \ddot{x}_{k+N-1} \end{bmatrix}$$

方程简写形式。

$$Z_{k+1} = P_x \hat{x}_k + P_u \ddot{X}_k$$

基于零力矩点轨迹估计误差及质心变加速度正则项定义代价函数。

$$\min_{\ddot{X}_k} \frac{1}{2} Q \left(Z_{k+1} - Z_{k+1}^{ref} \right)^2 + \frac{1}{2} R \ddot{X}_k^2$$

基于二次规划法优化代价函数，可估算质心未来 N 步的变加速度。

$$\ddot{X}_k = - \left(P_u^T P_u + \frac{R}{Q} I_{N \times N} \right)^{-1} P_u^T \left(P_x \hat{x}_k - Z_k^{ref} \right)$$

基于状态运动方程，利用估算的质心在第一时间步的变加速度，算得下一步质心状态。

基于模型逆向运动学、动力学方程，用估算的质心状态算得机器人关节速度、力矩控制量。

机器人基于控制量执行，观测质心状态作反馈量，进行下一次优化估算。

4. 整体控制(WBC, Whole Body Control)

机器人整体运动控制方法，基于控制器输出的控制量类型可分作基于关节速度、基于关节力矩的两类方法；基于动力学方程解算方法，可分作解析法(Closed-Form)及优化法。

1)、解析法

通过投影(Projection)、伪逆运算等解算任务控制方程。

基于速度的整体控制器任务方程。

$$\dot{q} = \tilde{J}_1^\# \dot{x}_1 + \tilde{J}_2^\# \dot{x}_2 + \cdots + \tilde{J}_k^\# \dot{x}_k$$

$\tilde{J}_i$ ，可微分变换。比方，运动学雅可比矩阵。

雅可比矩阵伪逆定义。

$$\tilde{J}_i^\# = \tilde{J}_i^T \left(\tilde{J}_i \tilde{J}_i^T \right)^{-1}$$

基于任务优先级进行层级化顺序(Hierarchical Ordering)的任务方程。

$$\dot{q} = J_1^\# \dot{x}_1 + J_{2|1}^\# \dot{x}_2 + \cdots + J_{k|p(k)}^\# \dot{x}_k$$

$$J_{i|p(i)} = J_i N_{p(i)}$$

第 i 个任务此前全部高优先级任务 p(i) 的零空间(Null-Space)定义。

$$N_{p(i)} = \sum_{j=1}^{i-1} N_j$$

第 j 个任务的零空间定义。

$$N_j = (I - J_j^\# J_j)$$

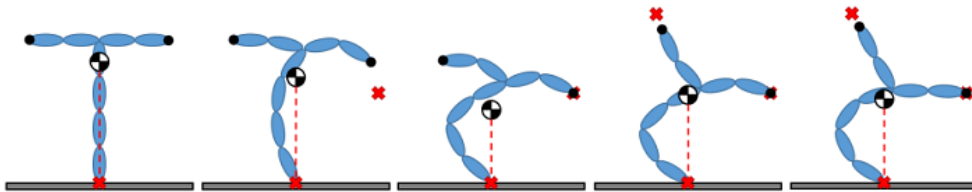
I , 单位矩阵, $n \times n$ 维。 n , 机器人自由度。

零空间, 使矩阵 A 的向量乘运算等于零向量的全部乘向量的集合。 $\{v \mid A \cdot v = 0\}$ 。

逻辑意义, 零空间向量与矩阵 A 行向量正交无关。

层级化任务控制中, 雅可比矩阵进行零空间投影的意义在于, 使雅可比矩阵的向量转换到与高优先级任务雅可比矩阵向量空间正交的零空间, 以便低优先级任务的控制不影响或几乎不影响此前对高优先级任务的控制。

比方, 高优先级任务, 控制机器人关节速度使质心(COM)投影到地面固定点; 次优先任务, 机器人右侧支终端执行器到固定位置; 第三优先任务, 机器人左侧支终端执行器到固定位置。



框架图, 可看到质心投影到地面固定位置; 右侧支到右固定点不改变质心投影位置; 左侧支到左固定点不改变前两个任务的位置。

层级化任务问题在于, 高优先级任务约束了次优先级任务的调整空间。

可通过雅可比伪逆加权以平衡各任务。

$$\tilde{J}_i^\# = W^{-1} \tilde{J}_i^T \left(\tilde{J}_i W^{-1} \tilde{J}_i^T \right)^{-1}$$

加权方法问题在于各任务可能都不准确。

可用混合法，高优先级任务基于层级化方法，以确保高优先级任务的准确性；低优先级任务基于加权方法，以平衡对准确度要求不高的低优先级任务的实现。

基于力矩的整体控制。

与基于速度的整体控制方法相似。

任务方程。

$$\tau = \tilde{J}_1^T F_1 + \tilde{J}_2^T F_2 + \cdots + \tilde{J}_k^T F_k$$

层级化任务方程。

$$\tau = J_1^T F_1 + J_{2|1}^T F_2 + \cdots + J_{k|p(k)}^T F_k$$

加权形式任务方程。

$$\tau = w_1 J_1^T F_1 + w_2 J_2^T F_2 + \cdots + w_k J_k^T F_k$$

2)、优化法

基于速度的整体控制任务定义。

$$A_i x = b_i$$

$$C_i x \leq d_i$$

x , 关节速度量。

b_i , 第 i 任务。 m 维。

d_i , 第 i 约束。 m 维。

A_i, C_i , 矩阵 $m * n$ 维。

目标函数定义。

$$S_i = \{\operatorname{argmin}_{x \in S_{i-1}} \|A_i x - b_i\|^2 + \|w\|^2\}$$

$$C_i x - w \leq d_i, w \in \mathbb{R}^{m+}$$

基于递归算法解目标函数示例。

Set the system of equality constraints: $\bar{A}_0, \bar{b}_0$.
Set the system of inequality constraints: $\bar{C}_0, \bar{d}_0$.

$$x_i^* = \{\operatorname{argmin}_{x \in S_i} \|x\|^2\}$$

for $k = 0$ to $p - 1$ do

Solve the Quadratic Program to obtain S_{k+1}

$$\bar{A}_{k+1} \leftarrow \begin{bmatrix} \bar{A}_k \\ A_k \end{bmatrix}, \quad \bar{b}_{k+1} \leftarrow \begin{bmatrix} \bar{b}_k \\ A_k x_k^* \end{bmatrix}.$$

$$\bar{C}_{k+1} \leftarrow \bar{C}_k, \quad \bar{d}_{k+1} \leftarrow \bar{d}_k.$$

for all c_k^j in C_k do
if $c_k^j x_k^* \leq d_k^j$ then

$$\bar{C}_{k+1} \leftarrow \begin{bmatrix} \bar{C}_{k+1} \\ c_K^j \end{bmatrix}, \quad \bar{d}_{k+1} \leftarrow \begin{bmatrix} \bar{d}_{k+1} \\ d_K^j \end{bmatrix}$$

else

$$\bar{A}_{k+1} \leftarrow \begin{bmatrix} \bar{A}_{k+1} \\ c_K^j \end{bmatrix}, \quad \bar{b}_{k+1} \leftarrow \begin{bmatrix} \bar{b}_{k+1} \\ c_K^j x_k^* \end{bmatrix}$$

end if

end for

end for

基于力矩的整体控制(Torque-Based WBC)。

与基于速度的整体控制相似。

区别在于要分析关节加速度、关节力矩、连接力(Contact Forces)。

$$x_i = [\ddot{q}_i, \tau_i, F_{ei}]$$

同时，应满足浮动基(Floating-Base)机器人运动方程约束。

$$M\ddot{q} + C\dot{q} + h - J_e^T F_e = S^T \tau$$

M ，惯性量矩阵。

C ，科氏力项。

h ，重力向量。

F_e ，应用于机器人的外部扭力(Wrench)。

J_e ，扭力点的雅可比矩阵。

S ，选择矩阵(Selection Matrix)。

$$S_0 = \begin{bmatrix} 0_{n_a \times n_{fb}} & I_{n_a \times n_a} \end{bmatrix}$$

n_a ，机器人主动关节(Active Joints)数。

n_{fb} ，机器人浮动基自由度。

τ ，关节力矩向量。

机器人运动方程实质，基于欧拉-拉格朗日方程动能、势能(做功)推算内、外力与关节加速度、速度的关系。

5. 混合零动力(HZD, Hybrid Zero Dynamics)控制

基于摆动阶段模型(Swing Phase Model)及冲击模型(Impact Model)的机器人步态混合模型(Hybrid Model Of Walking)。

$$\Sigma : \begin{cases} \dot{x} = f(x) + g(x)u, & x^- \notin \mathcal{S} \\ x^+ = \Delta(x^-), & x^- \in \mathcal{S} \end{cases}$$

交换集合(Switching Set) $\mathcal{S}$ 定义。

$$\mathcal{S} := \{(q, \dot{q}) \in T\mathcal{Q} \mid p_2^z(q) = 0, \dot{p}_2^z(q, \dot{q}) < 0\}$$

具备冲击效应(Impulse Effects)的非线性系统。

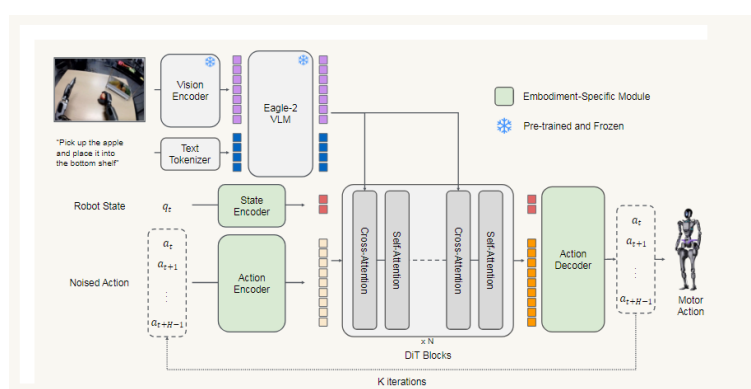
$$\Sigma_{\text{zero}} : \begin{cases} \dot{z} = f_{\text{zero}}(z), & z^- \notin \mathcal{S} \cap \mathcal{Z} \\ z^+ = \Delta_{\text{zero}}(z^-), & z^- \in \mathcal{S} \cap \mathcal{Z} \end{cases}$$

注，可基于强化学习及混合零动力进行双足机器人控制，省去大量训练。

第三章. 机器人运动控制-VLA

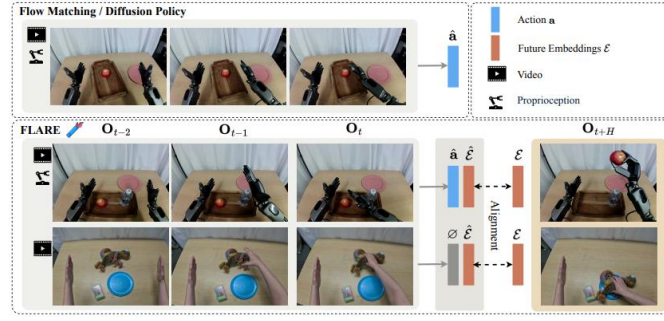
Nvidia 发布的视觉语言动作模型 N1.5，基于 GR00T N1 基础模型，通过简化视觉语言模型的多层感知器、对视觉及文本标记嵌量进行层归一化，提高语言指令遵循及泛化能力(Language Following And Generalization)；基于流匹配及未来隐表征对齐 (FLARE，Future LATent Representation Alignment) 目标函数改进机器人关节策略模型性能，实现视频学习；利用 DreamGen 合成机器人数据训练扩大模型泛化能力。

一、 总体框架

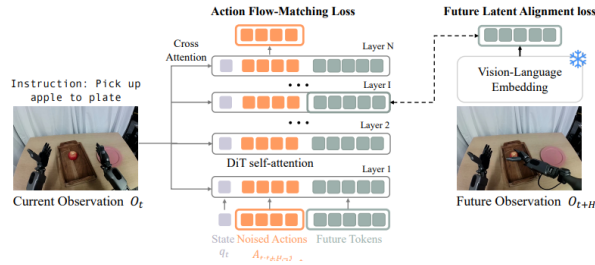


框架图，基本与 GR00T N1 相同。区别，预训练及优调时，冻结视觉语言模型；简化连接视觉编码器与视觉语言模型的多层感知器；对视觉及文本标记嵌量进行层归一化；基于流匹配及未来隐表征对齐损失训练动作生成模型；利用 DreamGen 合成数据训练扩展模型泛化能力。

二、 未来隐表征对齐(FLARE)



图，顶部，基于流匹配或扩散模型预测动作状态；居中，基于动作流匹配及未来隐表征对齐目标函数训练模型；底部，基于未来隐表征对齐目标函数训练模型。



框架图，左侧，计算视觉语言嵌量；居中，基于流匹配损失函数的扩散变换器(DiT, Diffusion Transformer)对机器人状态标记嵌量、动作标记嵌量及未来标记嵌量的拼接序列进行自注意力计算，通过视觉语言嵌量交叉注意力计算调控动作生成；右侧，扩散变换器中间层，相应于未来标器嵌量的激活特征与观测的未来视觉语言嵌量，基于余弦相似度(Cosine Similarity)计算未来隐特征对齐损失。

1. 损失函数

1)、流匹配

损失函数定义。

$$L_{fm}(\theta) = E_{\tau}[\|V_{\theta}(\phi_t, A_t^{\tau}, q_t) - (\epsilon - A_t)\|^2]$$

动作序列推理定义。

基于前向欧拉积分(Forward Euler Integration)方程，通过 K 步去噪生成动作序列。

$$A_t^{\frac{\tau+1}{K}} = A_t^\tau + \frac{1}{K} V_\theta(\phi_t, A_t^\tau, q_t)$$

时步(Timestep) τ 的分布定义， $s = 0.999$ 。

$$p(\tau) = \text{Beta}(\frac{s-\tau}{s}; 1.5, 1)$$

2)、隐表征对齐

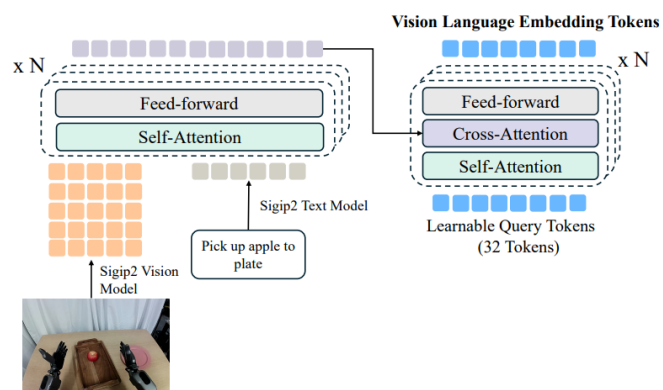
隐表征对齐损失函数定义。

$$L_{align}(\theta) = -E_\tau[\cos(f_\theta(\phi_t, A_t^\tau, q_t), g(\phi_t + H))]$$

3)、模型损失函数

$$L = L_{fm} + \lambda L_{align}$$

2. 视觉语言模型

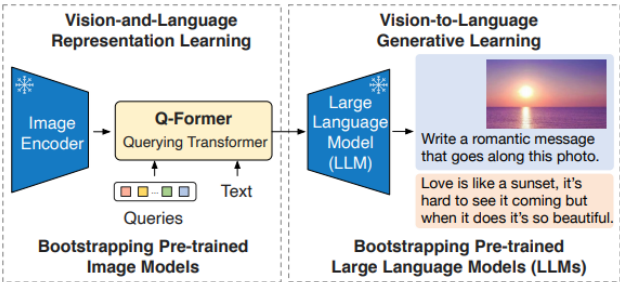


框架图，左侧，基于 SigLIP-2 编码视觉及语言嵌量，通过多个自注意力变换器块(Self-Attention Transformer Blocks)计算视觉语言注意力(Cross-modal Dependencies)；右侧，视觉语言注意力及查询标记(32 Learnable Query Tokens, Randomly Initialized)基于 Q-former 模块生成压缩的视觉语言嵌量标记(32 Compressed Vision-Language Tokens)。

3. 查询变换器(Q-Former, Querying Transformer)

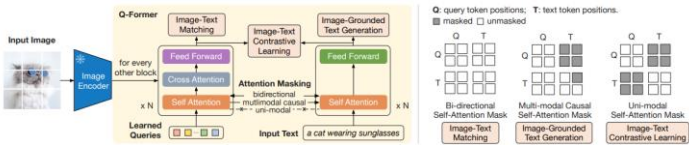
改进语言图像预训练(BLIP-2, Bootstrapping Language-Image Pre-training)框架，基于已有固参图像编码器及大语言模型改进视觉语言预训练，利用查询变换器提取文本最相关的视觉特征、弥合模态间隙(Modality Gap)。

查询变换器预训练分两阶段。



框架图，左侧，第一阶段，通过固参图像编码器(frozen image encoder)改进视觉语言表征学习(Bootstraps Vision-Language Representation Learning)；右侧，第二阶段，通过固参大语言模型改进视觉转语言生成学习(Bootstraps Vision-To-Language Generative Learning)。

查询变换器框架及预训练第一阶段。

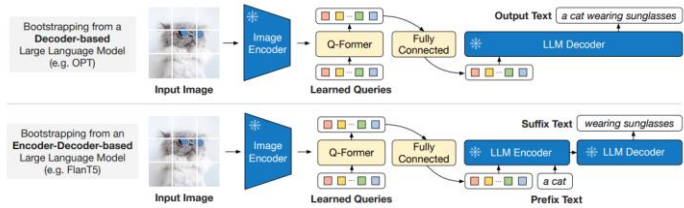


框架图，左侧，查询变换器，基于共享自注意力层的图像变换器及文本变换器亚模块(Transformer Submodules)实现，利用多个目标函数进行预训练；图像变换器通过交叉注意力与图像编码器交互，提取视觉特征；文本变换器作文本编解码器，提取文本表征；自注意力计算时基于不同的预训练任务及自注意力遮码(Self-Attention Masks)策略进行查询量与文本的交互。

右侧，预训练目标函数相应的自注意力遮码策略，图文匹配(ITM, Image-Text Matching) 基于双向自注意力遮码(Bi-Directional Self-Attention Mask)，图生文(ITG, Image-Grounded Text Generation)基于多模态因果自注意力遮码(Multimodal Causal Self-Attention Mask)，图文比对学习(ITC, Image-Text Contrastive Learning)基于单模态自注意力遮码(Unimodal Self-Attention Mask)。

预训练第二阶段。

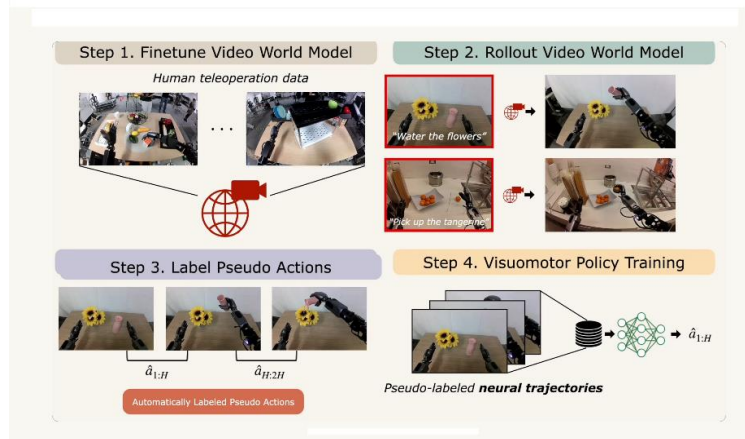
基于全连接层对查询变换器计算的查询注意力嵌量进行投影，得到与大语言模型的文本嵌量等维度的嵌量，前置(Prepnd)到文本嵌量前，调控语言模型生成文本。



三、 DreamGen

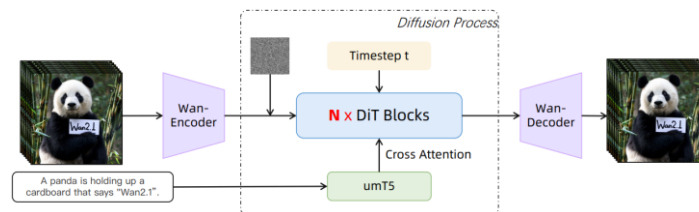
基于图生视频模型合成适用于目标机器人具身的各类视频(Photorealistic Synthetic Videos)，再利用隐动作模型(Latent Action Model)或逆动力学模型(IDM, Inverse-Dynamics Model)还原动作序列(Pseudo-Action Sequences)。

1. 总体框架

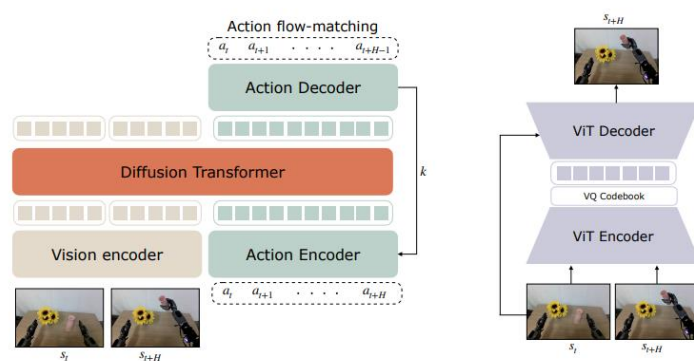


框架图，基于遥操作机器人轨迹微调视频世界模型(Video World Model)，比方，WAN；基于给定的起始视频帧及语言指令，生成执行指令意图的视频；基于隐动作模型或逆动力学模型标注视频中的动作，形成神经轨迹(Neural Trajectories)；基于神经轨迹训练视觉机器人策略(Visuomotor Robot Policies)。

2. 视频世界模型

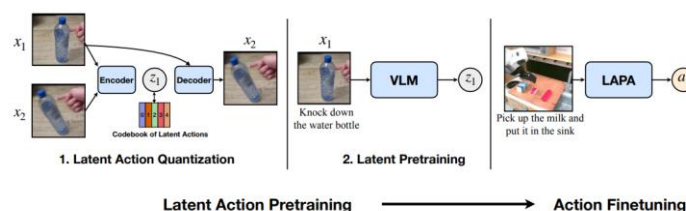


3. 还原动作模型



框架图，左侧，逆动力学模型框架，不同时刻的两个视频帧通过 SigLIP-2 编码器计算嵌量，基于流匹配损失函数训练，调控扩散变换器生成两个视频帧之间的动作序列；右侧，基于视频隐动作预训练模型(LAPA, Latent Action Pretraining for General Action Models)，利用 VQ-VAE 损失函数训练，生成隐动作。

隐动作预训练。



框架图，左侧，基于噪声替代(NSVQ, Noise Substitution In Vector Quantization)方法的向量量化-变分自编码器模型学习连续视频帧所示的量化隐动作；居中，利用学习的隐动作训练视频语言模型，实现行为克隆(Behavior Cloning)；右侧，基于少量机器人动作数据集微调视觉语言动作模型，实现隐动作到机器人动作的映射。

Nvidia GR00T 模型框架与 Physical Intelligence 的机器人视觉语言动作模型相似。模型当前主要侧重于短程(Short-Horizon)控制任务，要实现长程运动控制(Long-Horizon Loco-Manipulation)可能需机器人硬件、模型架构及训练数据的改进。

基于隐动作学习方法，利用云端计算微调模型应可实现机器人实时模仿学习。基于多模态动作模型，利用推理、三维空间信息、步骤对齐及强化训练，应该可实现机器人长程灵活运动控制。

第四章. 机器人运动控制-RL

基于 PPO 方法训练机器人策略动作模型。

状态，机器人质心的位置、方向、线速度、角速度及各关节的角度、速度等。

动作，机器人关节控制量。

选 N 段机器人轨迹样本。

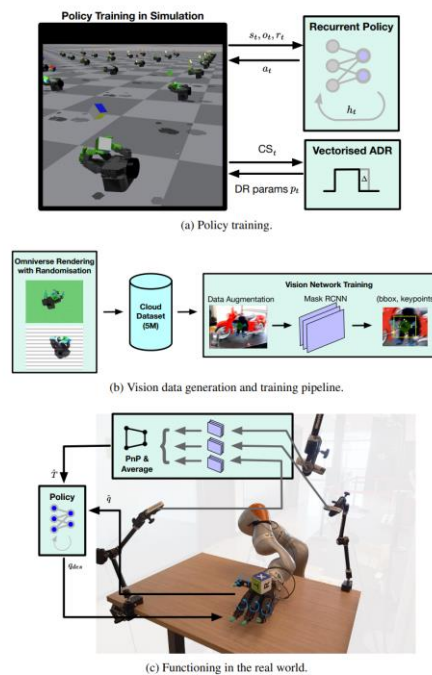
基于价值网络预测误差损失函数优化更新价值网络。

基于策略网络目标函数优化更新策略网络。

第五章. 机器人手操作-RL

Nvidia Allegro 机器人手运动控制，仿照 OpenAI Shadow Hand 方法，基于 PPO 强化学习算法框架与视觉追踪系统，在 Isaac Gym 仿真环境下，利用向量化自动场景域随机化(Vectorised Automatic Domain Randomisation)机制生成多样化的训练数据、尽可能扩大域随机化区间，保持策略性能的同时，提高策略的泛化能力及仿真到现实的迁移能力。

总体框架，Allegro 机器人手，基于 RGB 相机与 Mask R-CNN 网络实现对机器人手心中目标物的位姿的预测；基于 PPO 算法训练的策略指导机器人手指的运动。PPO 算法的策略(Actor)与价值函数(Value Function, Critic)基于长短期记忆网络(LSTM)。



一、 任务定义

通过对 Allegro 机器人手指的控制，将随机放置在机器人手心、带颜色及标记的立方体目标物不断地重定向到指定的目标方向。

步骤，

- 1、立方体随机放置在机器人手心；
- 2、基于特定正交矩阵群($SO(3)$)采样，随机指定期望的立方体目标方向；
- 3、基于策略模型指挥机器人手指运动，将立方体调整到期望的目标方向；
- 4、立方体调整后与目标方向偏离小于阈值度(比方，0.4 弧度)，则采样指定新的目标方向；

依此循环，不断控制机器人手指将立方体从当前方向调整到新的目标方向；

在立方体不掉落的情况下，连续将立方体调整到目标方向的次数或者保持立方体在同一个状态下的时长可作判断机器人手控制成功与否的标准；

二、 环境定义

机器人手，Allegro Hand + 机械臂。

视觉追踪系统，3 个基于 Intel D415 的 RGB 相机，相机相对于机器人手心连杆(Palm Link of the Hand)进行外部校准；

目标物，各面不同颜色及标记的 6.5 厘米的立方体，基于机器人手心表示立方体的位姿；

仿真器，Isaac Gym；

训练平台，8 块 Nvidia A40 GPU + 32 Nvidia V100 GPU；

通过相机-相机外部校准、相机-机器人校准可实现相机参考坐标框(Reference Frame)到机器人掌心坐标系进行变换。

三、 策略学习

机器人手通过运动控制调整立方体到期望方向，是一个串行决策问题。

策略代理与环境交互实现最大化折扣奖励和。

基于 PPO 算法框架学习随机策略(Actor)与价值函数(Critic)。

1、策略框架

LSTM 网络, 层归一化隐单元(Hidden Units of Layer Normalization)的)*1024 个; 多层感知器(MLP)*2 个(512 units); 激活函数(ELU, Exponential Linear Units);

输入, 观测量(50 维度)及此前的隐状态;

输出, 动作, 通过指数移动均值(EMA, Exponential Moving Average)平滑因子(Smoothing Factor)进行低通过滤(Low-Pass Filtered)以防电机线缆等损坏;

动作空间, 基于机器人手 16 个关节对应的比例差分控制器的目的位置(PD Controller Target)定义;

2、价值函数

LSTM 网络, 层归一化的隐单元*2048 个; 多层感知器(1024 units, 512 units); ELU 激活*2 个;

输入, 观测量(265 维度);

输出, 价值;

3、训练方法

策略, 采用用线性调度学习率(Linear Scheduling of the Learning Rate);

价值函数, 采用固定学习率(Fixed Learning Rate);

优化计算, 时序反向传播(BPTT, Backpropagation Through Time), 时窗截取长度 16;

4、输入定义

Input	Dimensionality	Actor	Critic
Object position with noise	3D	✓	✓
Object orientation with noise	4D (quaternion)	✓	✓
Target position	3D	✓	✓
Target orientation	4D (quaternion)	✓	✓
Relative target orientation	4D (quaternion)	✓	✓
Last actions	16D	✓	✓
Hand joints angles	16D	✓	✓
Stochastic delays	4D	×	✓
Fingertip positions	12D	×	✓
Fingertip rotations	16D (quaternions)	×	✓
Fingertip velocities	24D	×	✓
Fingertip forces and torques	24D	×	✓
Hand joints velocities	16D	×	✓
Hand joints generalised forces	16D	×	✓
object scale, mass, friction	3D	×	✓
Object linear velocity	3D	×	✓
Object angular velocity	3D	×	✓
Object position	3D	×	✓
Object rotation	4D	×	✓
Random forces on object	3D	×	✓
Domain randomisation params	78D	×	✓
Gravity vector	3D	×	✓
Rotation distances	2D	×	✓
Hand scale	1D	×	✓

四、 奖励定义

Reward	Formula	Weight	Justification
Rotation Close to Goal	$1/(d + 0.1)$	1.0	Shaped reward to bring cube close to goal
Position Close to Fixed Target	$\ p_{\text{object}} - p_{\text{goal}}\ $	-10.0	Encourage the cube to stay in the hand
Action Penalty	$\ a\ ^2$	-0.001	Prevent actions that are too large
Action Delta Penalty	$\ targ_{\text{curr}} - targ_{\text{prev}}\ ^2$	-0.25	Prevent rapid changes in joint target
Joint Velocity Penalty	$\ v_{\text{joints}}\ ^2$	-0.003	Stop fingers from moving too quickly
Reset Reward	Condition	Value	
Reach Goal Bonus	$d < 0.1$	250.0	Large reward for getting the cube to the target

奖励，奖励项加权后的和。

d ，立方体当前方向到指定方向的旋转距离；

p_{object} ，立方体位置；

p_{goal} ，目标位置；

a ，当前动作；

$targ_{\text{curr}}$ ，当前关节位置目标；

$targ_{\text{prev}}$ ，此前关节位置目标；

v_{joints} ，当前关节速度向量；

五、 仿真

基于 Isaac Gym 物理仿真器，对连接力进行建模；基于单个 GPU 进行并行仿真。

Nvidia Allegro 基于向量化自动域随机化机制生成的数据训练近端策略优化(PPO)模型与视觉追踪系统，降低了训练成本，改进了机器人手控制目标进行重定向的泛化能力。

机器人仿真篇

机器人仿真相当于传统机器人运动控制的正向运动学、动力学过程。

仿真步骤。

一、基于真实机器人创立仿真本体模型。

二、计算或预测机器人动作控制量

传统机器人方法，基于特定任务，利用逆向运动学或动力学计算机器人关节速度或力矩。

机器学习方法，基于深度学习动作模型或深度强化学习策略模型预测机器人关节动作量。

三、基于机器人本体模型，利用正向运动学或动力学计算机器人终端执行器位姿、速度。

四、基于渲染器进行展示。

仿真迁移。

仿真与真实环境的差异(Gap)，使得仿真训练模型不易迁移到真实环境中应用。原因可能在于，基于机器学习方法定义的机器人本体隐模型与仿真环境正向运动学/动力学的物理模型区别很大，或者说正向、逆向动力学方法不对称；同时，仿真与真实环境力的作用关系不同。

因此，基于状态预测模型替代传统仿真中的正向运动学/动力学过程，可能是改进仿真迁移的方法。

第一章. 动力学

人形机器人关节刚体系统的动力学方程推理。

一、粒子动力学

假定人形机器人由一系列刚性粒子组成，每个粒子 $\mathbf{r}_i$ 由一系列对应于机器人关节设置的广义坐标定义。

$$\mathbf{r}_i = \mathbf{r}_i(q_1, q_2, \dots, q_{n_j}, t)$$

粒子 $\mathbf{r}_i$ 的极小变化等于 $\mathbf{r}_i$ 在广义坐标轴 q_j 方向上的变化的矢量和。

$$\delta \mathbf{r}_i = \sum_j \frac{\partial \mathbf{r}_i}{\partial q_j} \delta q_j$$

基于惯性力的虚功计算等式、惯性力在广义坐标轴 q_j 方向上的分量的计算等式、刚性粒子的动能计算等式，可以推理得到刚性粒子的拉格朗日方程，基于刚性粒子动能和推理得到多刚体系统的拉格朗日方程。

惯性力的虚功。

$$\mathbf{f}_i \cdot \delta \mathbf{r}_i = \mathbf{f}_i \cdot \sum_j \frac{\partial \mathbf{r}_i}{\partial q_j} \delta q_j \equiv \sum_j Q_{ij} \delta q_j = \mathbf{Q}_i \cdot \delta \mathbf{q}$$

惯性力在广义坐标轴 $\mathbf{q}_j$ 方向上的分量。

$$\mu_i \ddot{\mathbf{r}}_i \cdot \frac{\partial \mathbf{r}_i}{\partial q_j} = \frac{d}{dt} \left(\frac{\partial}{\partial \dot{q}_j} \left(\frac{1}{2} \mu_i \dot{\mathbf{r}}_i^T \dot{\mathbf{r}}_i \right) \right) - \frac{\partial}{\partial q_j} \left(\frac{1}{2} \mu_i \dot{\mathbf{r}}_i^T \dot{\mathbf{r}}_i \right)$$

刚性粒子的动能。

$$T_i = \frac{1}{2} \mu_i \dot{\mathbf{r}}_i^T \dot{\mathbf{r}}_i$$

多刚体系统的刚性粒子对应的拉格朗日动力学方程。

$$\frac{d}{dt} \left(\frac{\partial T_i}{\partial \dot{q}_j} \right) - \frac{\partial T_i}{\partial q_j} - Q_{ij} = 0$$

多刚体粒子系统的拉格朗日动力学方程。

$$\frac{d}{dt} \frac{\partial T}{\partial \dot{\mathbf{q}}} - \frac{\partial T}{\partial \mathbf{q}} = M\ddot{\mathbf{q}} + \dot{M}\dot{\mathbf{q}} - \frac{1}{2}\dot{\mathbf{q}}^T \left(\frac{\partial M}{\partial \mathbf{q}} \right)^T \dot{\mathbf{q}} \equiv M\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}})$$

二、刚体动力学

基于线动量、角动量计算刚体动能等式，再利用质心线速度、角速度化简动能等式，然后基于刚性粒子的拉格朗日动力学等式计算刚体拉格朗日动力学等式，与基于牛顿-欧拉等式计算获得的刚体拉格朗日动力学等式比对，可得质量矩阵、科氏力与离心力、力矩的计算等式。

基于线动量、角动量计算获得的动能等式。

$$\begin{aligned} T &= \frac{1}{2} m \mathbf{v}^T \mathbf{v} + \frac{1}{2} \boldsymbol{\omega}^T \left(\sum_i -\mu [\mathbf{r}'_i] [\mathbf{r}'_i] \right) \boldsymbol{\omega} \\ &= \frac{1}{2} m \mathbf{v}^T \mathbf{v} + \frac{1}{2} \boldsymbol{\omega}^T I_c \boldsymbol{\omega} \end{aligned}$$

通过线速度、角速度化简后的动能等式。

$$T = \frac{1}{2} \dot{\mathbf{q}}^T J^T M_c J \dot{\mathbf{q}}$$

刚体拉格朗日动力学等式。

$$\frac{d}{dt} \left(\frac{\partial T}{\partial \dot{\mathbf{q}}} \right) - \frac{\partial T}{\partial \mathbf{q}} = (J^T M_c J) \ddot{\mathbf{q}} + (J^T M_c \dot{J} + J^T [\tilde{\omega}] M_c J) \dot{\mathbf{q}}$$

基于牛顿-欧拉等式计算得到的刚体动力学等式。

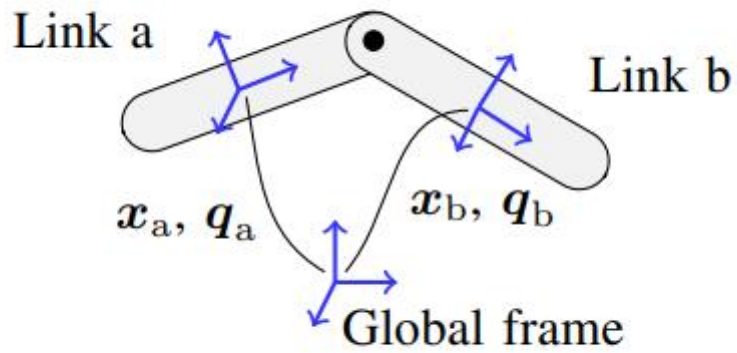
$$(J^T M_c J) \ddot{\mathbf{q}} + (J^T M_c \dot{J} + J^T [\tilde{\omega}] M_c J) \dot{\mathbf{q}} = J_v^T \mathbf{f} + J_\omega^T \boldsymbol{\tau}$$

可知质量矩阵、科氏力与离心力、力矩等式。

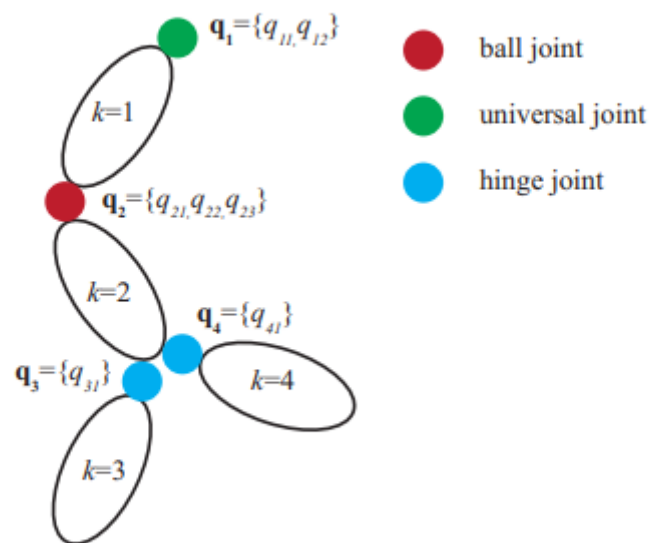
$$\begin{aligned} M(\mathbf{q}) &= J^T M_c J \\ C(\mathbf{q}, \dot{\mathbf{q}}) &= (J^T M_c \dot{J} + J^T [\tilde{\omega}] M_c J) \dot{\mathbf{q}} \\ \mathbf{Q} &= J_v^T \mathbf{f} + J_\omega^T \boldsymbol{\tau} \end{aligned}$$

三、关节刚体动力学

基于旋转关节的两连杆。



基于不同类型关节的多连杆刚体系统。



基于机械臂原理连杆之间的线性、旋转变换雅可比矩阵，再基于刚体拉格朗日动力学等式可知关节刚体系统的运动计算等式。

第二章. 笛卡尔坐标系物理引擎

对于基于笛卡尔坐标系的物理引擎，仿真器仿真时基于广义坐标系计算控制力，然后转换为笛卡尔坐标系下的笛卡尔力(Cartesian Forces)，再基于物理引擎进行前向仿真(Forward Simulation)。

一、速度转换

$$\mathbf{V} \equiv \begin{pmatrix} \mathbf{v}_1 \\ \vdots \\ \mathbf{v}_m \\ \boldsymbol{\omega}_1 \\ \vdots \\ \boldsymbol{\omega}_m \end{pmatrix} = \begin{pmatrix} J_{v1} \\ \vdots \\ J_{vm} \\ J_{\omega 1} \\ \vdots \\ J_{\omega m} \end{pmatrix} \begin{pmatrix} \dot{\mathbf{q}}_1 \\ \vdots \\ \dot{\mathbf{q}}_m \end{pmatrix} \equiv \begin{pmatrix} J_v \\ J_\omega \end{pmatrix} \dot{\mathbf{q}} \equiv J \dot{\mathbf{q}}$$

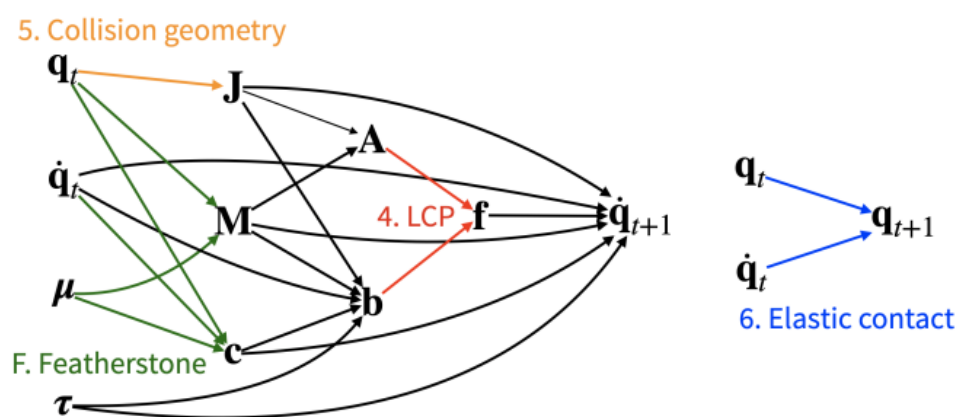
二、力转换

$$\mathbf{Q} = \sum_k J_{vk}'^T \mathbf{f}_k + J_{\omega k}^T \boldsymbol{\tau}_k' = \begin{pmatrix} J_v'^T & J_\omega^T \end{pmatrix} \begin{pmatrix} \mathbf{f}' \\ \boldsymbol{\tau}' \end{pmatrix}$$

第三章. 广义坐标系物理引擎

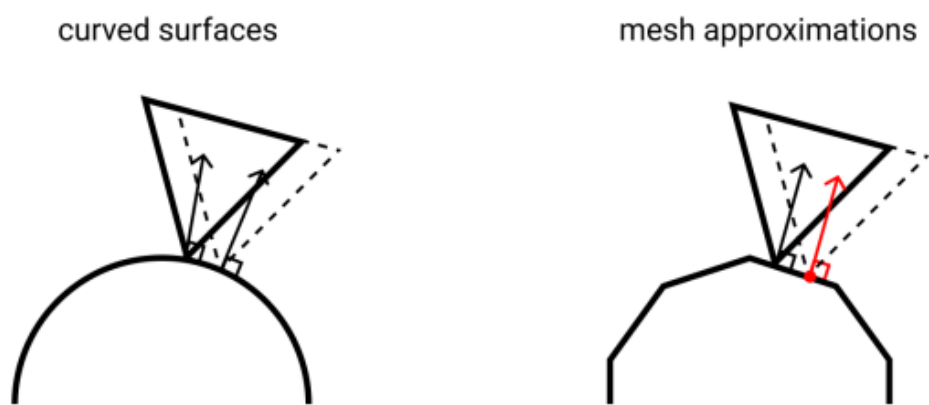
对于基于广义坐标系的物理引擎，前向仿真时输出下一时刻广义坐标系位置及速度。

可微分广义坐标系物理引擎前向仿真示例，基于当前位置、速度、控制力(τ)及惯性特性(μ)计算连接力雅可比矩阵(J)、质量矩阵(M)及科氏力与重力(c)，基于线性互补问题(LCP, Linear Complementarity Problem)计算连接力。



一、碰撞计算

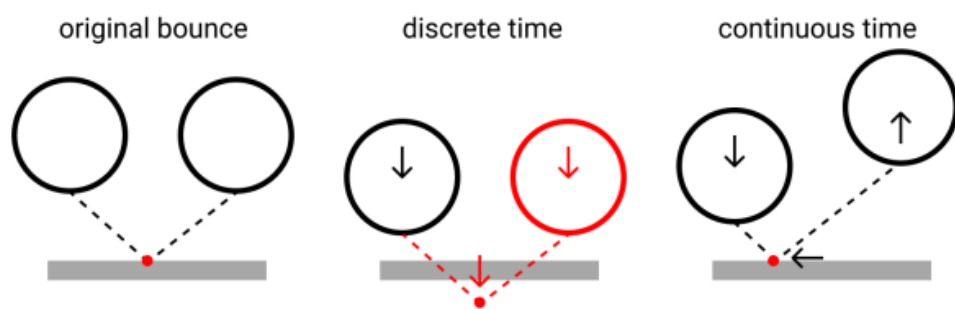
碰撞示例。



曲面碰撞计算不同于多面体。

二、弹性碰撞计算

弹性碰撞示例。



基于连续时间定义弹性碰撞的动力学，离散时间雅可比不足以定义弹性碰撞的动力学。

第四章. 仿真引擎-MuJoCo

MuJoCo，主要是基于物理引擎计算前向动力学(已知力矩计算加速度)、逆向动力学(已知加速度计算力矩)。

相对于传统基于笛卡尔坐标的物理引擎(ODE、PhysX、Bullet 等)，利用很多新的数值优化方法计算接触力(contact forces),提高仿真的准确度及效率;

MuJoCo(Multi-Joint Dynamics with Contact)，基于广义坐标(Generalized Coordinates)、利用递归算法计算机器人动力学状态，基于速度相关(Velocity-Stepping)算法仿真连接点力。

一、 MuJoCo 特性

传统基于笛卡尔坐标的物理引擎(ODE、PhysX、Bullet 等)在仿真人形机器人等多关节系统时，准确度与效率都不高；在应用线性互补问题(LCP, Linear Complementarity Problem)计算连接动力学(Contact Dynamics)时，需手动调节及很小的时间步长(time-steps)来获得稳定的移动。

基于广义坐标的物理引擎(SD/FAST、OpenSim)或者忽略连接力，或者利用弹簧阻尼(Spring Damper)方法导致碰撞变成重合(Large Penetration)或使得连接过于刚性(Stiff Dynamics)。

MuJoCo 改进了这些问题，同时，集成了很多仿真可能不需要却便于实际应用的特性。比方，可基于共享内存或分布架构等并行计算不同的状态及控制量；可进行逆向运动学计算以便分析记录的数据或需计算力矩的应用；可对执行器动力学(液压、气压动力，生物肌肉等)建模；可利用整个计算流程或选择部分流程进行目标计算；模型支持 XML 格式及 C++ API 形式等。

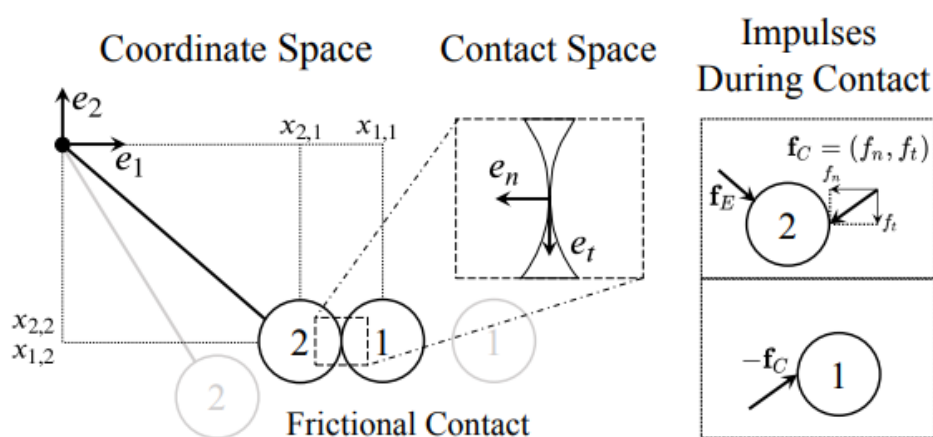
二、 动力学解算

基于微分形式的机器人连续时间的运动动力学等式。

$$M(\mathbf{q}) d\mathbf{v} = (\mathbf{b}(\mathbf{q}, \mathbf{v}) + \boldsymbol{\tau}) dt + J_E(\mathbf{q})^T \mathbf{f}_E(\mathbf{q}, \mathbf{v}, \tau) + J_C(\mathbf{q})^T \mathbf{f}_C(\mathbf{q}, \mathbf{v}, \tau)$$

$\mathbf{M}$ ，广义坐标惯性矩阵； $\mathbf{b}$ ，偏制力，科氏力、离心力、重力、弹簧力； J_E ，等式约束的雅可比矩阵； J_C ，连接力雅可比矩阵； $\mathbf{f}_E$ ，等式约束对应的冲力； $\mathbf{f}_C$ ，连接对应的冲力。

碰撞冲力示例说明。



运动等式的解算过程。

- 1、计算所有刚体的笛卡尔位置(Positions)及方向(Orientations)，检测潜在碰撞(Potential Collisions)，建立 J_E 及 J_C ；
- 2、利用复合刚体算法(CRB, Composite Rigid Body Algorithm)计算惯性矩阵 $\mathbf{M}$ ，利用递归牛顿-欧拉算法(RNE, Recursive Newton Euler Algorithm)计算偏置力 $\mathbf{b}$ 等；

3、建立 $\mathbf{f}_E$ 与 $\mathbf{f}_C$ 的函数关系式，将碰撞时连接力在切线方向的切线(Subspace Tangent)力投影到机器人等式约束空间(Equality Constraint Manifold)；

由于冲力(约束力、连接力)不连续，用 $\mathbf{v}_{t+h}-\mathbf{v}_t$ 替代 $d\mathbf{v}$ ，用 h 替代 dt ，可得机器人离散时间运动动力学等式，

$$M(\mathbf{q}_t) \mathbf{v}_{t+h} = \mathbf{s}(\mathbf{q}_t, \mathbf{v}_t, \tau_t) + J_E(\mathbf{q}_t)^T \mathbf{f}_E + J_C(\mathbf{q}_t)^T \mathbf{f}_C$$

$$\mathbf{s}(\mathbf{q}_t, \mathbf{v}_t, \tau_t) = M(\mathbf{q}_t) \mathbf{v}_t + h\mathbf{b}(\mathbf{q}_t, \mathbf{v}_t) + h\tau_t$$

机器人期望的等式约束速度与下一时刻广义坐标速度的雅可比变换等式。

$$J_E(\mathbf{q}_t) \mathbf{v}_{t+h} = \mathbf{v}_E^*(\mathbf{q}_t)$$

机器人连接冲速度与下一时刻广义坐标速度的雅可比变换等式。

$$J_C(\mathbf{q}_t) \mathbf{v}_{t+h} = \mathbf{v}_C$$

上述方程联立可得 $\mathbf{f}_E$ 与 $\mathbf{f}_C$ 的函数关系式。

$$\mathbf{f}_E = (J_E M^{-1} J_E^T)^{-1} (\mathbf{v}_E^* - J_E M^{-1} (\mathbf{s} + J_C^T \mathbf{f}_C))$$

将 $\mathbf{f}_E$ 代入机器人离散时间运动动力学等式，可得机器人速度($\mathbf{v}$)计算等式。

$$\mathbf{v} = P J_C^T \mathbf{f}_C + \mathbf{r}$$

$$P = M^{-1} - M^{-1} J_E^T (J_E M^{-1} J_E^T)^{-1} J_E M^{-1}$$

$$\mathbf{r} = P \mathbf{s} + M^{-1} J_E^T (J_E M^{-1} J_E^T)^{-1} \mathbf{v}_E^*$$

4、投影连接力(到机器人等式约束空间)，计算连接冲力 $\mathbf{f}_C$ 及连接冲速度 $\mathbf{v}_C$ ；

机器人速度($\mathbf{v}$)计算等式与机器人连接冲速度雅可比变换等式联立可得连接坐标系下的连接冲力($\mathbf{f}_C$)与冲速度($\mathbf{v}_C$)计算等式。

$$A \mathbf{f}_C + \mathbf{v}_0 = \mathbf{v}_C$$

$$A = J_C P J_C^T$$

$$\mathbf{v}_0 = J_C \mathbf{r}$$

连接冲力法线分量与连接冲速度法线分量形成的约束等式(互补条件)。

$$f^{\mathcal{N}} \geq 0, \quad v^{\mathcal{N}} \geq 0, \quad f^{\mathcal{N}} v^{\mathcal{N}} = 0$$

虽然，约束等式与连接冲力($\mathbf{f}_C$)及冲速度($\mathbf{v}_C$)计算等式联立可形成线性互补问题(LCP, Linear Complementarity Problem)。

但是，连接冲力摩擦力方向分量与连接冲速度摩擦力方向分量方向相反，连接冲力在摩擦力矢量锥体内，不适于线性互补问题。

因此，MuJoCo 提供的是隐式互补、凸解算及对角解算三类不同的解算算法。

5、进行数值积分获得机器人下一刻状态(位置信息等)；

基于机器人速度($\mathbf{v}$)计算等式及连接冲力得到下一刻速度，然后利用 Semi-Implicit Euler 或四阶 Runge-Kutta 积分器进行四元数积分得到机器人下一刻位置状态信息等。

三、 连接仿真解算算法

MuJoCo 连接仿真基于数值优化算法，依据解算速度与准度分为隐式互补解算器(Implicit Complementarity Solver)、凸解算器(Convex Solver)、对角解算器(Diagonal Solver)。

1. 隐式互补解算器

准确度优先解算器。基于连接冲力与连接冲速互补条件，将两者映射作变量 $\mathbf{x}$ 的函数。基于连接冲力($\mathbf{f}_C$)与冲速度($\mathbf{v}_C$)计算等式可得优化目标函数，由此可得近似解。

$$\min_{\mathbf{x}} \|\mathbf{A}\mathbf{f}_C(\mathbf{x}) + \mathbf{v}_0 - \mathbf{v}_C(\mathbf{x})\|^2$$

2. 凸解算器

兼顾速度与准确度的解算器。基于连接冲力总是减缓碰撞表面之间的相对速度，因此可对连接坐标系下的动能进行最小化计算，同时满足在摩擦力矢量锥内及碰撞约束条件，由此利用凸优化问题替代连接冲力与连接冲速度法线及摩擦力方向的约束条件得到近似解。

连接坐标系动能最小时的连接冲力的计算式。

$$\mathbf{f}_C = \arg \min_{\mathbf{f} \in \text{Cone}} \frac{1}{2} \mathbf{f}^T (A + R) \mathbf{f} + \mathbf{f}^T \mathbf{v}_0 + q (A \mathbf{f} + \mathbf{v}_0)$$

3. 对角解算器

速度优先解算器。相当于质量-弹簧-阻尼系统，基于惯性相关矩阵 $\mathbf{A}$ 对角元素，调整弹性阻尼器为临界阻尼弹簧，使连接过程不振荡、可积分，由此可计算期望的下一状态的冲速及连接冲力。

MuJoCo 基于广义坐标仿真及优化计算连接动力学等特性实现机器人系统仿真的改进。

四、 MuJoCo 实践

机器人仿真编程与传统图形编程、库函数调用相似。

基于深度强化学习的机器人模型训练仿真步骤。

一、设置环境。创立机器人本体模型、渲染器等。

二、创立机器人策略模型。

三、训练策略模型。

基于策略模型预测机器人动作/力控制量。

基于控制量，调用正向运动学/动力学库计算机器人位姿、速度等运动量。

基于机器人运动观测，计算动作奖励。

累积奖励，直到训练局(Episode)结束。

基于累积奖励，更新策略模型权重。

四、训练过程中，同步渲染展示机器人运动。

1. 实例一

step1, 用 xml 文件建立双足机器人模型;

step2, 基于 mujoco 库编写 python 程序进行模型交互;

step3, 进行强化学习编程(

1、Observation, agent 观测当前环境状态, 比方关节位置、速度, 接触力等

2、Action, agent 选择动作, 比方设置电机的目标角或力矩(target angles or torques)

3、Reward, 环境基于动作执行结果提供奖励, 比方前进对应正奖励(Positive reward), 摔倒对应大的反奖励(Large negative reward), 用太多能量对应小的负值(Small negative reward)

4、State Transition

循环 1-4 步, 直到 agent 学到好的策略.

强化学习算法可以是 PPO, SAC, TD3 等. 可利用 gymnasium 单智能体强化学习环境, 基于 Pytorch 的强化学习库 stable-baselines3 进行编程.

);

step2 的程序实现,比方,

```
```python
import mujoco
import numpy as np
import mediapy as media # For displaying the simulation

加载 XML 定义的模型 Load the XML model
model = mujoco.MjModel.from_xml_path("trex.xml")

创建存储仿真状态的数据结构
```

```

data = mujoco.MjData(model)

创建渲染器,用于渲染及保存仿真的每一帧 Create a renderer
renderer = mujoco.Renderer(model)

仿真器循环 Simulation loop
frames = []
mujoco.mj_resetData(model, data) # Reset the simulation
while data.time < 5: # 运行 5 秒 Run for 5 seconds
 mujoco.mj_step(model, data) # 核心仿真函数, 步进调试仿真器 Advance the simulation
 by one step

 # 从默认相机视角渲染场景 Render the scene
 renderer.update_scene(data)
 frames.append(renderer.render())

运行仿真后, 以视频形式显示仿真 Display the simulation as a video
media.show_video(frames, fps=1/model.opt.timestep)
print("Simulation Complete")
'''

```

step3 的程序实现, <https://github.com/kuds/apex>

```

'''python
部分程序
import gymnasium
import mujoco
from stable_baselines3 import PPO
from stable_baselines3.common.env_checker import check_env
from stable_baselines3.common.vec_env import DummyVecEnv
from stable_baselines3.common.callbacks import EvalCallback
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.vec_env import VecVideoRecorder
from stable_baselines3.common.env_util import make_vec_env

env = gymnasium.make('Humanoid-v5', xml_file="/content/trex.xml")
env.close()

训练
Create Training environment
env = make_vec_env(lambda : gymnasium.make('Humanoid-v5',
 xml_file="/content/trex.xml"), n_envs=4)

Create Evaluation environment

```

```

env_val = make_vec_env(lambda : gymnasium.make('Humanoid-v5',
 xml_file="/content/trex.xml"), n_envs=1)

eval_callback = EvalCallback(env_val,
 best_model_save_path=log_dir,
 log_path=log_dir,
 render=False,
 deterministic=True,
 eval_freq=50_000)

Create the PPO model
model = PPO(
 policy="MlpPolicy", # Multi-layer perceptron policy
 env=env,
 verbose=0,
 learning_rate=0.0003,
 ent_coef=0.01,
 tensorboard_log=log_dir
)

Train the model
time_steps = 5_000_000
model.learn(total_timesteps=time_steps,
 callback=eval_callback,
 progress_bar=True)

Save the model
model.save(os.path.join(log_dir, "final_model"))

mean_reward, std_reward = evaluate_policy(model, env)
print(f"Mean reward: {mean_reward:.2f} +/- {std_reward:.2f}")

env.close()
env_val.close()

最佳模型的评估，利用模型进行预测
Create Evaluation environment
env = gymnasium.make('Humanoid-v5', xml_file="/content/trex.xml", render_mode="rgb_array")
env = DummyVecEnv([lambda: env])

Load the best model
best_model_path = os.path.join(log_dir, "best_model.zip")
best_model = PPO.load(best_model_path, env=env)

```



```

mean_reward, std_reward = evaluate_policy(best_model, env)
print(f"Best Model - Mean reward: {mean_reward:.2f} +/- {std_reward:.2f}")

Record video of the best model
env = VecVideoRecorder(env, os.path.join(log_dir, "videos"),
 video_length=10_000,
 record_video_trigger=lambda x: x == 0,
 name_prefix="best_model_{}".format(name_prefix))

obs = env.reset()
for _ in range(10_000):
 action, _states = best_model.predict(obs, deterministic=True)
 obs, rewards, dones, info = env.step(action)
 env.render()
 if dones:
 break

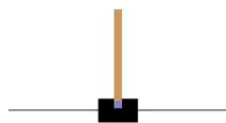
env.close()

'''

```

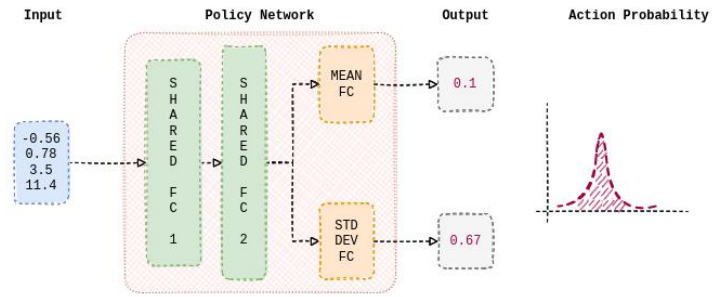
## 2. 实例二

运动小车倒立摆模型。左右移动小车，施加的力或动作使倒立摆保持平衡。



基于 REINFORCE( Reward Increment Non-negative Factor times Offset Reinforcement times Characteristic Eligibility )强化学习算法，学习策略模型以预测动作分布，再采样动作。

策略模型基于两个全连接层，预测动作分布的均值及标准差。



REINFORCE 学习过程。

...

# 重新初始化 REINFORCE 代理。

```
agent = REINFORCE(obs_space_dims, action_space_dims)
```

```
reward_over_episodes = []
```

# 全部训练局(Episodes)。

```
for episode in range(total_num_episodes):
```

```
 obs, info = wrapped_env.reset(seed=seed)
```

```
 done = False
```

# 开始一局的训练。

```
 while not done:
```

```
 # 基于策略模型预测动作分布，采样动作。
```

```
 action = agent.sample_action(obs)
```

```
 # 调用仿真机器人执行动作，返回状态观测量、动作奖励、训练局结束标记等。
```

```
 obs, reward, terminated, truncated, info = wrapped_env.step(action)
```

```
 # 记录动作奖励。
```

```
 agent.rewards.append(reward)
```

```
 # 训练局结束标识更新
```

```
 #
```

```
 #
```

```
 #
```

```
 done = terminated or truncated
```

```

reward_over_episodes.append(wrapped_env.return_queue[-1])

基于当前局(Episode)的奖励计算每个动作的折扣率回报(Discounted Return);
优化更新策略模型权重。
agent.update()

...

REINFORCE 算法类定义。

...

class REINFORCE:

 def __init__(self, obs_space_dims: int, action_space_dims: int):

 # 超参定义
 self.learning_rate = 1e-4 # 策略模型优化学习率
 self.gamma = 0.99 # 回报折扣率
 self.eps = 1e-6 #

 self.probs = [] # 采样动作概率序列
 self.rewards = [] # 动作相应的奖励序列

 # 初始化策略模型
 self.net = Policy_Network(obs_space_dims, action_space_dims)

 # 初始化策略模型优化器
 self.optimizer = torch.optim.AdamW(self.net.parameters(), lr=self.learning_rate)

 # 动作采样方法
 def sample_action(self, state: np.ndarray) -> float:

 state = torch.tensor(np.array([state]))

 # 基于策略模型预测动作分布
 action_means, action_stddevs = self.net(state)

 # 基于分布进行动作采样
 distrib = Normal(action_means[0] + self.eps, action_stddevs[0] + self.eps)
 action = distrib.sample()

```

```

动作概率对数
prob = distrib.log_prob(action)

action = action.numpy()

self.probs.append(prob)

return action

def update(self):

 running_g = 0
 gs = []

 # 计算动作对应的回报(Return)
 for R in self.rewards[::-1]:
 running_g = R + self.gamma * running_g
 gs.insert(0, running_g)

 deltas = torch.tensor(gs)

 log_probs = torch.stack(self.probs).squeeze()

 # 计算损失函数
 #
 loss = -torch.sum(log_probs * deltas)

 # 优化更新策略模型权重
 self.optimizer.zero_grad()
 loss.backward()
 self.optimizer.step()

 # 清理概率、奖励序列
 self.probs = []
 self.rewards = []

```

...

## 第五章. 仿真引擎-Newton

Newton , NVIDIA, Google DeepMind 及 Disney Research 合作开发的高可扩展物理引擎. 利用 OpenUSD 框架定义描述机器人及环境的数据流水线.

基于 Nvidia Warp (CUDA-X 加速库), 可以支持 MuJoCo 解析器(resolver , 计算接触力)、Disney Research 的解析器及自定义解析器.

基于 Newton 物理引擎可以支持 Nvidia Isaac Lab 仿真环境、MuJoCo Playground 或 别的仿真器.

## 第六章. 训练场-MuJoCo Playground

Google DeepMind 等发布的 MuJoCo Playground，基于可本地化运行的 MuJoCo GPU JAX 版本 (MJX, MuJoCo XLA) 创立，以实现机器人仿真、训练、仿真到现实迁移各步骤流水化作业的机器人学习开源框架。利用集成的物理引擎(Physics Engine)、Madrone 批渲染器(Batch Render)、训练环境栈(Stack)，可支持四足、人形、灵巧手、机械臂等各式机器人平台，以及零样本(Zero-Shot)仿真现实迁移。

### 总体框架

基于强化学习仿真、训练、迁移部署步骤。

一、数字孪生，创立与现实世界匹配的仿真环境。

二、定义奖励函数，以调对机器人目标动作(Encode Desired Robot Behavior)。

三、基于仿真环境训练策略。

四、模型部署到机器人。

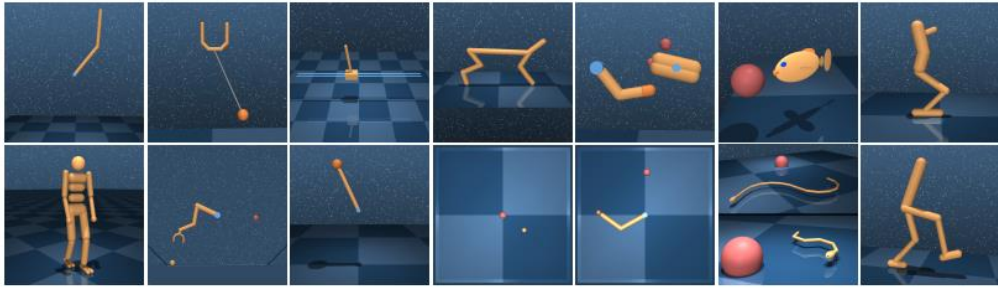
整个过程中，主要通过第二步、第三步及第四步的循环迭代，以使模型预测行为更贴合人类行为特性。

### 环境

环境主要分 DeepMind Control Suite、Locomotion 及 Manipulation 三类。机器人模型基于 MuJoCo Menagerie。

#### 一、DeepMind Control Suite

基于 MJX 实现 DM Control Suite 中的强化学习环境。



## 二、Locomotion

四足、双足机器人的移动任务测试环境。支持摇杆(Joystick)、站立、倒立、摔倒起来(Fall Recovery)等场景。

摇杆任务，用于学习追踪基座前向、侧向速度及偏转速率(Yaw Rate)。

Robot	Type	Environment
Google Barkour	Quadruped	JoystickFlatTerrain, JoystickRoughTerrain
Berkeley Humanoid	Biped	Joystick
Unitree G1	Biped	Joystick
Unitree Go1	Quadruped	JoystickFlatTerrain, JoystickRoughTerrain, Getup, Handstand, Footstand
Unitree H1	Biped	InplaceGaitTracking, JoystickGaitTracking
OP3	Biped	Joystick
Boston Dynamics Spot	Quadruped	JoystickFlatTerrain, JoystickGaitTracking, Getup

## 三、Manipulation

机器人抓取(Prehensile)、非抓取(Non-Prehensile)任务。比方，调整方块方向，基于视觉拾取方块，双臂夹取等。

Robot	Environment
Aloha	SinglePegInsertion
Franka Emika Panda	PickCube, PickCubeOrientation, PickCubeCartesian, OpenCabinet
Franka Emika Panda, Robotiq Gripper	PushCube
Leap Hand	Reorient, RotateZAxis

## 批渲染

MuJoCo Playground 通过 MJX 中 JAX 底层原语(Low Level JAX Primitives)集成 Madrona 渲染引擎，以支持视觉任务训练环境。

视觉任务环境可利用几何大小、色彩、光照等视觉特性的域随机化(Domain Randomization)进行自定义。

Madrona，基于 GPU 的实体组件系统(ECS, Entity-Component-System)，支持软件批光线追踪及 Vulkan 栅格渲染。

注，ECS，软件架构模式。通过继承复合(Composition Over Inheritance)；标识(Identity)、数据(Component)、行为(System)分离提高软件的灵活性、模块化及性能。。

光线追踪支持复杂光线场景、阴影、纹理、几何材质及可变形材质、移动照明(Moving Lights)、地形高度场(Terrain Height Fields)等。

## 训练

移动(Locomotion)、操控(Manipulation)任务基本都基于非对称表演-评价(Actor-Critic)强化学习算法框架进行训练。

移动任务，策略网络(Actor)输入，局域坐标(Body Frame)重力(Gravity)、机器人基座线速度及角速度、关节位置及速度、历史动作及用户任务指令等；价值网络(Critic)额外输入接触力、扰动力、关节力矩等。策略网络及价值网络模型，基于三层的多层感知器(MLP)实现，Swish 激活函数。

移动任务奖励函数。



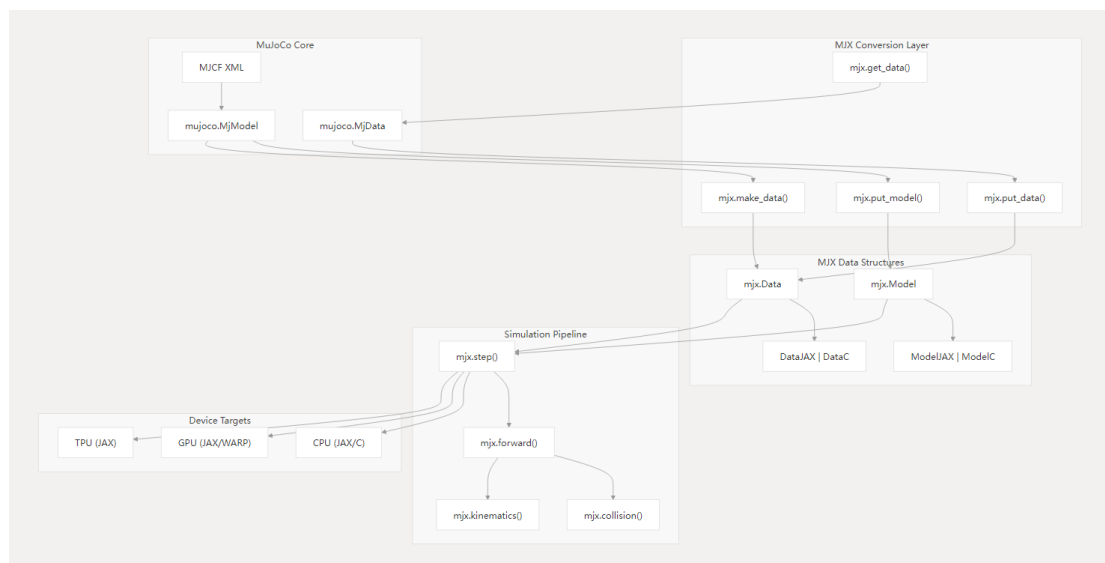
Reward	Expression
Linear Velocity Tracking	$r_v = k_v \exp(-\ cmd_{v,xy} - v_{xy}\ ^2 / \sigma_v)$
Angular Velocity Tracking	$r_\omega = k_\omega \exp(-\ cmd_{\omega,z} - \omega_z\ ^2 / \sigma_\omega)$
Feet Airtime	$r_{air} = \text{clip}((T_{air} - T_{min}) \cdot C_{contact}, 0, T_{max} - T_{min})$
Feet Clearance	$r_{clear} = k_{clear} \cdot \ p_{f,z} - p_{f,z}^{des}\ ^2 \cdot \ v_{f,xy}\ ^{0.5}$
Feet Phase	$r_{phase} = k_{phase} \cdot \exp(-\ p_{f,z} - r_z(\phi)\ ^2 / \sigma_{phase})$
Feet Slip	$r_{slip} = k_{slip} \cdot \ C_{f,i} \cdot v_{f,xy}\ ^2$
Orientation	$r_{ori} = k_{ori} \cdot \ \phi_{body,xy}\ ^2$
Joint Torque	$r_\tau = k_\tau \cdot \ \tau\ ^2$
Joint Position	$r_q = k_q \cdot \ q - q_{nominal}\ ^2$
Action Rate	$r_{rate} = k_{rate} \cdot \ a_t - a_{t-1}\ ^2$
Energy Consumption	$r_{energy} = k_{energy} \cdot \ \dot{q} \cdot \tau\ $
Pose Deviation	$r_{pose} = k_{pose} \cdot \exp(-\ q - q_{default}\ ^2)$
Termination (Penalty)	$r_{termination} = k_{termination} \cdot \text{done}$
Stand Still (Penalty)	$r_{standstill} = k_{standstill} \cdot \ cmd_{v,xy}\ $
Linear Velocity in Z (Penalty)	$r_{lin\_z} = k_{lin\_z} \cdot \ v_z\ ^2$
Angular Velocity in XY (Penalty)	$r_{ang\_xy} = k_{ang\_xy} \cdot \ \omega_{x,y}\ ^2$

操控任务，与移动任务训练框架相似，观测及动作基于不同任务进行不同定义。

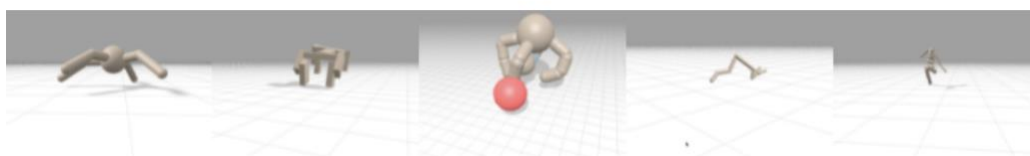
## 物理引擎

基于 GPU 的物理仿真，大部分机器人环境基于 PhysX GPU 实现的 IsaacGym, Isaac Lab 进行仿真现实迁移，少部分利用开源的 MJX, Brax, Warp, Taichi 物理引擎实现。

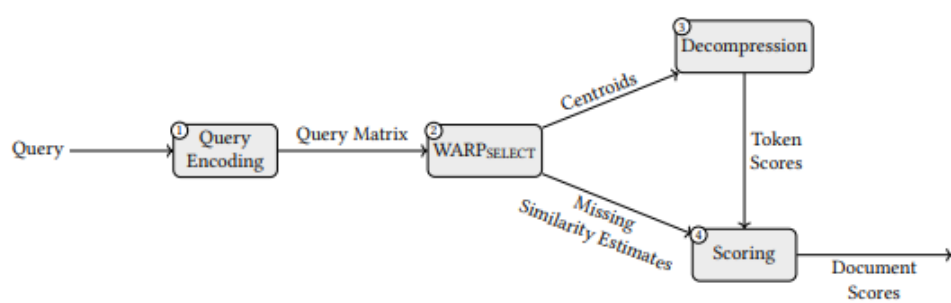
### 一、MJX



## 二、Brax



## 三、Warp

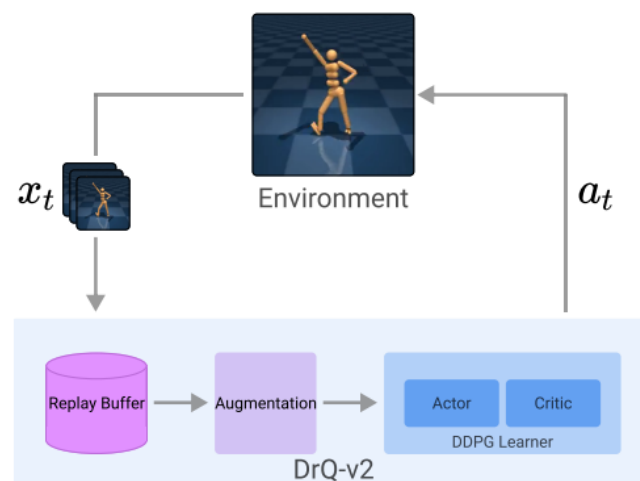


# 视觉强化学习

基于视觉的强化学习，DrQ, RAD, Dreamerv3, TD-MPC2 等。

## 一、DrQ(Data Regularized Q)

基于对输入进行扰动；对值函数及策略函数正则化的数据增强方法来改进强化学习算法。可直接通过像素学习，不用额外损失函数或预训练。

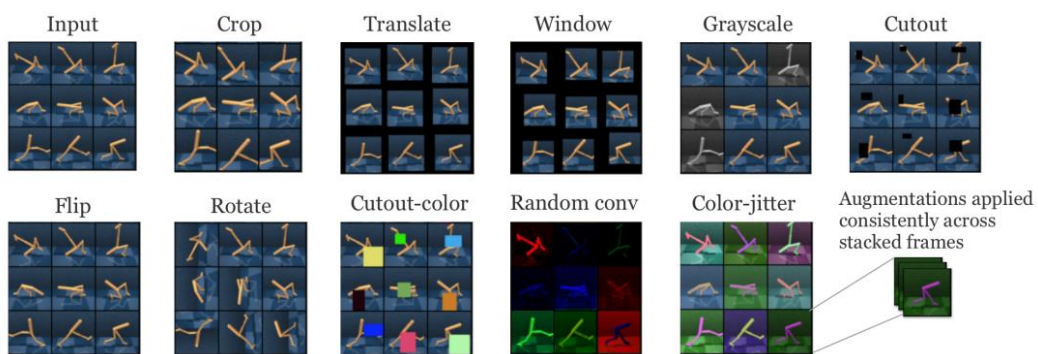


## 二、RAD(RL With Augmented Data)

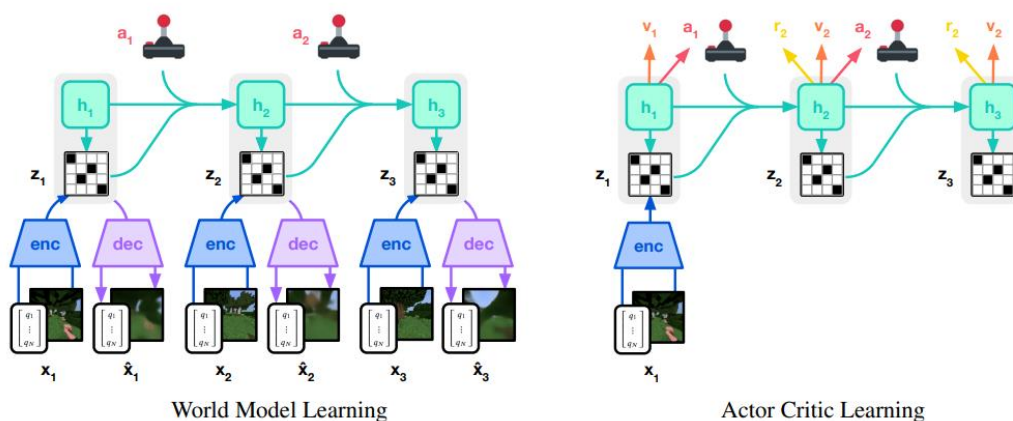
基于数据增强方法，改进数据有效性不足、不利模型泛化等问题。

基于像素(Pixel-Based)的强化学习算法数据增强方法，随机变换(Random Translate)、截取、色彩抖动、补丁裁剪、随机卷积。

基于状态(State-Based)的强化学习算法数据增强方法，随机尺度缩放(Random Amplitude Scale)、高斯噪声等。

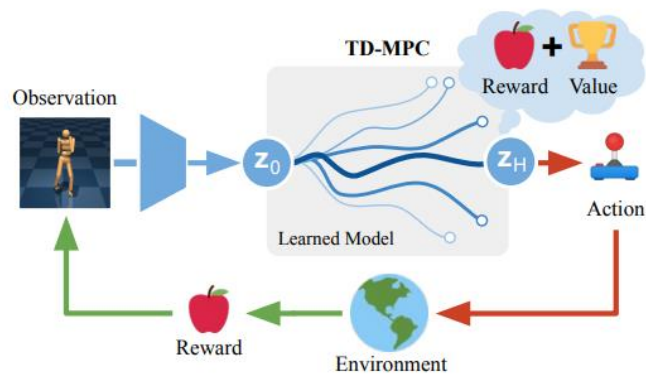


### 三、Dreamer



### 四、TD-MPC

时序差分-模型预测控制(TD-MPC)，基于时序差分联合学习隐动力学模型及价值函数，分别进行短时窗局部轨迹优化及长时窗奖励估计。



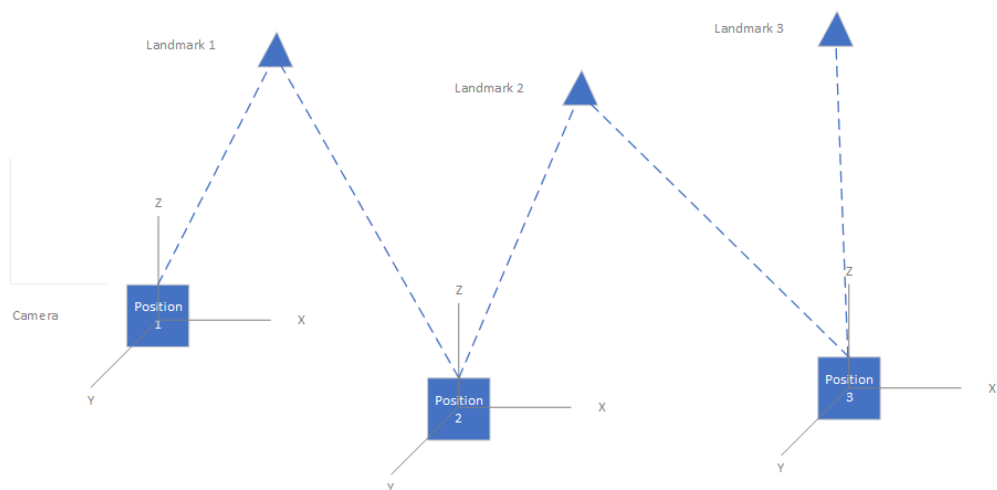
MuJoCo Playground，基于 JAX GPU 加速、DM Control Suite、Madrona 渲染器、Menagerie 模型、Actor-Critic 强化学习框架、MuJoCo 物理仿真引擎，利用不同数据增强方法等形成机器人仿真训练迁移平台。

同时，相对于更完备的物理仿真，应可整合三维重建、世界模型、超算、云计算等的发展以进行改进。

## 传统 SLAM 篇

基于视觉的机器人移动场景示例。

机器人在每个位置都会拍照，每个照片都会观测到一些路标或特征点。



传统 SLAM 的目标，准确估算相机在世界坐标系下移动轨迹的位姿及路标特征点位置；同时，建立地图。

假设，

在起始点，相机坐标系与世界坐标系重合。

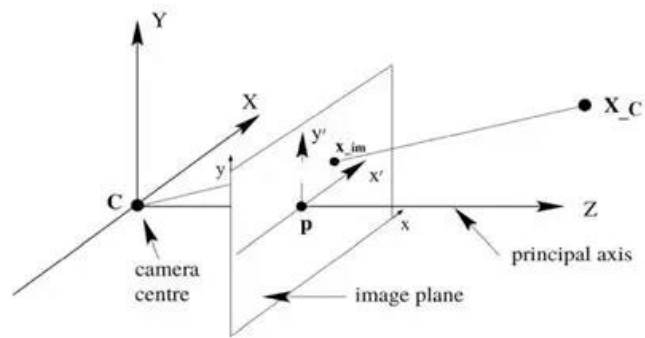
相机在相邻的两个位置总能观测到相同的路标。

可知，

基于惯性测量单元(IMU)等获得的加速度、旋转等信息，可通过旋转或变换矩阵估算相机在世界坐标系下不同时刻对应的位姿。

基于图像成像原理，可计算图像像素点对应目标特征点在相机或世界坐标系中的位置，或者，路标特征点在相机图像中的坐标。

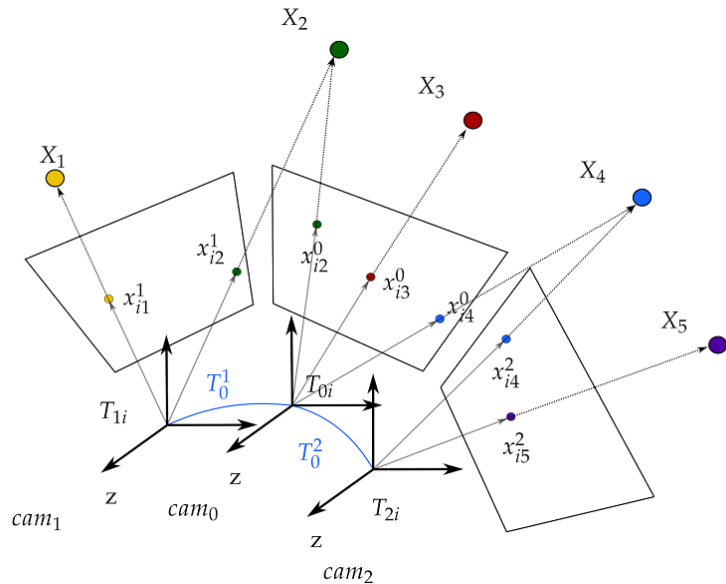
相机、图像平面及路标成像关系示例。



注，基于对称原理，图像平面可置于相机中心(Camera Center)与路标之间。

基于成像方程，对象、相机中心(位姿)、图像三要素，知道任意两要素，可计算第三要素。

相机移动过程中，相邻视频帧或图像与路标关系示例。



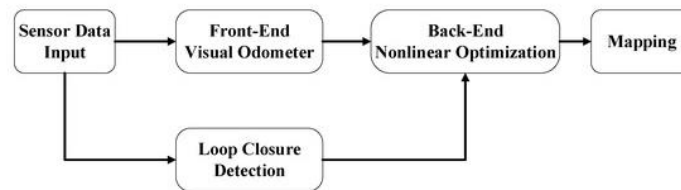
路标特征点可通过 SIFT 等方法人工提取。基于特征匹配，可计算相邻视频帧中的匹配特征点。

在现实中，相机运动信息的获取、路标特征点的计算都存在干扰，因此常通过目标函数或误差最小等约束条件来更正位姿、位置估算。

相机基于旋转或变换矩阵运算估计位姿，因此，对估计位姿的误差函数进行优化计算时，实际是对矩阵元素进行导数极值计算。

注，对矩阵导数计算无法像齐次方程一样能利用加法分配律，因此，常通过李群与李代数局部可逆的关系，使得对矩阵的导数计算转换成对向量导数的计算，以计算函数极值。

SLAM 的流程框架



前端视觉里程计模块为之前的推理计算步骤，后端优化模块为之前的优化步骤，增加的闭环检测模块在于机器人可能移动回之前的位置，此时不需再进行视觉里程计计算。

SLAM 运动与观测线性方程示例。

$$x_k = Ax_{k-1} + Bu_{k-1} + w_{k-1}$$

$$z_k = Cx_k + Du_k + v_k.$$

传统 SLAM 后端优化方法，BA、KF、EKF、PKF、UKF、MSCKF、GraphOptimization 等。



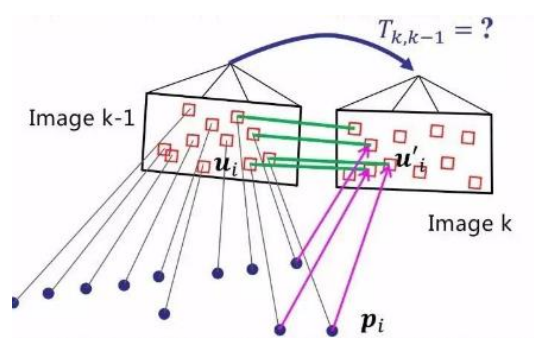
# 第一章. 光束平差(BA, Bundle Adjustment)

## 一、 算法思路

单目相机。基于世界坐标系、相机坐标系、图像坐标系(Image Framework)可以互相转换的关系，预测地标或特征(Landmarks or Features)在图像坐标系中的位置，该位置作为重投影与该特征在图像中的实际坐标存在误差(RPE, Reprojection Error)，对同一特征点在不同图像中的这些误差进行累加，然后进行优化计算，修正特征点实际坐标及相机位姿。

比方，世界坐标系下，相机从原点开始运动，基于相机坐标系、图像坐标系计算该点观测到的路标位置，同时，利用惯性单元(IMU)等里程计估算下一时刻相机位姿；在下一时刻，基于相机估算位姿，利用相机坐标系、图像坐标系计算前一时刻观测到的路标在当前图像中的坐标投影；相似方法，对同一路标，可估算多个图像坐标投影。基于路标图像投影与路标在图像中的真实坐标的误差和，进行优化计算，修正相机位姿及路标位置。

所谓重投影，就是基于各坐标系变换关系，通过计算认为场景中的特征应该投影在当前图像的哪个位置。



光束平差法与深度学习中的梯度下降算法类似，不同的是梯度下降算法更新的是神经网络模型参数，光束平差更新的是特征点三维坐标及相机位姿。

注，相似的方法，可基于相机起始位姿、图像及成像方程计算路标位置，相机在相邻位姿利用相同路标及成像方程计算当前位姿，与惯性测量单元估算的相机位姿进行误差计算。此方法或可称位姿重估法。

## 二、 优化过程

基于李群、李代数转换，实现误差函数对矩阵求导。

目标函数，

$$\min_{\mathbf{P}} J(\mathbf{T}) = \sum_{i=1}^N \|\mathbf{z}_i - \mathbf{P}\mathbf{X}_i\|_2^2$$

矩阵  $\mathbf{P}$  是投影矩阵，包含着相机位姿。

目标函数  $f(\mathbf{P})$  对  $\mathbf{P}$  求导，意味着对相机位姿的旋转矩阵或变换矩阵求导。

基于导数公式，

$$\frac{df(x)}{dx} = \lim_{\delta x \rightarrow 0} \frac{f(x + \delta x) - f(x)}{\delta x}$$

目标函数导数公式，

$$\frac{df(P)}{dP} = \lim_{\delta P \rightarrow 0} \frac{f(P + \delta P) - f(P)}{\delta P}$$

假定  $\mathbf{P}$  是旋转矩阵- $\text{SO}(3)$ 。因为， $\text{SO}(3)$ 群不满足加法封闭定律，也就是说  $\mathbf{P} + \delta \mathbf{P}$  不再是  $f(\mathbf{P})$  的自变量，这样就无法直接进行导数计算。因此，该公式这里需通过李代数转换来计算。

1)、旋转矩阵相关项的导数计算,

基于导数模式推导。

$$\begin{aligned}
\frac{\partial (\mathbf{RX})}{\partial \phi} &= \frac{\partial (\exp(\phi^\wedge) \mathbf{X})}{\partial \phi} \\
&= \lim_{\delta \phi \rightarrow 0} \frac{\exp((\phi + \delta \phi)^\wedge) \mathbf{X} - \exp(\phi^\wedge) \mathbf{X}}{\delta \phi} \\
&= \lim_{\delta \phi \rightarrow 0} \frac{\exp((\mathbf{J}_l \delta \phi)^\wedge) \exp(\phi^\wedge) \mathbf{X} - \exp(\phi^\wedge) \mathbf{X}}{\delta \phi} \quad (\text{BCH formula}) \\
&= \lim_{\delta \phi \rightarrow 0} \frac{(\mathbf{I} + (\mathbf{J}_l \delta \phi)^\wedge) \exp(\phi^\wedge) \mathbf{X} - \exp(\phi^\wedge) \mathbf{X}}{\delta \phi} \quad (\text{Exponential Map linearization}) \\
&= \lim_{\delta \phi \rightarrow 0} \frac{(\mathbf{J}_l \delta \phi)^\wedge \exp(\phi^\wedge) \mathbf{X}}{\delta \phi} \\
&= \lim_{\delta \phi \rightarrow 0} \frac{-(\exp(\phi^\wedge) \mathbf{X})^\wedge \mathbf{J}_l \delta \phi}{\delta \phi} \quad (\mathbf{a}^\wedge \mathbf{b} = -\mathbf{b}^\wedge \mathbf{a}) \\
&= -(\mathbf{RX})^\wedge \mathbf{J}_l
\end{aligned}$$

$\mathbf{R}$ , 旋转矩阵, 特定正交群  $\text{SO}(3)$ 、李群、三维流形元素。基于李群的平滑流形特性, 流形任意点可弯曲、连续可微, 因此, 流形中点与该点切空间中的元素存在对数、指数互逆映射。

旋转矩阵是流形中的点元素, 可通过相应的切空间元素的指数  $\exp(\phi^\wedge)$  表示。

基于扰动模式推导,

$$\begin{aligned}
\frac{\partial(\mathbf{RX})}{\partial \boldsymbol{\varphi}} &= \lim_{\boldsymbol{\varphi} \rightarrow 0} \frac{(\delta \mathbf{R})\mathbf{RX} - \mathbf{RX}}{\boldsymbol{\varphi}} \\
&= \lim_{\boldsymbol{\varphi} \rightarrow 0} \frac{\exp(\boldsymbol{\varphi}^\wedge) \exp(\boldsymbol{\phi}^\wedge) \mathbf{X} - \exp(\boldsymbol{\phi}^\wedge) \mathbf{X}}{\boldsymbol{\varphi}} \\
&= \lim_{\boldsymbol{\varphi} \rightarrow 0} \frac{(\mathbf{I} + \boldsymbol{\varphi}^\wedge) \exp(\boldsymbol{\phi}^\wedge) \mathbf{X} - \exp(\boldsymbol{\phi}^\wedge) \mathbf{X}}{\boldsymbol{\varphi}} \\
&= \lim_{\boldsymbol{\varphi} \rightarrow 0} \frac{\boldsymbol{\varphi}^\wedge \mathbf{RX}}{\boldsymbol{\varphi}} = \lim_{\boldsymbol{\varphi} \rightarrow 0} \frac{-(\mathbf{RX})^\wedge \boldsymbol{\varphi}}{\boldsymbol{\varphi}} = -(\mathbf{RX})^\wedge
\end{aligned}$$

## 2)、变换矩阵相关项的导数计算

基于扰动模式推导，

$$\begin{aligned}
\frac{\partial(\mathbf{TX})}{\partial \epsilon} &= \lim_{\epsilon \rightarrow 0} \frac{\exp(\epsilon^\wedge) \exp(\xi^\wedge) \mathbf{X} - \exp(\xi^\wedge) \mathbf{X}}{\epsilon} \\
&= \lim_{\epsilon \rightarrow 0} \frac{(\mathbf{I} + \epsilon^\wedge) \exp(\xi^\wedge) \mathbf{X} - \exp(\xi^\wedge) \mathbf{X}}{\epsilon} \\
&= \lim_{\epsilon \rightarrow 0} \frac{\epsilon^\wedge \exp(\xi^\wedge) \mathbf{X}}{\epsilon} \\
&= \lim_{\epsilon \rightarrow 0} \frac{\epsilon^\wedge \mathbf{TX}}{\epsilon} \\
&= \lim_{\epsilon \rightarrow 0} \frac{\epsilon^\wedge \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \mathbf{X}}{\epsilon} \\
&= \lim_{\epsilon \rightarrow 0} \frac{\begin{bmatrix} \boldsymbol{\varphi} & \delta \rho \\ \mathbf{0}^\top & 0 \end{bmatrix} \begin{bmatrix} \mathbf{RX} + \mathbf{t} \\ 1 \end{bmatrix}}{\epsilon} \quad (\text{Homogeneous coordinate}) \\
&= \lim_{\epsilon \rightarrow 0} \frac{\begin{bmatrix} \boldsymbol{\varphi}(\mathbf{RX} + \mathbf{t}) + \delta \rho \\ \mathbf{0}^\top \end{bmatrix}}{\begin{bmatrix} \boldsymbol{\varphi}, \delta \rho \end{bmatrix}^\top} \\
&= \begin{bmatrix} \mathbf{I} & -(\mathbf{RX} + \mathbf{t})^\wedge \\ \mathbf{0}^\top & \mathbf{0}^\top \end{bmatrix} \in \mathbb{R}^{4 \times 6} \\
&= (\mathbf{TX})^\odot
\end{aligned}$$

### 三、 SLAM 应用

ORB-SLAM、LOAM。

## 第二章. 卡尔曼滤波(KF, Kalman Filter)

卡尔曼滤波，核心在于两个误差、递归计算矩阵系数、迭代更新预测。

### 一、两个误差

基于运动方程预测的状态，再基于测量方程，形成测量量与状态预测量的误差。

基于状态迭代更新方程，形成预测的状态与更正后的状态量之间的误差。

更新误差等于测量预测误差的缩放。

因此，基于更新误差函数进行优化，可转换成对测量预测误差函数进行优化。

### 二、递归计算矩阵系数

测量预测误差函数是运动方程、测量方程中的矩阵系数的函数。

运动方程是时间递归方程。

因此，基于误差函数优化可推得矩阵系数的递归计算形式。

### 三、迭代更新预测

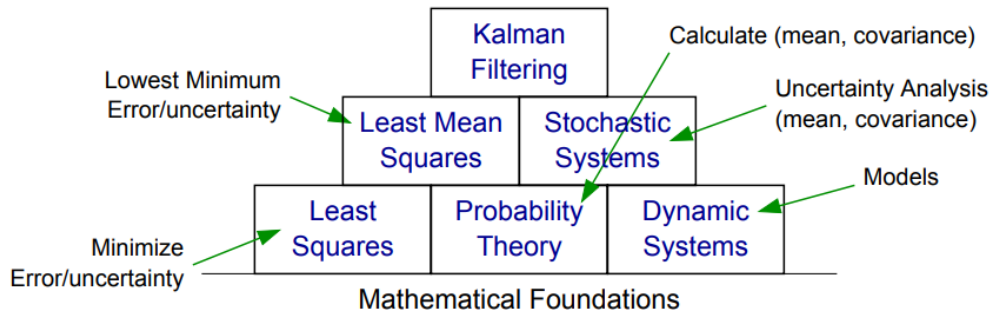
基于运动方程预测状态，再基于测量方程计算测量预测误差，然后利用该误差加权更新预测的状态，更新的状态基于运动方程进行下一时刻状态预测，以此形成迭代更新预测过程。

卡尔曼滤波，利用任何可用信息及带噪测量信息估计带噪动态系统(Dynamic System)状态或参数的优化递归数据处理算法(Optimal Recursive Data Processing Algorithm)。可用于目标追踪、导航控制、体重估计、电压测量等场景。

任何可用信息及带噪测量信息指，基于系统及测量设备的动力学知识，对系统噪声、测量误差、动力学方程不确定信息的统计，以及待估计量的任何可用起始信息。这意味着卡尔曼滤波适用于传感器数据混合。

滤波指，过滤掉噪声、从噪声中找到最优估计。

卡尔曼滤波的数学基础。



## 一、 算法思路

相较于深度学习等，依然是找到误差函数，进行最优化计算。

只是每个时步，预测函数的结果需要利用加权的测量误差进行校准更新后，再联合真实值定义误差。

使误差函数对加权参数导数为零以最小化误差，同时，基于预测函数的递归形式，可推得误差及加权参数的递归形式定义。

这样，每个时步只需利用误差及加权参数的递归公式便可校准更新预测结果。

## 二、 数学定义

算法略数学化的定义，假定基于多维向量定义的目标是线性运动的、起始状态已知、噪声呈高斯分布，则对照三维空间物体运动，可利用矩阵变换来定义目标运动方程及目标测量方程。然后利用目标测量结果与目标运动方程预测结果的差分，矩阵化加权后校正更新预测。加权矩阵可基于更新后的预测结果与真实值的差的协方差最小化来计算。

因此，卡尔曼滤波的核心就是基于运动方程及测量方程中的矩阵推得协方差矩阵及加权矩阵的定义。

### 三、 举例推导

已知，

1) 、小车的线性递归形式的运动方程，可用来预测小车当前时步(Time step)的状态，比方，位置、速度等；

$$x_k = Ax_{k-1} + Bu_{k-1} + w_{k-1}$$

$x_k$ ,  $k$  时步的预测状态;  $A$ , 变换矩阵;  $x_{k-1}$ ,  $k-1$  时步的已校准更新后的状态;  $B$ , 系统控制量的变换矩阵;  $u_{k-1}$ , 系统控制量;  $w$ , 噪声,  $\sim$ 正态分布  $Q$ 。

2) 、小车状态测量结果与运动方程预测结果的测量方程，可用来定义测量结果与预测结果的误差；

$$z_k = Hx_k + v_k$$

$z_k$ , 测量结果;  $H$ , 校准更新前的  $k$  时步状态  $x_k$  的变换矩阵;  $v$ , 噪声,  $\sim$ 正态分布  $R$ 。

基于已知信息，可利用运动方程预测的先验估计  $\hat{x}_k^-$  及测量方程对应的估计差分

$z_k - H\hat{x}_k^-$  来定义后验状态估计  $\hat{x}_k$ 。



$$\hat{x}_k = \hat{x}_k^- + K(z_k - H\hat{x}_k^-)$$

$K$  是估计差分  $z_k - H\hat{x}_k^-$  的权重，或增益(Gain)、混合因子。

后验状态估计  $\hat{x}_k$  代到后验估计误差  $e_k$  及协方差  $P_k$  的定义式。

$$e_k \equiv x_k - \hat{x}_k$$

$$P_k = E[e_k e_k^T]$$

使误差协方差对矩阵  $K$  的导数为零以计算最小协方误差。

基于该协方误差导数方程，可推得加权矩阵  $K$  的递归定义。

$$\begin{aligned} K_k &= P_k^- H^T (H P_k^- H^T + R)^{-1} \\ &= \frac{P_k^- H^T}{H P_k^- H^T + R} \end{aligned}$$

基于先验估计误差协方差及后验估计误差协方差定义，可推得相应的协方差递归定义。

先验估计误差  $e_k^-$  及协方差  $P_k^-$  的定义式。

$$e_k^- \equiv x_k - \hat{x}_k^-$$

$$P_k^- = E[e_k^- e_k^{-T}]$$

先验估计误差协方差  $P_k^-$  的递归定义。

$$P_k^- = A P_{k-1} A^T + Q$$

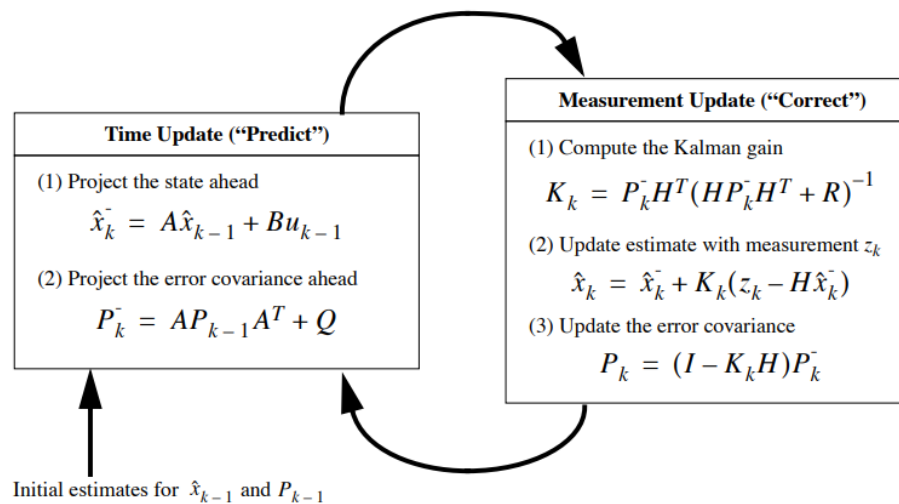
后验估计误差协方差  $P_k$  的递归定义。

$$P_k = (I - K_k H) P_k^-$$

## 四、 预测校准

这样，基于已知的运动方程、测量方程及推导得到的估计误差协方差矩阵、加权矩阵的递归

定义，可预测、校准更新每个时步的状态估计。



## 五、 推导方法

卡尔曼滤波方程的推导分两类，基于正交投影法(这是 KF 发表时用的方法，证明推导过程比较复杂)；基于贝叶斯最优滤波(推导过程相对简单，形成预测方程及更新方程，一般用该形式)

## 六、 SLAM 应用

在 SLAM 后端优化时，可基于惯性单元定义相机运动方程，基于 Lidar 定义测量方程。

## 第三章. 扩展卡尔曼滤波 (EKF, Extend Kalman Filter)

扩展卡尔曼滤波器与卡尔曼滤波区别在于，假定系统的运动方程、测量方程是非线性。

$$x_k = f(x_{k-1}, u_{k-1}, w_{k-1})$$

$$z_k = h(x_k, v_k)$$

运动方程、测量方程可用类似泰勒展开一阶导的形式展开进行近似。

### 一、 线性推导

函数在一点展开的泰勒展开一阶近似形式是线性的。

因此, 可以利用无噪声的运动方程、测量方程分别在  $(\hat{x}_{k-1}, u_{k-1}, 0)$  ,  $(\tilde{x}_k, 0)$  作展开点。

$$\tilde{x}_k = f(\hat{x}_{k-1}, u_{k-1}, 0)$$

$$\tilde{z}_k = h(\tilde{x}_k, 0)$$

使得非线性的运动方程、测量方程可以对照泰勒展开一阶形式展开。

$$x_k \approx \tilde{x}_k + A(x_{k-1} - \hat{x}_{k-1}) + Ww_{k-1}$$

$$z_k \approx \tilde{z}_k + H(x_k - \tilde{x}_k) + Vv_k$$

A, 无噪声运动方程对  $x$  的偏导的雅可比矩阵(Jacobian Matrix)。

$$A_{[i, j]} = \frac{\partial f_{[i]}}{\partial x_{[j]}}(\hat{x}_{k-1}, u_{k-1}, 0)$$

W, 无噪声运动方程对  $w$  的偏导的雅可比矩阵。

$$W_{[i, j]} = \frac{\partial f_{[i]}}{\partial w_{[j]}}(\hat{x}_{k-1}, u_{k-1}, 0)$$

H, 无噪声测量方程对  $\mathbf{x}$  的偏导的雅可比矩阵。

$$H_{[i,j]} = \frac{\partial h_{[i]}}{\partial x_{[j]}}(\tilde{x}_k, 0)$$

V, 无噪声测量方程对  $\mathbf{v}$  的偏导的雅可比矩阵。

$$V_{[i,j]} = \frac{\partial h_{[i]}}{\partial v_{[j]}}(\tilde{x}_k, 0)$$

## 二、 误差方程

预测误差及测量残差(Measurement Residual)。

$$\tilde{e}_{x_k} \equiv x_k - \tilde{x}_k$$

$$\tilde{e}_{z_k} \equiv z_k - \tilde{z}_k$$

非线性运动方程及测量方程的泰勒展开分别代到误差、残差方程。

$$\tilde{e}_{x_k} \approx A(x_{k-1} - \hat{x}_{k-1}) + \epsilon_k$$

$$\tilde{e}_{z_k} \approx H\tilde{e}_{x_k} + \eta_k$$

$$\epsilon_k, \quad WQW^T \quad , \quad \eta, \quad VRV^T \quad .$$

这些方程联立，可得非线性方程的后验状态估计。

$$\hat{x}_k = \tilde{x}_k + \hat{e}_k$$

$$\hat{e}_k, \quad \text{预测误差} \quad \tilde{e}_{z_k} \quad \text{的估计。}$$

同时， $\tilde{e}_{z_k}$ 、 $\epsilon$ 、 $\eta$  都是均值为 0 的正态分布。

基于正态分布运算规则，预测误差  $\tilde{e}_{z_k}$  的估计可以定义。

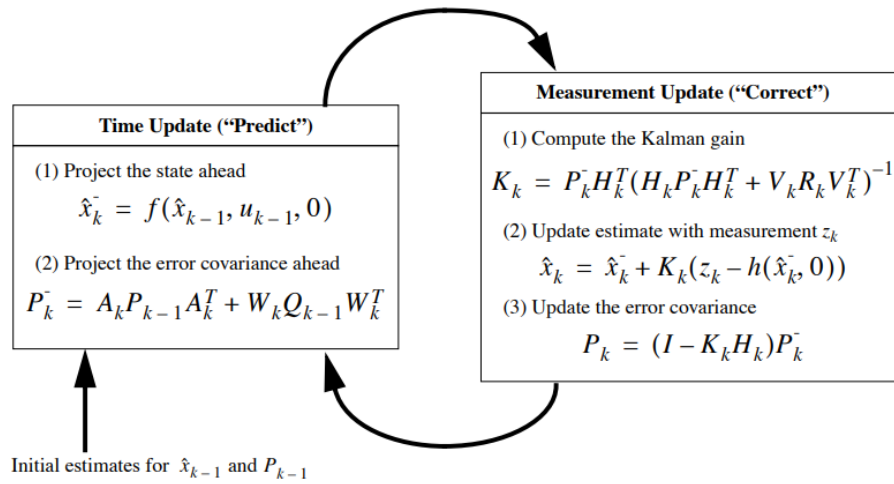
$$\hat{e}_k = K_k \tilde{e}_{z_k}$$

$K_k$ , 第  $k$  时步对应的卡尔曼滤波中的权重或增益。

这两个预测误差估计的方程联立，可得预测校准更新方程。

$$\begin{aligned}\hat{x}_k &= \tilde{x}_k + K_k \tilde{e}_{z_k} \\ &= \tilde{x}_k + K_k (z_k - \tilde{z}_k)\end{aligned}$$

### 三、 预测校准





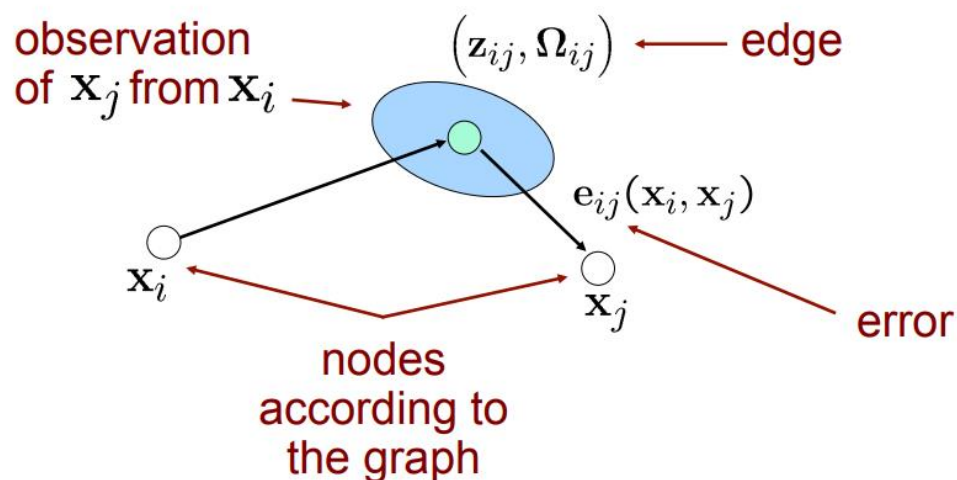
## 四、 SLAM 应用

GMapping、Point-LIO。

## 第四章. 图优化(Graph Optimization)

### 一、 优化原理

图优化的原理，依然是找到目标误差函数，进行最优化计算。



**Goal:** 
$$\mathbf{x}^* = \operatorname{argmin}_{\mathbf{x}} \sum_{ij} \mathbf{e}_{ij}^T \Omega_{ij} \mathbf{e}_{ij}$$

假定图中节点表示相机位姿或地标特征位姿，节点之间的边表示用作约束条件的测量值。

基于节点信息，利用世界坐标系、相机坐标系、图像坐标系相互转换关系预测节点之间的距

离或方位关系，然后与边约束信息( $z_{ij}$ )比对误差( $e_{ij}$ )，再进行误差平方计算( $e_{ij}^T \Omega_{ij} e_{ij}$ )。

所有存在着边约束关系的任意两点之间的误差平方的和( $\sum_{ij} e_{ij}^T \Omega_{ij} e_{ij}$ )作为要优化的目标误差函数(Least Squares)。

目标误差函数通过泰勒展开成线性形式进行优化计算。

## 二、 数学定义

相机位姿之间的误差，

$$e_{ij}(x_i, x_j) = \text{t2v}(Z_{ij}^{-1}(X_i^{-1}X_j))$$

$X_i, X_j$ , 节点  $i, j$  相应的变换矩阵(Transform Matrix)。

$Z_{ij}$ , 从节点  $i$  对节点  $j$  的观测。

$\text{t2v}$ , 变换矩阵到位姿向量(Pose Vector)的转换函数。

相机预测的地标位姿、观测的地标位姿之间的误差，

$$e_{ij}(x_i, x_j) = R_i^T(X_j - T_i) - z_{ij}$$

$R_i$ , 相机节点  $i$  的旋转矩阵。

$X_j$ , 地标节点  $j$  的平移矩阵(Translate Matrix)。

$T_i$ , 相机节点  $i$  的平移矩阵。

$z_{ij}$ , 基于相机节点  $i$  对地标节点  $j$  的观测。

最小平方和形式的目标函数。

$$x^* = \min_x F(x)$$

$$x^* = \operatorname{argmin}_x \sum_{ij} e_{ij}^T \Omega e_{ij}$$

或

$$F(x) = \sum_{k=1}^n e_k(x_k, z_k)^T \Omega_k e_k(x_k, z_k)$$

信息矩阵(Information matrix)  $\Omega$  是位姿的协方差矩阵的逆(Inverse Of The Covariance Matrix Of Poses), 编码着每条边的不确定信息(Uncertainty)。  $e$  是实际测量与估计测量之间的误差;

通过最小化  $F(x)$ , 找到最优定义测量量  $z$  的一系列位姿  $x$ 。

每个误差项进行近似泰勒展开。

$$e_{ij}(x + \Delta x) \cong e_{ij} + J_{ij} \Delta x$$

代到最小平方和目标函数, 可得相应的线性系统参数项。

$$\begin{aligned}\mathbf{b} &= \sum_{ij} \mathbf{b}_{ij} = \sum_{ij} \mathbf{J}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij} \\ \mathbf{H} &= \sum_{ij} \mathbf{H}_{ij} = \sum_{ij} \mathbf{J}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{J}_{ij}\end{aligned}$$

解线性系统项。

$$\Delta \mathbf{x} = \text{solveSparse}(\mathbf{H} \Delta \mathbf{x} = -\mathbf{b})$$

更新位姿。

$$\mathbf{x} = \mathbf{x} + \Delta \mathbf{x}$$

### 三、 优化算法

目标函数的优化算法可以用高斯-牛顿法 (Gauss-Newton)、列文伯格-马尔夸特法 (Levenberg-Marquardt)等。

### 四、 SLAM 应用

前端进行图创立，后端进行图优化。

Cartographer、LIO-SAM。



## 深度学习 SLAM 篇

基于深度学习的视觉 SLAM , DROID-SLAM、DeepVO、UnDeepVO、DeepFactors 等；激光 Lidar SLAM, Lo-Net、SHINE-Mapping 等。

CodeSLAM，基于可推理深度地图的自编码器编码后的隐变量进行优化。

Tandem，基于单目深度估计的预训练神经网络 MVSNet-Style 重建三维场景，通过执行 frame-to-model 光度追踪分解位姿/深度问题。

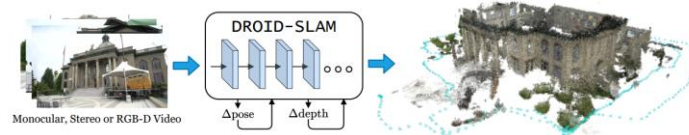
DROID-SLAM，利用稠密光流估计架构来解决视觉里程计问题，通过降采样深度地图再升采样来避免维度问题。

Volumetric Fusion[Rosinol]，利用边际协方差加权深度，在体素表征中进行融合，实现的网面(mesh)几何准确，但是光度细节不足。

NeRF-SLAM，基于 Volumetric Fusion 思路，用层级化体神经辐射场替代 TSDF 作为地图表征，改进了地图的光度准确度，同时允许并行优化位姿及地图。

更具体的说，NeRF-SLAM 用稠密单目 SLAM 提供的稠密深度地图、相机位姿、对深度及位姿的不确定估计来监督训练神经辐射场。用深度边际协方差加权的稠密深度损失作损失函数。

# 第一章. DROID-SLAM

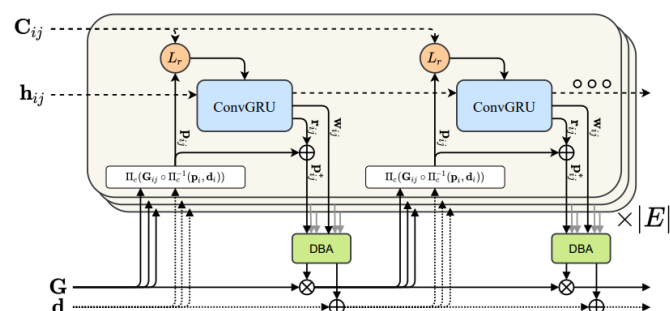


DROID(Differentiable Recurrent Optimization-Inspired Design), 基于单目、立体或 RGB-D 相机及深度学习的端到端可微分架构, 利用光流预测循环迭代校准投影预测, 再通过神经网络稠密光束平差层(DBA, Dense Bundle Adjustment Layer)预测优化相机位姿及像素(Pixelwise)深度, 实现实时稠密 SLAM。

## 一、 总体框架

相似于卡尔曼滤波测量方程加权运动方程更新预测, DROID 利用光流循环网络模型加权重投影函数更新预测; 预测更新后的像素栅格坐标, 基于神经网络稠密光束平差层预测更新相机位姿及像素深度。

光流预测模型基于 RAFT(Recurrent All-Pairs Field Transforms For Optical Flow) 循环迭代更新机制, 实现基于任意数量的视频帧迭代更新相机位姿及像素深度。





框架图，左下，起始(Init)或预测的相机位姿  $G$ 、像素深度  $d$ ，联合视频帧  $i$  的像素栅格坐标

$p_i$ ，基于逆投影  $\Pi_c^{-1}$  转换成三维点云坐标，左乘以相机在视频帧  $i$  的位姿到相机在视频帧

$j$  的位姿变换  $G_{ij}$ ，再进行投影变换  $\Pi_c$ ，得到视频帧  $i$  像素栅格  $p_i$  对应应在视频帧  $j$  中的坐标  $p_{ij}$ ；

左上，计算得到视频帧像素之间的关联信息  $C_{ij}$ ，基于关联查询器(Correlation Lookup Operator)  $L_r$  查询坐标  $p_{ij}$  关联的信息；

左中，查询到的关联信息连同隐状态  $h_{ij}$  输入卷积门循环单元(ConvGRU)，输出图像间光流  $r_{ij}$  及置信图  $w_{ij}$ ；

中下，光流模型门循环单元预测的光流加重投影预测的坐标  $p_{ij}$ ，连同置信图输入稠密光束平差层模块，预测相机位姿及像素深度变化，并更新  $G$ 、 $d$ ；

右侧，基于图像序列共视关系循环迭代更新相机位姿及像素深度。

## 1. 基本定义

### 1)、目标函数

相机位姿损失及光流损失函数的加权。

### 2)、输入表示(Representation)

基于图像序列创建的共视图的共识图像对作输入。

$I_{t=0}^N$ ，图像序列。

基于视频帧图(Frame-Graph)  $(V, E)$  表示图像之间的共视关系。

$(i, j) \in E$ ，边，表示图像  $i, j$  存在重合视域。

基于共视图的边  $(i,j) \in E$  确定的图像对序列作输入。

注，训练及推理时，动态创建帧图。

### 3)、输出表示

待预测或更新的变量。

$G_t \in SE(3)$  ，相机位姿，特定欧几里德群元素。

$d_t \in R_+^{H \times W}$  ，逆向深度(Inverse Depth)张量。

注，相机位姿或像素深度更新后，重新计算视频帧可视关系、更新帧图。

## 2. 重投影

计算视频帧的像素坐标栅格在共视视频帧中的稠密对应域(Dense Correspondence Field)  $p_{ij}$ 。

注，栅格作用在于便于描述协同区域、进行旋转变换。

重投影计算方程。

$$p_{ij} = \Pi_c(G_{ij} \circ \Pi_c^{-1}(p_i, d_i))$$

$p_i \in R^{H \times W \times 2}$  ， 视频帧 i 的像素坐标栅格。

$d_i$ , 逆深度图(Inverse Depth Map)。

$\Pi_c$ , 相机模型, 投影三维空间点到图像。

$\Pi_c^{-1}$ , 深度图及坐标栅格到三维空间点的逆投影。

$G_{ij} = G_j \circ G_i^{-1}$ , 相机位姿  $i$  到  $j$  的变换。

$p_{ij} \in R^{H \times W \times 2}$ , 视频帧  $i$  的像素坐标栅格在视频帧  $j$  中的重投影。

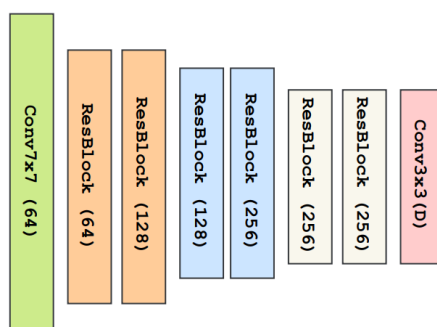
### 3. 修正投影

基于提取的图像像素特征创立共视图像素关联信息; 基于上下文特征、查找到的关联信息及以前的光流残差预测光流; 利用预测的光流修正重投影。

#### 1)、特征提取

基于相似框架分别提取图像特征及上下文(Context)特征。图像特征编码器应用于每个图, 上下文特征编码器只用于第一个图。

特征提取网络框架。



框架图，6 个参差块，3 个下采样层使得每两个参差块的分辨率减半，生成的稠密特征图分辨率为输入图像的  $1/8$ 。

## 2)、像素关联

### 体张量

基于帧图中的每条边 $(i, j)$ ，通过特征编码器提取视频帧特征， $g_{\theta}(I_i)$ 及  $g_{\theta}(I_j)$ 。

基于帧特征之间的特征向量的点积计算得到特征的 4 维关联体张量(4D Correlation Volume)，

$$C(g_{\theta}(I_1), g_{\theta}(I_2)) \in R^{H \times W \times H \times W}$$

。

体张量的每个元素的定义。

$$C_{u_1 v_1 u_2 v_2}^{ij} = \langle g_{\theta}(I_i)u_1 v_1, g_{\theta}(I_j)u_2 v_2 \rangle$$

### 关联金字塔

四维体张量  $R^{H \times W \times H \times W}$  的后两维度进行池化计算，形成 4 个级别(4-Level)的关联金字塔。逻辑意义，视频帧  $i$  的像素与视频帧  $j$  的不同感知域(Receptive Field)的关联度。

### 关联查询

$$L_r : R^{H \times W \times H \times W} \times R^{H \times W \times 2} \rightarrow R^{H \times W \times (r+1)^2}$$

逻辑意义，以每个元素为中心、曼哈顿距离(L1 Distance) $r$  作半径的坐标栅格(Grid of

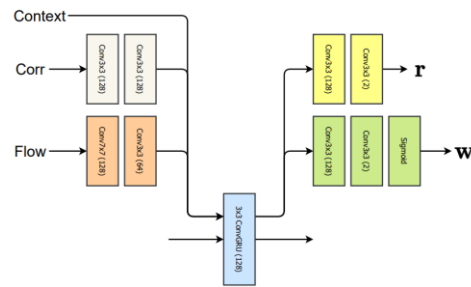
Coordinates,  $R^{H \times W \times 2}$ ), 在关联体中查询元素的邻域(Local Neighborhood,

$(r + 1)^2$ )的关联信息, 形成坐标栅格查询特征( $R^{H \times W \times (r+1)^2}$ )。

相似的查询方法应用于关联金字塔的每个关联体, 得到相应的查询特征。所有查询特征拼接形成关联特征图。

### 3)、更新器(Update Operator)

更新器框架。



框架图, 上下文特征、关联特征及估计的光流的特征的拼接作卷积门循环单元的输入, 生成的隐状态基于两个卷积层分别映射为光流域(Flow Field)及相关的置信图(Associated Confidence Map)。

修订的流域(Revision Flow Field),  $r_{ij} \in R^{H \times W \times 2}$

相关的置信图,  $w_{ij} \in R_+^{H \times W \times 2}$

修正后的投影,  $p_{ij}^* = r_{ij} + p_{ij}$

## 4. 预测位姿及深度变化

基于稠密光束平差层(DBA, Dense Bundle Adjustment Layer)映射修正为相机位姿及像素深度。

1)、帧图的代价函数(Cost Function)

$$E(G', d') = \sum_{(i,j) \in E} p_{ij}^* - \Pi_c(G'_{ij} \circ \Pi_c^{-1}(p_i, d'_i))^2_{\Sigma_{ij}}$$

$$\Sigma_{ij} = \text{diag} w_{ij}$$

$\|\cdot\|_{\Sigma}$  , 距离(Mahalanobis distance)。

2)、方程计算

基于局部参数化(Local Parameterization)线性化代价函数方程，利用高斯-牛顿法(Gauss-Newton Algorithm)进行一阶泰勒展开解相机位姿及像素深度变化  $(\Delta \xi, \Delta d)$ 。

海森矩阵(Hessian Matrix)简化后的块对角形式(Block Diagonal Structure)。

$$\begin{bmatrix} B & E \\ E^T & C \end{bmatrix} \begin{bmatrix} \delta \epsilon \\ \delta d \end{bmatrix} = \begin{bmatrix} v \\ w \end{bmatrix}$$

### 3)、互补解计算

相机位姿及像素深度变化基于互补形式(Schur Complement)计算解。

$$\Delta \xi = [B - EC^{-1}E^T]^{-1}(v - EC^{-1}w)$$

$$\Delta d = C^{-1}(w - E^T \Delta \xi)$$

## 5. 更新预测

更新的相机位姿、像素深度。

$$G^{(k+1)} = \text{Exp}(\Delta \xi^{(k)}) \circ G^{(k)}, d^{(k+1)} = \Delta d^{(k)} + d^{(k)}$$

迭代更新位姿及深度，收敛到固定点或最接近真实位姿及深度的点，

$$G^{(k)} \rightarrow G^*, d^{(k)} \rightarrow d^*$$

## 二、 训练

SLAM 系统基于 PyTorch 实现，利用 LieTorch 扩展在群元素(Group Elements)的切空间(Tangent Space)执行反向传播。

### 1. 计量自由(Gauge Freedom)

在单目环境下，神经网络只能基于相似变换(Similarity Transformation)计算获得相机轨迹。计量自由会严重影响线性系统的条件及训练时的梯度稳定性。

解决方法，使损失函数对相似变换保持不变；同时，固定前两个位姿为每个训练序列的真实位姿。固定第一个位姿可移去计量的 6 个自由度(三维空间平移、旋转)，固定第二个位姿可去掉缩放自由度。

### 2. 训练视频

每个训练示例基于一组 7 帧的视频。这样的一系列视频形成训练集。

对长度  $N_i$  的视频  $V_i$ ，计算两两帧之间的距离，形成  $N_i * N_i$  的距离矩阵。矩阵元素为每对视频帧之间的平均光流量(Optical Flow Magnitude)。

对于重合度低于 50% 的视频帧对，距离设置为无限(Infinity)。

训练时，通过在距离矩阵中采样动态生成视频，以便相邻视频帧的平均光流介于 8px 到 56px 之间。

### 3. 监督

神经网络的训练基于位姿损失及光流损失函数的加权。



光流损失，基于预测的深度及位姿信息推导相邻帧之间的光流，与真实深度及位姿推导的光流进行比对。

位姿损失，基于整个视频帧图序列的真实位姿  $T_i^N$  与预测位姿  $G_i^N$  之间的距离。

$$L_{pose} = \sum_i ||Log_{SE3}(T_i^{-1} \cdot G_i)||_2$$

$T_i^N$ ，真实的相机位姿。

$G_i^N$ ，预测的相机位姿。

光流损失，基于整个视频帧图序列的真实光流与预测光流的距离。

$$\mathcal{L} = \sum_{i=1}^N \gamma^{N-i} ||\mathbf{f}_{gt} - \mathbf{f}_i||_1$$

### 三、 推理

视频流作输入，实时进行重建及定位。系统异步运行两个线程，前端线程接收新视频帧，提取特征，选取关键帧，执行局部光束平差。后端线程同时基于整个关键帧序列执行全局光束平差。

## 1. 初始化

积累光流大于 16px 的 12 个视频帧；创建关键帧之间的边，初始化帧图(Frame Graph)；运行 10 次更新器(Update Operator)。

## 2. 前端

基于输入的视频流，维持一系列关键帧及存储着共视关键帧之间边的帧图。对关键帧位姿及深度进行优化。

从输入的帧中提取特征。然后，添加新的视频帧到帧图中。同时基于平均光流，在图中添加 3 个与之最相邻的帧的边。

位姿基于线性运动模型初始化，然后，迭代执行多次更新器以更新关键帧位姿及深度。

对新的视频帧追踪后，基于视频帧对之间的平均光流距离，移去冗余帧。无合适帧时，移去最早的关键帧。

## 3. 后端

基于整个关键帧，执行全局光束平差。每次迭代过程中，用关键帧之间的光流距离重建帧图。

先添加时域相邻的关键帧之间的边，然后依照光流矩阵中光流增加的顺序，采样新的边。

对于每个选取的边，抑制距离在 2 以内的相邻边。

距离基于车氏距离(Chebyshev Distance)定义。

$$\|(i, j) - (k, l)\|_{\infty} = \max(|i - k|, |j - l|)$$

对整个帧图应用更新器。帧图常包含有数千帧及边，基于 RAFT 方法进行内存管理。

注，可用绝对轨迹误差(ATE, Absolute Trajectory Error)评估 SLAM 系统准确度。

## 四、 优化计算

计算重投影函数关于相机位姿及像素深度的雅可比。

### 1. 投影函数

#### 1)、坐标表示

齐次坐标系(homogeneous coordinates)表示三维空间点，便于投影空间计算。

$$X = (X, Y, Z, W)^T$$

$(x/w, y/w, z/w)$ 对应三维空间点坐标。

#### 2)、重投影

基于扭曲函数(Warping Function)，帧 i 中的深度为 d 的点 p 反向投影(Reprojected)到帧 j 中。

$$p' = \Pi_c(G_{ij} \bullet \Pi^{-1}(p, d)) G_{ij} = G_j \circ G^{-1}$$

### 3)、相机投影

针孔相机投影函数(Pinhole Projection Function)。

$$\Pi_c(x) = \begin{bmatrix} f_x \frac{X}{Z} + c_x \\ f_y \frac{Y}{Z} + c_y \end{bmatrix}$$

### 4)、逆投影

$$\Pi_c^{-1}(p, d) = \begin{bmatrix} \frac{p_x - c_x}{f_x} \\ \frac{p_y - c_y}{f_y} \\ 1 \\ d \end{bmatrix}$$

相机内参， $c = (f_x, f_y, c_x, c_y)$

## 2. 投影及逆投影函数的雅可比

$$\frac{\partial \Pi_c(\mathbf{X})}{\partial \mathbf{X}} = \begin{pmatrix} f_x \frac{1}{Z} & 0 & -f_x \frac{X}{Z^2} & 0 \\ 0 & f_y \frac{1}{Z} & -f_y \frac{Y}{Z^2} & 0 \end{pmatrix} \quad \frac{\partial \Pi_c^{-1}(\mathbf{p}, d)}{\partial d} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}$$

## 3. 三维点变换的雅可比

局部参数化相机位姿， $e^{\xi_i} G_i$  及  $e^{\xi_j} G_j$ ；深度视作向量。

三维点变换  $\mathbf{X}'$  的表示。

$$\mathbf{X}' = \text{Exp}(\xi_j) \cdot \mathbf{G}_j \cdot (\text{Exp}(\xi_i) \cdot \mathbf{G}_i)^{-1} \cdot \mathbf{X} = \text{Exp}(\xi_j) \cdot \mathbf{G}_j \cdot \mathbf{G}_i^{-1} \cdot \text{Exp}(-\xi_i) \cdot \mathbf{X}$$

利用伴随算子(Adjoint Operator)移动  $\xi_i$  项到表示项的前端。

$$\mathbf{X}' = \text{Exp}(\xi_j) \cdot \text{Exp}(-\text{Adj}_{\mathbf{G}_j \mathbf{G}_i^{-1}} \xi_i) \cdot \mathbf{G}_j \cdot \mathbf{G}_i^{-1} \cdot \mathbf{X}$$

计算点变换的雅可比矩阵。

$$\frac{\partial \mathbf{X}'}{\partial \xi_j} = \begin{pmatrix} W' & 0 & 0 & 0 & Z' & -Y' \\ 0 & W' & 0 & -Z' & 0 & X' \\ 0 & 0 & W' & Y' & -X' & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

$$\frac{\partial \mathbf{X}'}{\partial \xi_i} = - \begin{pmatrix} W' & 0 & 0 & 0 & Z' & -Y' \\ 0 & W' & 0 & -Z' & 0 & X' \\ 0 & 0 & W' & Y' & -X' & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \cdot \text{Adj}_{\mathbf{G}_j \mathbf{G}_i^{-1}}$$

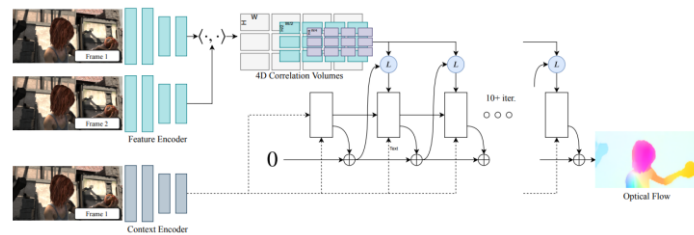
基于链式规则，计算重投影函数关于各变量的雅可比。

$$\frac{\partial \mathbf{p}'}{\partial \xi_j} = \frac{\partial \Pi_c(\mathbf{X}')}{\partial \mathbf{X}'} \frac{\partial \mathbf{X}'}{\partial \xi_j}, \quad \frac{\partial \mathbf{p}'}{\partial \xi_i} = \frac{\partial \Pi_c(\mathbf{X}')}{\partial \mathbf{X}'} \frac{\partial \mathbf{X}'}{\partial \xi_i}$$

$$\frac{\partial \mathbf{p}'}{\partial d} = \frac{\partial \Pi_c(\mathbf{X}')}{\partial \mathbf{X}'} \frac{\partial \mathbf{X}'}{\partial \mathbf{X}} \frac{\partial \Pi^{-1}(\mathbf{p}, d)}{\partial d} = \frac{\partial \Pi_c(\mathbf{X}')}{\partial \mathbf{X}'} = \frac{\partial \Pi_c(\mathbf{X}')}{\partial \mathbf{X}'} \begin{pmatrix} t_x \\ t_y \\ t_z \\ 1 \end{pmatrix}$$

## 五、 RAFT(Recurrent All-Pairs Field Transformers)

基于神经网络方法直接预测光流。提取每个像素对应的特征向量，计算图像帧像素对之间的关联体(Correlation Volume)，利用循环门单元预测、更新光流。



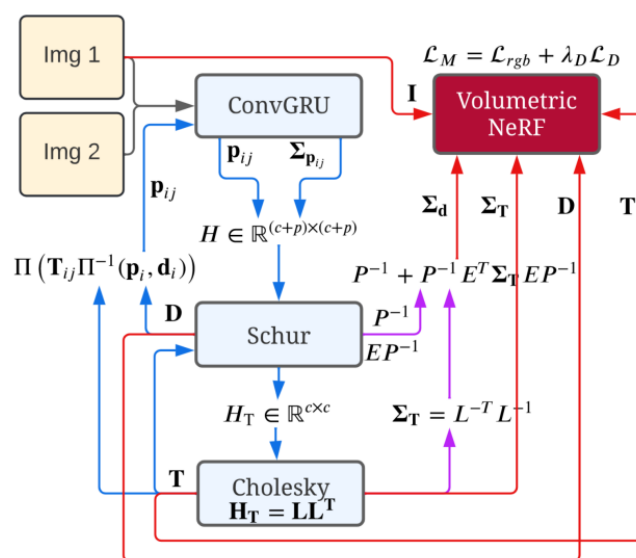
浅析，结合特征点及光流优点，基于神经网络光流模型预测加权重投影预测，利用神经网络光束平差循环预测更新相机位姿及像素深度实现实时、健壮、准确的稠密 SLAM，是对利用传统方法与深度学习方法改进 SLAM 的很好探索。

## 第二章. NERF-SLAM

NeRF-SLAM，基于单目稠密 SLAM 系统估计的位姿及深度图，通过置信加权，监督训练哈希层级表征的神经辐射场，实现定位追踪及光学、几何准确的三维重建。

### 一、 总体框架

NeRF-SLAM 基于 Instant-NGP 的哈希层级表征方法拟合神经辐射场，利用 DROID-SLAM 系统估计的相机位姿、像素深度图及深度置信(Uncertainty)进行色彩、深度对齐，优化模型。实际实现时，分别基于追踪线程(Tracking Thread)生成关键帧，优化光束平差，计算位姿、深度及置信；基于建图线程(Mapping Thread)进行建图、渲染及展示。



框架图，左侧，相似于 DROID-SLAM 方法，基于起始(Init)或预测的相机位姿、像素深度，利用逆投影、投影变换计算视频帧  $i$  到  $j$  的光流  $\mathbf{p}_{ij}$ ；再结合 RAFT 的卷积门循环单元(ConvGRU, Convolutional GRU)预测更新光流  $\mathbf{p}_{ij}$  及相应的权重  $\Sigma_{\mathbf{p}_{ij}}$ ；基于视频帧图(Frame Graph)的光束平差法代价函数利用高斯-牛顿法进行一阶泰勒展开，得到包括相机位姿矩阵及像素深度矩

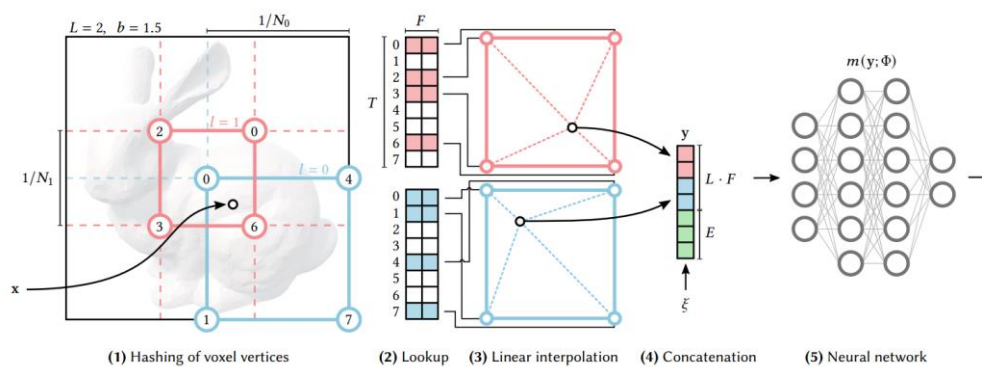
阵的稀疏块状的海森矩阵(Block-Sparse Hessian) $H$ ；利用块状矩阵的互补方法(Schur Complement)计算相机位姿矩阵  $HT$ ；矩阵分解(Cholesky factorization)为下三角矩阵(Lower-Triangular Matrix)与上三角矩阵(Upper-Triangular Matrix)积；利用前代、回代算法(Front And Back Substitution)分别计算下、上三角矩阵，得到相机位姿；基于相机位姿计算像素深度；

右侧，下方，基于矩阵互补及分解计算，可分别得到像素深度、相机位姿的边缘协方差(Marginal Covariances)  $\Sigma d$ 、 $\Sigma T$ ；

右侧，上方，基于协方差及预测的像素深度  $D$ 、相机位姿  $T$  定义哈希层级表征的神经辐射场的损失函数，进行优化训练；

## 二、 表征

相对于 NeRF 原有的对三维空间采样点方位每个坐标分量进行位置编码，通过神经网络实现坐标到色彩、密度映射的方法，哈希层级表征坐标有利于减少多层感知器网络的大小，提高计算性能。



基于二维图像示例，左侧，图像可划分为  $L$  个级别、不同分辨率的栅格；坐标  $x$  存在于不同级别的栅格的单元(Cell)中；对  $x$  的坐标分量分别计算上边界、下边界，形成 4 个坐标对(Pairs)；每个坐标对的分量进行加权异或、模  $T$  计算，得到 4 个独立或重合的哈希表索引；

居中，基于索引找到 4 个维度  $F$  的特征量表项；基于特征量，进行双线性插值(Bi-Linear Interpolation)，计算坐标  $x$  的特征量坐标；每个坐标  $x$  对应的  $L$  个特征量坐标与额外的维度  $E$  的视野方向、纹理等信息进行拼接，形成维度  $L \cdot F + E$  的编码；

右侧，编码传给多层感知器网络计算；训练优化时，损失梯度反向传播过多层感知器、拼接、线性插值，基于特征量(Looked-Up Feature Vectors)进行累加(Accumulated)。



注，三维空间进行体格划分；相应的坐标对数乘倍 ( $2^d$ )；基于三线性插值 (Trilinear Interpolation) 计算特征量坐标。

## 三、 追踪

### 1. 光流更新

基于视频帧对关系、投影计算的光流，计算更新的光流及权重。

$$\sum p_{ij}$$

光束平差法的最小方差代价函数基于高斯牛顿法展开为一阶泰勒级数，得到稀疏块状海森矩阵。

$$H \in R^{(C+P) \times (C+P)}$$

海森矩阵分区 (Block-Partition) 展开，利用互补 (Schur Complement)、矩阵分解 (Cholesky Decomposition) 等方法计算位姿、深度更新。

基于 DROID-SLAM 的迭代框架计算更新光流及后序的位姿、深度。

### 2. 更新计算

计算相机位姿、像素深度更新。

海森矩阵进行分区。

$$H\mathbf{x} = \mathbf{b}, \quad i.e. \quad \begin{bmatrix} C & E \\ E^T & P \end{bmatrix} \begin{bmatrix} \Delta\boldsymbol{\xi} \\ \Delta\mathbf{d} \end{bmatrix} = \begin{bmatrix} \mathbf{v} \\ \mathbf{w} \end{bmatrix}$$

H, 海森矩阵。

b, 残差。

C, 相机位姿矩阵。

P, 像素逆深度对角矩阵。

E, 相机/深度的非对角块矩阵。

$\Delta\boldsymbol{\xi}$ , 相机位姿特定欧几里德群 SE(3)的李代数更新(Delta Updates)。

$\Delta\mathbf{d}$ , 像素逆深度的更新。

$\mathbf{v}$ , 位姿残差。

$\mathbf{w}$ , 深度残差。

海森矩阵 H 关于 P 的互补(Schur Complement)H/P。

$$(H/P)\Delta\boldsymbol{\xi} = [C - EP^{-1}E^T] \Delta\boldsymbol{\xi} = (\mathbf{v} - EP^{-1}\mathbf{w})$$

基于矩阵分解(Cholesky Decomposition)相机位姿矩阵,  $H^T = LL^T$ 。

通过前向、反向替代计算位姿更新 $\Delta\boldsymbol{\xi}$ 。

基于位姿更新计算深度更新 $\Delta\mathbf{d}$ 。

### 3. 置信计算

基于海森块状矩阵方程，利用同样的互补、分解法，计算位姿、深度图的边缘协方差(Marginal Covariances)或不确定性(Uncertainty)、置信度(Confidence)。

$$\begin{aligned}\Sigma_d &= P^{-1} + P^{-T} E^T \Sigma_T E P^{-1} \\ \Sigma_T &= (L L^T)^{-1}\end{aligned}$$

$\Sigma_d$ ， 逆深度图的边缘协方差。

$\Sigma_T$ ， 位姿的边缘协方差。等于海森矩阵 H 关于 P 的互补 H/P 的逆。

### 四、 建图

DROID-SLAM 的稠密深度图噪声很大，不适合用来监督神经网络的三维体空间。因此，可利用色彩损失及深度损失的加权重作建图目标函数。

$$\mathcal{L}_M(\mathbf{T}, \Theta) = \mathcal{L}_{\text{rgb}}(\mathbf{T}, \Theta) + \lambda_D \mathcal{L}_D(\mathbf{T}, \Theta)$$

$\mathcal{L}_M$ ， 建图损失函数。

$\mathcal{L}_{\text{rgb}}$ ， 色彩损失函数。

$\mathcal{L}_D$ ， 深度损失函数。

$\lambda_D$ ， 平衡色彩及深度损失的超参。

深度损失函数。

$$\mathcal{L}_D(\mathbf{T}, \Theta) = \|D - D^*(\mathbf{T}, \Theta)\|_{\Sigma_D}^2$$

$D^*$ , 基于神经辐射场渲染的深度;

$D$ , 追踪模块估算的深度, 作对齐真值;

$\Sigma_D$ , 估算的置信度, 对  $D-D^*$  的平方进行加权;

渲染深度定义, 相似于标准体渲染方法, 基于像素射线方向采样三维空间位置, 预测相应的体密度及透明度, 进行累积透过率(Accumulated Transmittance)加权。

$$d^* = \sum_i \mathcal{T}_i \left(1 - \exp(-\sigma_i \delta_i)\right) d_i$$

$d_i$ , 射线方向采样点  $i$  的深度。

$\delta_i$ , 连续采样点之间的距离。

$\sigma_i$ , 多层感知器计算的采样点  $i$  的体密度。

$\mathcal{T}_i$ , 射线方向、采样点  $i$  前的透过率。

$$\mathcal{T}_i = \exp\left(-\sum_{j<i} \sigma_j \delta_j\right)$$

渲染色彩定义, 同 NeRF。

$$\mathcal{L}_{\text{rgb}}(\mathbf{T}, \Theta) = \|I - I^*(\mathbf{T}, \Theta)\|^2$$

$I$ ， 图像。

$I^*$ ， 渲染的图像。计算方法与渲染深度相似，采样点深度  $d_i$  替换为色彩  $c_i$ 。

## 五、 训练

环境，RTX 2080 Ti GPU，11G 内存，PyTorch，CUDA。

追踪线程，基于活跃的关键帧时窗优化计算光束平差重投影误差；在当前帧与此前关键帧的平均光流大于设定的阈值时，生成新的关键帧；生成新关键帧时，向建图线程发送图像帧、当前关键帧的相机位姿、估计的深度图、深度边缘协方差。

建图线程对收到的所有关键帧进行优化，同时进行渲染展示；建图基于 Instant-NGP 方法，同时修改添加建图损失 LM。

浅析，NeRF-SLAM 基于哈希层级表征坐标；基于矩阵分区、互补、分解计算置信度，改进 DROID-SLAM 稠密深度信息，优化对齐，提高场景光学及几何准确度。

## 第三章. 3D Gauss Splatting SLAM

3D Gaussian Splatting SLAM，基于三维空间类椭圆球体的三维高斯位置及形状泼溅或投影到相机图像平面，计算像素颜色、深度信息与观测的真值进行对齐优化，以更新相机位姿，调整三维空间三维高斯的稠密度、形状等。

### 一、 三维重建-Gaussian Splatting

#### 1. 总体框架

基于运动重建(SfM, Structure From Motion)及多视觉(MVS, Multi-View Stereo)工具 COLMAP，利用相机图像序列，大略计算相机位姿及三维空间点云，基于点云初始化三维高斯的位置均值、形状协方差、色彩及不透明度(Opacities)等。

基于这些信息对三维空间场景进行栅格化投影，预测图像像素，与真实图像像素进行对齐优化。

优化时，更新三维高斯参数；同时每更新一定次数时，对三维空间三维高斯进行剪枝(Prune)或稠密化(Densification)。

#### 2. 栅格化(Rasterization)算法

基于相机图像定义视圆锥体或梯形金字塔区域。

基于世界坐标系、相机坐标系、射线空间坐标系、图像坐标系变换关系，对金字塔区域的三

维高斯进行渲染，得到在二维图像平面的椭圆均值及协方差。变换过程中，维护协方差矩阵的半正定性(PSD, Positive Semi-Definition)。

图像分割为多个  $H \times W$  像素的瓦片，每个瓦片可能对应多个高斯泼溅椭球投影。利用投影对应的深度和瓦片索引作键值(key)，对相应的三维高斯排序。因此，每个瓦片像素可对应一系列高斯，利用这些高斯的颜色、透明度加权计算像素颜色。

### 3. 像素颜色合成

$$C_p = \sum_{i \in N} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$

## 二、 3D Gaussian Splatting SLAM

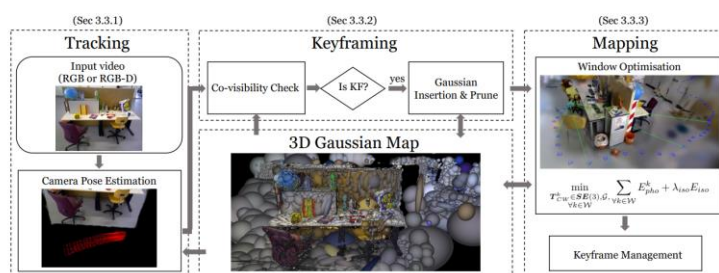
3DGS SLAM 基于三维高斯泼溅，改进实现在线 SLAM。

1、基于栅格化(Rasterization Pipeline)优化时，利用相机位姿李群关于三维高斯的解析雅可比进行计算，以更新相机位姿及场景几何结构。同时，禁用球谐波(Spherical Harmonics)表示三维高斯色彩，以降低计算量。

2、在轨迹追踪及建图时，使三维高斯维持各向同性(Isotropic)，以免连续(Continuous)SLAM 三维稠密重建(Incremental 3D Dense Reconstruction)时，三维高斯在视线方向过度拉伸(Elongation)带来模糊遮挡噪点(Artefacts)。

3、基于高斯资源分配及剪枝方法使三维重建几何特性明确(Clean)、相机轨迹追踪准确。

### 三、 总体框架



框架图，左侧(Tracking)，基于相机帧图，估计相机位姿，实例化三维高斯；居中(Keyframing)，基于相机帧图共视特性，提炼关键帧；右侧，基于栅格化图像损失函数进行优化计算，更新三维高斯参数；居中(3D Gaussian Map)，剪枝、稠密化三维高斯，更新相机位姿估计。

#### 1. 投影变换(Projective Transformation)

世界坐标系下的三维高斯与二维图像平面中的二维高斯，基于投影变换的关系。

$$\mu_I = \pi(T_{CW} \bullet \mu_W), \Sigma_I = JW \Sigma_W W^T J^T$$

$\mu$ ，图像平面三维高斯投影均值。

$\Sigma$ ，图像平面三维高斯投影协方差。



$\pi$ , 投影计算。

$$T_{CW} \in SE(3)$$

,相机坐标系到世界坐标系变换，三维高斯位姿。

$\mu_w$ , 三维高斯位置均值。

$\Sigma_w$ , 三维高斯形状协方差。

$J$ , 投影变换的线性近似的雅可比矩阵。

$W$ ,  $T_{CW}$  的旋转分量。

## 2. 相机位姿与三维高斯位姿导

图像平面投影高斯均值及协方差关于三维高斯位姿  $T_{CW}$  的偏导。

$$\frac{\partial \Sigma_I}{\partial T_{CW}} = \frac{\partial \Sigma_I}{\partial J} \frac{\partial J}{\partial \mu_C} \frac{D\mu_C}{DT_{CW}} + \frac{\partial \Sigma_I}{\partial W} \frac{DW}{DT_{CW}}$$

$$\frac{\partial \mu_i}{\partial T_{CW}} = \frac{\mu_I}{\mu_C} \frac{D\mu_C}{DT_{CW}}$$

已知流形导数定义。

$$\frac{Df(T)}{DT} = \lim_{\tau \rightarrow 0} \frac{\text{Log}(f(\text{Exp}(\tau) \circ T) \circ f(T)^{-1})}{\tau}$$

$$T \in SE(3)$$

$$\tau \in se(3)$$

因此，相机位置均值关于三维高斯位姿的导数。

$$\frac{D\mu_C}{DT_{CW}} = [I \quad -\mu_C^x]$$

$$\frac{DW}{DT_{CW}} = \begin{bmatrix} 0 & -W_{:,1}^x \\ 0 & -W_{:,2}^x \\ 0 & -W_{:,3}^x \end{bmatrix}$$

## 四、 追踪

### 1. 光学残差

单视(Monocular)相机，基于光学残差(Photometric Residual)损失函数优化计算，更新相机位姿。

$$E_{pho} = ||I(G, T_{CW}) - \bar{I}||_1$$

### 2. 几何残差

具备深度观测信息时，可基于几何残差(Geometric Residual)加权定义损失函数。

$$E_{geo} = ||D(G, T_{CW}) - \bar{D}||_1$$

$D(G, T_{CW})$ ，深度栅格化(Depth Rasterization)。

$D^-$ ，观测的深度。

基于  $\alpha$  混合栅格化(Rasterized)的像素深度定义。

$$D_p = \sum_{i \in N} z_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$

### 3. 目标函数

$$\lambda_{pho} E_{pho} + (1 - \lambda_{pho}) E_{geo}$$

$\lambda_{pho}$ ，权重超参。

## 五、 关键帧(Keyframing)

基于当前视频帧与最近关键帧三维高斯共视(Covisibility)，选观测到的高斯交并比大于特定阈

值的作新的关键帧。

基于相机图像射线方向，用作栅格化、透明度大于阈值的三维高斯标记作可视。

基于每一视频关键帧，稠密化三维高斯；对在别的多视频帧中可观测到，在最近的三个视频关键帧中的新的三维高斯进行剪枝。

## 六、 建图

### 1. 各向同性残差

三维高斯各向同性残差  $E_{iso}$ ，基于椭圆柱轴向拉伸参数  $s_i$  与椭圆柱均值的差定义。

$$E_{iso} = \sum_{i=1}^{|G|} ||s_i - \tilde{s}_i \cdot 1||_1$$

在损失函数中，基于各向同性调整项 (Isotropic Regularisation) 可使三维高斯球状化 (Sphericity)，以免视线方向拉伸带来噪点。

### 2. 目标函数

$$\min_{T_{CW}^k \in SE(3), g, \forall k \in W} \sum_{\forall k \in W} E_{pho}^k + \lambda_{iso} E_{iso}$$

### 3. RGB-D

具备深度观测信息时，损失函数中可加权 Egeo 项。

浅析，三维空间点实例、特性及渲染方法等是光稠 SLAM 基本要素。

Gaussian Splatting SLAM 基于稀疏三维高斯，利用高斯投影、栅格化像素进行对齐，实时更新三维高斯及相机位姿。存在帧率不高、大规模场景可能不适用等问题。

NeRF-SLAM 基于 DROID-SLAM 估计的相机位姿及深度信息，改进空间点表征，优化场景光学及几何准确度。存在的空射线渲染问题，应可利用图像语义优化。

基于稠密及光学对齐 SLAM 的发展，利用三维空间点及图像，设计端到端 SLAM，或者，基于扩散模型等方法，利用图像序列调控预测三维空间高斯位置等应是可能的。

## 三维重建篇

三维重建可以分为传统三维建模、基于深度学习的三维建模；静态三维重建、动态三维重建；物体重建、场景重建、人体动态重建；室内三维重建、室外三维重建；离线三维重建，在线实时三维重建；基于单目、多目、深度相机、激光等的三维重建等。

利用深度学习学习三维形状的四类方法。

- 1、基于点，神经网络直接预测三维点坐标。
- 2、基于网面，基于对二维平面进行变形来描述三维表面。不过，对于表面光滑连续，支持任意形状，不能兼顾。
- 3、基于体素，基于三维体素栅格表示形状，栅格存放占据信息或符号距离函数信息。
- 4、基于连续隐函数，学习连续的隐函数来表示形状。比方 occupancy network, PiFU 用连续指示函数(indicator function)来指定目标形状占据的是哪些三维空间。DeepSDF 用符号距离场(SDF, Signed Distance Field)来近似目标形状。

# 第一章. 截取符号距离(TSDF)

这里以室内三维地图重建简单分析一下 Kinect 的基于 RGBD 的三维重建原理。Kinect 的三维重建主要用到 TSDF 算法，即截断符号距离函数(Truncated Signed Distance Function)。在分析 TSDF 算法之前，先直观来看看 Kinect 三维重建的原理，以免在分析算法时只见树木不见森林。

## 一、三维表面重建

这里三维重建特指三维表面重建，实现三维表面重建主要需知道表面每个点的三维坐标。在知道每个点的三维坐标后，则需知道每个点的颜色。因此，三维重建问题可以表述为这样的数学问题：

已知场景中物体表面点与相机的距离；

已知相机坐标系与世界坐标系关系；

已知相机移动时的位姿变换关系；

求解物体三维表面模型？

求解过程

1. 一帧深度图像即是相机深度传感器在某一刻获取到的相机距离物体表面点的距离集合；由此可知此刻感知到物体表面点在相机坐标系中的坐标；

2. 由于已知相机坐标系与世界坐标系关系，因此可知此刻感知到的物体表面点在世界坐标系中的坐标；此时可以认为这些物体表面点的三维模型已建立，只是还不精细，对于整个物体而言还不全面；

3. 当相机移动时，参考前 2 步骤可以获得新的物体表面点的三维模型；这两个三维表面模型有不同部分，有相同部分，此时需要像全景照相一样对照两个模型的相同部分将不同的两部分拼接起来；这个对照过程称为配准(Registration)，对照与拼接的整个过程称为融合(Integration)；

4. 对于配准过程，想象一下，相机围绕着一个物体曲面拍照，站在微积分的角度，物体曲面



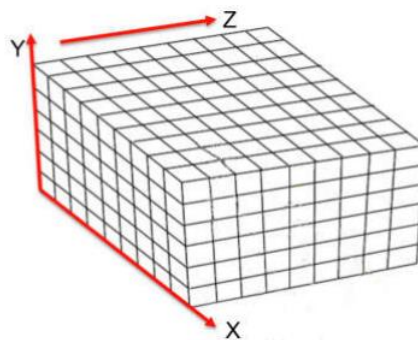
由很多微小的  $\Delta$  平面形成；相机拍摄的一系列深度图像与该  $\Delta$  平面对应的部分越平行，与该  $\Delta$  平面的法线越垂直则对于该  $\Delta$  平面的描述越准确，即描述该  $\Delta$  平面的深度图像的部分数据的权重越大，否则越小；经过这样的配准、融合，物体表面的三维重建模型越准确精细；这个过程主要用到 TSDF 算法。注：如果相机是绕着柱形物体转一周获取深度信息，则可以建立物体的柱形表面模型；如果相机只是移动经过物体，则只是获取物体部分表面深度信息。

5.当物体的三维表面模型建立好之后，为了更精细的表现模型，可以用等值面算法(Marching Cube)来重构三维表面模型；

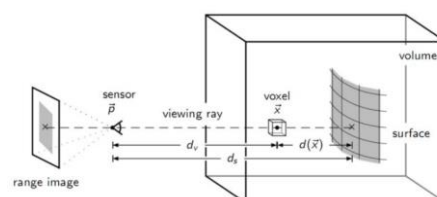
6.当三维表面模型建立好之后，用光线投影算法，可以计算三维模型的表面与 RGB 图像的哪些像素点对应，这样可以将这些像素点颜色信息与三维表面点对应，以实现三维表面模型渲染；

## 二、TSDF 算法浅析

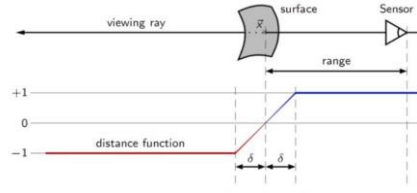
TSDF 算法中，先假设将 Depth 或 Lidar 等传感器探测到的周边物体框在一个立方体中。



该立方体被划分为多个小立方体，这些小立方体称为体素(Voxel)。想象一下，从相机传感器引出射线直达已重建的物体三维表面，在该射线上最接近物体表面的体素可用来近似描述物体表面点。



该图中， $d(v)$ 为深度传感器探测到的距离，该距离位置  $x$  作为体素位置； $d(x)$ 为体素距离物体表面的距离，以射线与物体表面相交的点为基准，体素在物体表面之前时， $d(x)$ 为正，体素在物体表面之后时， $d(x)$ 为负。



其中， $\delta$  为截取范围。

依照 TSDF 算法，深度图像数据与已重建的物体三维表面数据进行比对得到配准数据。

注：对  $\delta$  范围内的 TSDF 归一化，超出  $\delta$  范围的数据，标记为 1 或 -1。

TSDF 融合公式。

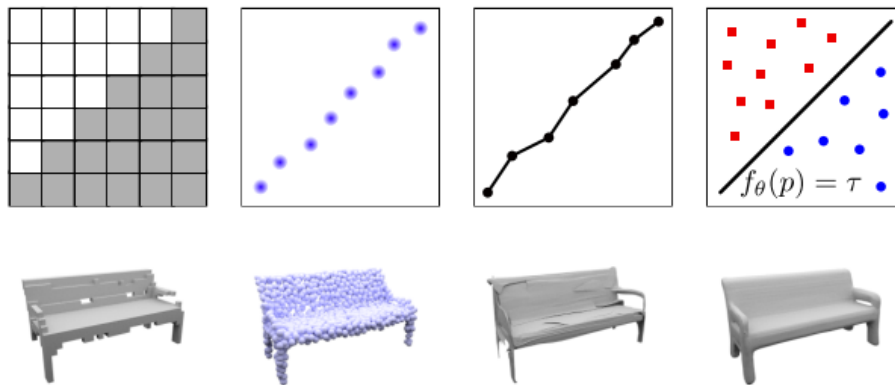
$$TSDF_{i+1}(p) = \frac{W_i(p)TSDF_i(p) + w_{i+1}(p)tsdf_{i+1}(p)}{W_i(p) + w_{i+1}(p)}$$

$$W_{i+1}(p) = W_i(p) + w_{i+1}(p)$$

## 第二章. 占据网络(Occupancy Networks)

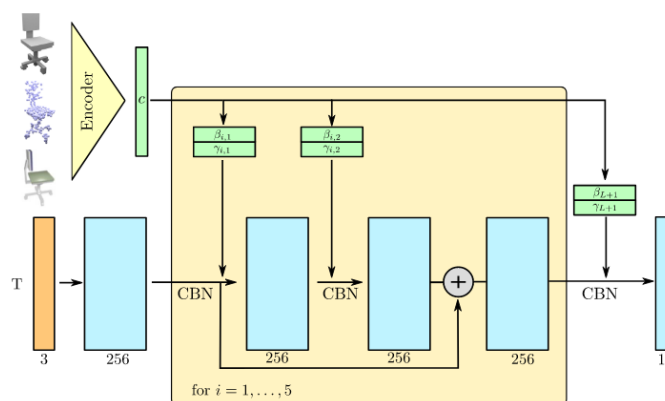
占据网络(Occupancy Networks), 通过直接学习连续三维占据函数(Occupancy Function)实现目标三维几何重建的方法。

基于深度神经网络对连续三维空间点占据情况进行预测, 使目标三维表面(3D Surface)得以隐式的表示作神经网络分类器的连续决策边界(Continuous Decision Boundary)。



### 一、 模型框架

基于观测特征量调控基于 ResNet-Block 的神经网络以实现三维空间点占用概率的预测。同时, 模型框架可用于三维模型生成。



框架图左侧，输入，三维空间点集  $T=\{p_i\}$ ，基于图像、点云或体素编码器(Encoder)提取的特征。

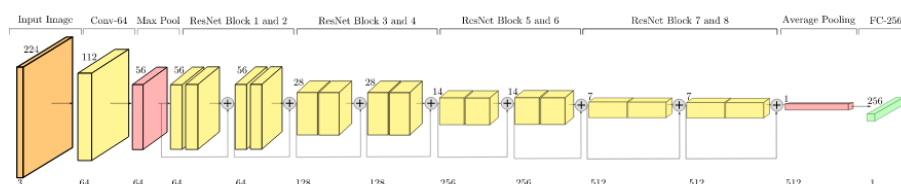
居中，三维空间点基于全连接层生成空间点特征向量，执行 5 次 **ResNet-Block**，通过条件批归一层(CBN，Conditional Batch Normalization Layer)、ReLU、全连接层及 Sigmoid 激活函数进行计算。

右侧，输出，三维空间点占据概率。

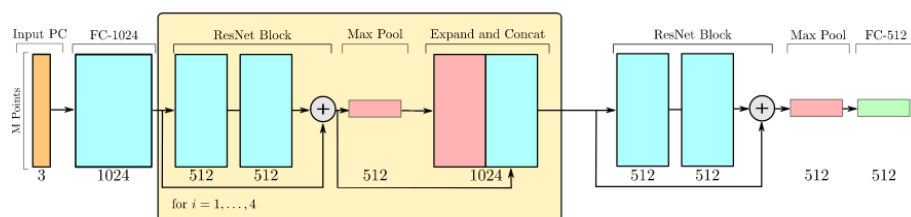
## 1. 调控模型

不同的观测类型对应不同的编码器(Encoder)框架。

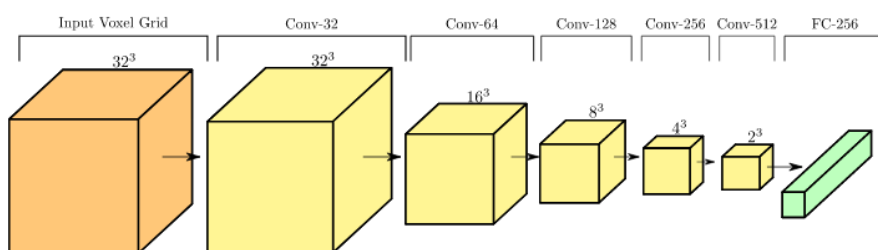
单视图输入类型，基于 **ResNet** 改进的框架。



点云输入类型，基于多个 **PointNet ResNet-Blocks** 改进的框架。



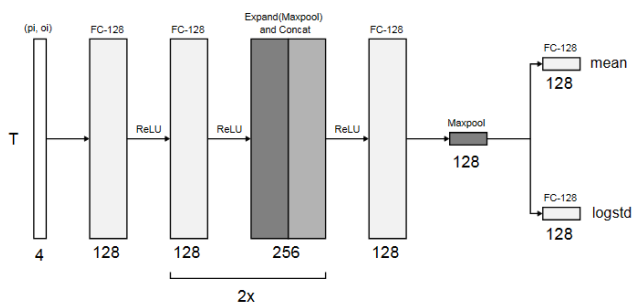
体素输入类型，基于多个三维卷积神经网络的框架。



## 2. 生成模型(Unconditional Mesh Generation Model)

基于变分自编码器(VAE, Variational AutoEncoder)框架，变分编码器(Variational Encoder)学习三维空间信息隐变量分布，采样的隐变量与三维空间点通过变分解码器(Variational Decoder)预测三维空间点占据概率。

变分编码器框架。



## 二、 模型定义

$$f_{\theta} : \mathbb{R}^3 \times \mathcal{X} \rightarrow [0, 1]$$

### 1. 可调控模型

输入,  $(p, x)$ , 三维空间任意点  $p$  与图像或点云等观测特征量  $x$ 。

输出, 任意点  $p$  的占据概率。

### 2. 生成模型

输入,  $(p, z)$ , 三维空间任意点  $p$  与三维空间信息隐变量分布的采样  $z$ 。

输出, 任意点  $p$  的占据概率。

## 三、 模型训练

## 1. 可调控模型

损失函数定义。

$$\mathcal{L}_{\mathcal{B}}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} \sum_{j=1}^K \mathcal{L}(f_{\theta}(p_{ij}, x_i), o_{ij})$$

$\mathcal{L}$ ，交叉熵损失函数。

$\mathcal{B}$ ，训练批次。

$i$ ，一个训练批次中的第  $i$  个样本。

$K$ ，一个样本中随机均匀采样的目标三维空间点数量。

$j$ ，一个样本中随机采样的第  $j$  个三维空间点。

$p_{ij}$ ，第  $i$  个样本的第  $j$  个三维空间点。

$x_i$ ，第  $i$  个样本对应的观测量。

$O_{ij}$ ，三维空间点  $p_{ij}$  占据真值。

## 2. 生成模型

损失函数定义。

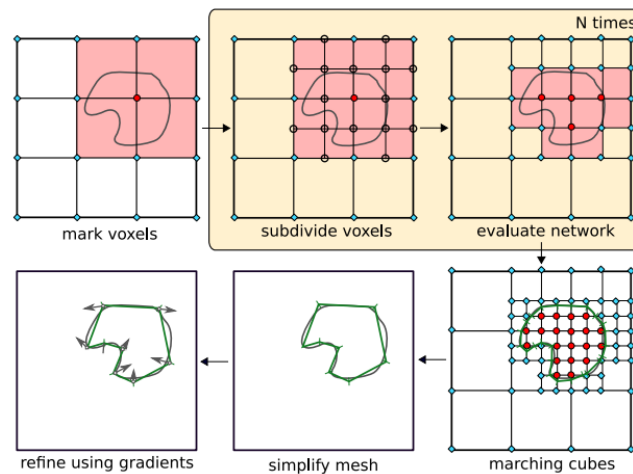
$$\mathcal{L}_{\mathcal{B}}^{\text{gen}}(\theta, \psi) = \frac{1}{|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} \left[ \sum_{j=1}^K \mathcal{L}(f_{\theta}(p_{ij}, z_i), o_{ij}) + \text{KL}(q_{\psi}(z | (p_{ij}, o_{ij})_{j=1:K}) \| p_0(z)) \right]$$

$z$ ，三维空间点隐变量。

$p_0(z)$ ，隐变量的先验分布(高斯分布)。

KL，KL-Divergence，隐变量的散度(相对熵)。

## 四、 模型推理



基于多分辨率等值面提取(MISE，Multiresolution IsoSurface Extraction)算法逐步逼近、提取目标三维表面。

### 1. 体素划分

- 1)、基于既定分辨率对三维空间进行体素划分。
- 2)、基于占据网络模型(Occupancy Network)预测栅格点占据情况。
- 3)、对于预测占据概率大于一定阈值的占据点标记红圆圈，未占据点标记青色圆圈。
- 4)、对至少存在两个占据情况不同的相邻栅格点、与网面(Mesh)相交的体素标记为活跃。比



方，存在红圆圈及青圆圈的体素标记为浅红色。

5)、基于更高的分辨率对标记活跃的体素进行亚体素划分。

6)、标记空圈的新的栅格点的占据标记、体素标记方法与上述步骤相似。

7)、依上述步骤，直到获得期望的分辨率。

## 2. 网面提取

基于 Marching Cubes 算法提取网面。

## 3. 网面简化

基于 Fast-Quadric-Mesh-Simplification 算法简化网面。

## 4. 网面优化

从获得的网面的每一面随机采样  $K$  个点，利用梯度信息优化网面。

$$\sum_{k=1}^K (f_{\theta}(p_k, x) - \tau)^2 + \lambda \left\| \frac{\nabla_p f_{\theta}(p_k, x)}{\|\nabla_p f_{\theta}(p_k, x)\|} - n(p_k) \right\|^2$$

$n(p_k)$ ，网面在  $p_k$  点的法线向量。

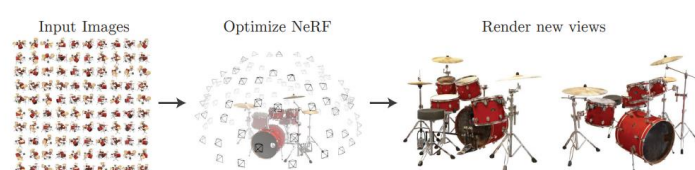
$\lambda = 0.01$ 。

法线向量损失基于双反向传播(Double-Backpropagation)计算。

占据网络(Occupancy Network), 基于深度神经网络直接学习连续三维空间点占据网络, 以实现高分辨率三维重建, 改进了传统基于图像、点云或体素的方法。

## 第三章. 神经辐射场(NeRF)

NeRF, 基于辐射场概念、光线可逆及相机成像原理, 利用二维图像, 通过多层感知器(MLP, Multilayer Perceptrons)神经网络对相机视域三维空间点位姿及相应的颜色、体密度(Volume Density)的映射函数进行拟合, 利用函数预测的泛化能力实现不同视角的视图的合成。

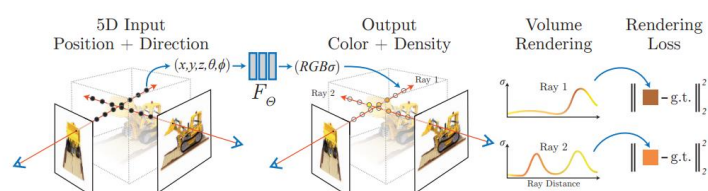


- 1、核心, 误差函数, 预测的图像与真实的图像进行比对。本质与光束平差法有些相似, 对多个相机位姿观测与预测的误差和进行优化计算。区别在于, 优化训练获得的神经网络具有泛化能力, 可以推理生成别的相机位姿看到的空间目标。
- 2、预测图像的方法, 基于相机位姿沿光线方向定义空间目标; 再对定义的目标进行体渲染计算, 得到预测的图像。
- 3、具体, 不同的相机位姿输入到 MLP 网络, 输出空间目标点的颜色及密度(控制透明度)。对光线方向的空间目标点颜色及密度进行积分或和计算, 得到渲染图像。
- 4、MLP 实际分两部分, 一部分以相机坐标为输入, 输出体密度及特征向量; 另一部分, 以该特征向量及相机光线方向作输入, 输出颜色。
- 5、为了 MLP 可以更好的表示目标几何及颜色变化剧烈的高频区域, 可通过对坐标及方向分别进行类似蛋白质结构预测、Transformer 架构中的位置变换的高维变换来改进。
- 6、为提高渲染效率, 规避目标空间空闲区及遮挡区, 分别利用一个粗神经网络、一个细神经网络来表示场景。先选取采样片段, 对粗神经网络的输出进行渲染评估, 找到目标体相关区域, 筛选掉那些无目标体的图像区; 再选取一些采样片段, 基于所有的采样片段, 对细神经网络的输出进行渲染评估。

# 一、 模型框架

基于辐射场及光线可逆原理，将三维空间物体反射光看作是相机光源发射的、辐射到或穿过物体的光。基于图像(像素)平面、成像平面与相机坐标系的变换关系，相机光源发射的光线方向可由相机光心与图像平面像素在成像平面位置的连线确定，称为相机光线方向、三维空间点视线方向( Viewing Direction)。

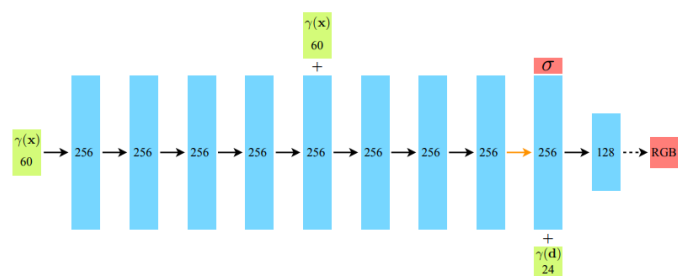
## 1. NeRF 框架



框架图，左侧，沿着相机光线方向进行三维空间点坐标 $(x, y, z, \theta, \phi)$ 采样；居中，基于多层感知器得到三维空间采样点颜色及体密度；右侧，基于体渲染(Volume Rendering)流程，对相机光线方向三维空间采样点颜色及密度信息进行加权和计算，得到光线在二维图像平面投影，对光线投影得到的二维图像与真实图像的方差进行优化计算，更新多层感知器网络参数。

## 2. 多层感知器框架

基于多个全连接层，对三维空间采样点坐标位置及方向分量的位置编码， $\gamma(\mathbf{x})$ 与 $\gamma(\mathbf{d})$ ，进行特征计算，得到采样点体密度及颜色。



框架图，左侧，三维空间采样点坐标位置向量  $\gamma(x)$  通过一系列全连接层进行特征提取；居中， $\gamma(x)$  通过跳连(Skip Connection)与全连接层激活进行拼接(Concatenation)；右侧，通过不带激活的全连接层得到体密度及特征向量，特征向量与三维空间采样点坐标方向向量  $\gamma(d)$  进行拼接，通过全连接 ReLU 层及全连接 Sigmoid 层得到采样点的 RGB 颜色。

## 二、 体渲染

基于体渲染方法渲染穿过三维场景的相机光线的颜色。

### 1. 相机光线定义

$$r(t) = o + td$$

$o$ ，相机位置。

$t$ ，相机光线方向三维空间点相对相机的距离。

$d$ ，相机视线方向  $(\theta, \phi)$  对应的笛卡尔三维坐标系的单位向量表示。

### 2. 相机光线颜色定义

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt, \text{ where } T(t) = \exp\left(-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds\right)$$

$t_f$ ，相机光线方向三维空间采样区段的远端。

$t_n$ ，相机光线方向三维空间采样区段的近端。

$\sigma$ ，相机光线方向三维空间采样点的体密度或不透明度(Opacity)，等作于光线在无限小粒子空间的差分概率( $dP$ , Differential Probability)。

$\mathbf{c}$ ，相机光线方向三维空间采样点颜色。

$T(t)$ ，光线从三维空间采样区段近端到当前采样点的透光率，即光线未遇到任何粒子的概率。

### 3. 相机光线颜色估计

将相机光线方向三维空间采样区段均匀划分为  $N$  个区间(Bins)，每个区间进行随机均匀采样。

$$t_i \sim \mathcal{U}\left[t_n + \frac{i-1}{N}(t_f - t_n), t_n + \frac{i}{N}(t_f - t_n)\right]$$

基于相机光线方向三维空间离散采样样本计算光线颜色。

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^N T_i (1 - \exp(-\sigma_i \delta_i)) \mathbf{c}_i, \text{ where } T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right)$$

$\delta_i = t_{i+1} - t_i$ ，相邻样本之间的距离。

$\alpha_i = 1 - \exp(-\sigma_i \delta_i)$ ，对应图像透明度通道(Alpha)。

## 三、 优化计算

### 1. 位置编码(Positional Encoding)

对于存在颜色与几何特性高频变化的三维空间目标，基于高频函数将三维空间点坐标(x, y, z,  $\theta$ ,  $\phi$ )映射为高维空间量，改进神经网络计算性能，便于多层感知器拟合高频函数。

相机光线方向三维空间采样点坐标位置分量及视线向量分量的位置编码定义。

$$\gamma(p) = (\sin(2^0 \pi p), \cos(2^0 \pi p), \dots, \sin(2^{L-1} \pi p), \cos(2^{L-1} \pi p))$$

位置分量，L 选 10；视线向量分量，L 选 4。

### 2. 层级采样(Hierarchical Volume Sampling)

通过对相机光线方向三维空间采样区段进行稀疏、稠密的层级化采样及计算，逐步优化实现相机光线渲染，使相机光线方向三维空间采样点查询尽量集中于三维目标区域，减少对自由空间及遮挡区域的采样，提高计算性能。

#### 1)、渲染过程

在相机光线方向三维空间采样区段进行稀疏粗采样，基于粗(Coarse)神经网络计算采样点体密度及颜色。

基于粗采样点颜色权重分布情况，在采样区段相关部分进行稠密细采样，基于细(Fine)神经网络计算粗、细采样点及光线颜色。

#### 2)、粗渲染

选  $N_c$  个采样点，基于粗神经网络进行计算。

$$\hat{C}_c(\mathbf{r}) = \sum_{i=1}^{N_c} w_i c_i, \quad w_i = T_i(1 - \exp(-\sigma_i \delta_i))$$

对权重  $w_i$  进行归一化计算，生成相机光线方向三维空间采样区段的基于分段常量 (Piecewise-Constant) 的概率密度函数(PDF, Probability Density Function)，以反映采样区段采样点颜色分布情况。

### 3)、细渲染

基于粗神经网络计算得到的三维空间采样点颜色分布情况，在与空间目标相关的高密度区域选  $N_f$  个采样点，基于细神经网络计算所有采样点( $N_c+N_f$ )的颜色，进行光线渲染投影。

## 3. 损失函数

基于相机光线粗采样渲染、细采样渲染投影颜色与真实图像像素颜色的方差和。

$$\mathcal{L} = \sum_{\mathbf{r} \in \mathcal{R}} \left[ \left\| \hat{C}_c(\mathbf{r}) - C(\mathbf{r}) \right\|_2^2 + \left\| \hat{C}_f(\mathbf{r}) - C(\mathbf{r}) \right\|_2^2 \right]$$

$\mathbf{r}$ ，光线。

$\mathcal{R}$ ，每个训练批次的光线集。

NeRF 基于辐射场概念，利用多层感知器及光线渲染技术实现视图合成方法的改进。

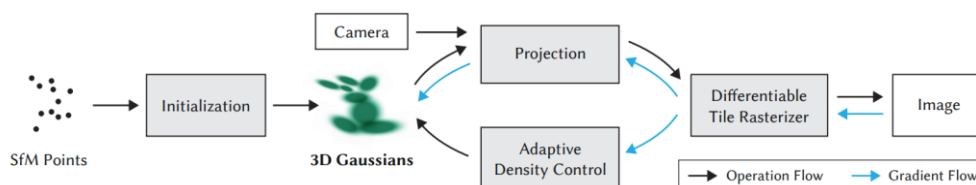


## 第四章. 三 维 高 斯 泼 溅 (3D Gaussian Splatting)

3D Gaussian Splatting, 基于辐射场原理、三维高斯函数及瓦片栅格化(Tile-Based Rasterization)方法实现三维场景椭球体素显式表示与实时渲染。

### 一、 模型框架

基于三维场景空间稀疏点云实例化三维高斯椭球体(Ellipsoids), 利用图像瓦片投影原理(Tile-Based Splatting)及相机光线方向椭球体光辐射量计算图像像素颜色, 通过误差优化计算更新三维高斯参数, 同时调控三维场景椭球体数量及形状。



框架图, 左侧, 基于移动重建(SfM, Structure from Motion)算法、利用相机图像序列生成三维场景空间的稀疏点云, 实例化以点云点位置作均值的三维高斯; 居中, 基于相机视锥区域及光辐射场原理计算相机光线方向三维高斯椭球体到相机图像空间的投影; 右侧, 基于瓦片渲染器(Differentiable Tile Rasterizer)计算投影像素对应的颜色。

优化控制(蓝线), 基于图像像素颜色误差及图像相似度误差进行优化计算, 更新三维高斯参数, 同时基于视域空间位置梯度(View Space Positional Gradients)等信息对三维高斯椭球体进行调控。

## 二、 模型定义

### 1. 三维高斯定义

$$G(x) = e^{-\frac{1}{2}(x)^T \Sigma^{-1}(x)}$$

$x$ ，三维场景空间稀疏点云点位置向量。

$\Sigma$ ，协方差矩阵。

### 2. 高斯投影

#### 1)、三维高斯协方差定义

三维高斯的协方差矩阵定义，基于椭球体轴缩放(Scaling)及旋转(Rotation)，以确保矩阵的物理意义及半正定性(Positive Semi-Definition)。

$$\Sigma = RSS^T R^T$$

$R$ ，旋转矩阵(Rotation Matrix)，四元数(Quaternion)表示。

$S$ ，缩放矩阵(Scaling Matrix)，三维向量(3D Vector)表示。

#### 2)、投影的协方差定义

三维高斯椭球体投影到相机二维图像空间椭圆的协方差。

$$\Sigma' = JW \Sigma W^T J^T$$

$\Sigma'$ , 二维图像空间椭圆的协方差。

$J$ , 相机投影变换的仿射近似的雅可比(Jacobian)。

$W$ , 相机视域变换(Viewing Transformation)。

### 3. 像素颜色定义

$$C = \sum_{i \in N} \prod_{j=1}^{i-1} (1 - a_j)$$

$c$ , 相机光线方向三维空间高斯椭球体颜色, 基于球谐函数(SH, Spherical Harmonics)计算。

$a_i$ , 相机光线方向三维空间椭球体投影的不透明度(Opacity), 基于二维图像空间投影椭圆高斯与三维空间高斯椭球体的不透明度的乘积计算。

### 4. 损失函数定义

$$L = (1 - \lambda)L_1 + \lambda L_{D-SSIM}$$

$L_1$ , 均值误差(MAE, Mean Absolute Error)。

$LD-SSIM$ , 图像相似度(D-SSIM, Displacement Structural Similarity Index Measure)。

优化计算时, 基于链式规则分别计算 $\Sigma'$ 对三维缩放向量及旋转四元数的导数。

### 三、 优化控制

#### 1. 优化控制

```
Algorithm 1 Optimization and Densification
w, h: width and height of the training images
M ← SfM Points » Positions
S, C, A ← InitAttributes() » Covariances, Colors, Opacities
i ← 0 » Iteration Count
while not converged do
 V, I ← SampleTrainingView() » Camera V and Image
 I ← Rasterize(M, S, C, A, V) » Alg. 2
 L ← Loss(I, Ĩ) » Loss
 M, S, C, A ← Adam(∇L) » Backprop & Step
 if IsRefinementIteration(i) then
 for all Gaussians (μ, Σ, c, α) in (M, S, C, A) do
 if α < ε or IsTooLarge(μ, Σ) then » Pruning
 RemoveGaussian()
 end if
 if ∇μL > τμ then » Densification
 if ||S|| > τS then » Over-reconstruction
 SplitGaussian(μ, Σ, c, α)
 else » Under-reconstruction
 CloneGaussian(μ, Σ, c, α)
 end if
 end if
 end for
 end if
 i ← i + 1
end while
```

##### 1)、三维高斯实例化

定义三维高斯均值参数集(M)，基于相机图像序列与移动重建(SfM)生成三维场景稀疏点云。

定义协方差(S)、颜色(C)及不透明度(A)参数集，基于稀疏点云实例化三维高斯集。

##### 2)、优化计算

选择相机视域(V)及样本图像。

基于三维高斯及相机视域进行栅格化渲染得到投影图像(I)。

基于投影图像与样本图像的损失函数进行优化计算，通过随机梯度下降法反向传播更新三维高斯参数。

##### 3)、稠密控制

对三维高斯椭球体集进行裁剪调控。

三维高斯椭球体不透明度太小或椭球体太大，则去掉(Prune)该三维高斯椭球体。

损失函数位置梯度大于阈值，过重建(Over-Reconstruction)，协方差大于阈值，用两个小的三维高斯进行替换；欠重建(Under-Reconstruction)，协方差小于等于阈值，在位置梯度方向对三维高斯进行复制。

## 2. 栅格渲染

基于图像瓦片化与基数排序(Radix Sort)算法，利用近似图层混合( $\alpha$ -Blending)方法实现三维场景高斯各向投影(Gaussian Anisotropic Splats)的栅格化渲染。

---

**Algorithm 2** GPU software rasterization of 3D Gaussians  
 $w, h$ : width and height of the image to rasterize  
 $M, S$ : Gaussian means and covariances in world space  
 $C, A$ : Gaussian colors and opacities  
 $V$ : view configuration of current camera

---

```

function RASTERIZE(w, h, M, S, C, A, V)
 CullGaussian(p, V) $\triangleright$ Frustum Culling
 $M', S' \leftarrow$ ScreenspaceGaussians(M, S, V) $\triangleright$ Transform
 $T \leftarrow$ CreateTiles(w, h)
 $L, K \leftarrow$ DuplicateWithKeys(M', T) $\triangleright$ Indices and Keys
 SortByKey(K, L) $\triangleright$ Globally Sort
 $R \leftarrow$ IdentifyTileRanges(T, K)
 $I \leftarrow 0$ $\triangleright$ Init Canvas
 for all Tiles t in I do
 for all Pixels i in t do
 $r \leftarrow$ GetTileRange(R, t)
 $I[i] \leftarrow$ BlendInOrder(i, L, r, K, M', S', C, A)
 end for
 end for
 return I
end function

```

---

去掉在相机视锥体区域(Frustum)远端、近端边界及外部的三维高斯。

基于相机视域信息、三维高斯均值集与协方差集，计算三维高斯投影到二维图像椭圆位置的均值与协方差集。

基于需栅格化图像的宽、高，定义瓦片集(T)，瓦片(16\*16 Pixel)。

基于三维高斯投影均值集与瓦片集定义高斯投影对应的瓦片索引与键值列表。每个瓦片对应一个投影序列(Splat List)；每个投影实例对应一个键值；每个键值(Key, 64bits)，高位，瓦片索引，低位，投影深度。

基于瓦片索引，利用基数排序算法对键值列表进行排序。

基于瓦片集与排序的键值列表定义瓦片对应的键值区间。

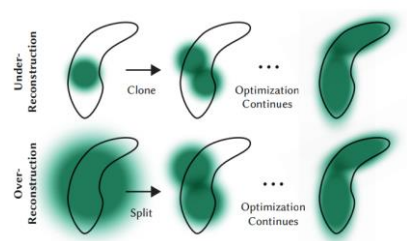
实例化画布。

基于瓦片索引获得瓦片对应的键值区间。

基于像素索引、瓦片索引集、键值区间、投影均值、投影协方差及三维场景高斯颜色等信息，利用图层混合方法计算像素颜色。实际计算时，可通过基于共享内存的线程块(Thread Block)实现并行化的栅格渲染。

生成图像。

### 3. 稠密控制



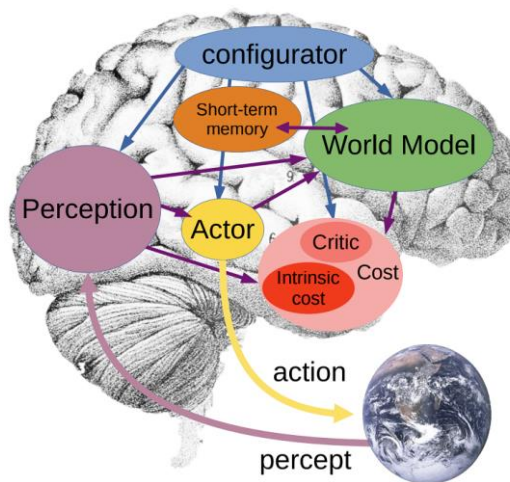
高斯投影小于小尺度几何(Small-Scale Geometry)时，复制三维高斯；大于时，拆分三维高斯。

## 世界模型篇

世界模型是机器人理解真实世界的基石。当前世界模型的五+1 类实现方法。

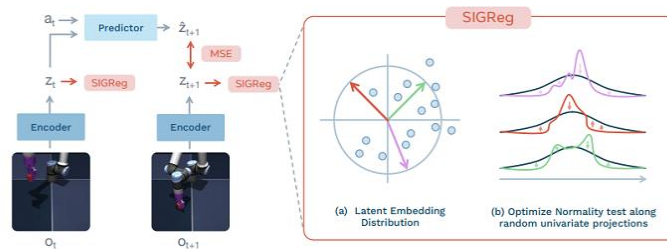
- 1、基于图像、视频理解的扩散生成式模型。比方，Google Genie3 等。
- 2、基于生成式 AI、侧重于内部世界(Internal world)的世界模型。比方，Li fei fei world lab 实验室的, Marble。
- 3、基于联合嵌量预测(Joint Embedding Predictive Architecture)。比方，Yann LeCun, LeJEPa。

LeJEPa 框架示例。



框架图，策略模块(Actor)基于感知(Perception)及记忆(Short-Term Memory)预测动作；世界模型模块(World Model)可视为仿真器，对动作可能产生的影响及环境外来状态进行预测；评价模块(Critic)基于内在代价(Intrinsic Cost)，比方，情感等，及外在代价，比方，执行结果等，进行代价判断。

基于视觉的 JEPA，LeWorldModel 端到端训练管程(Pipeline)。

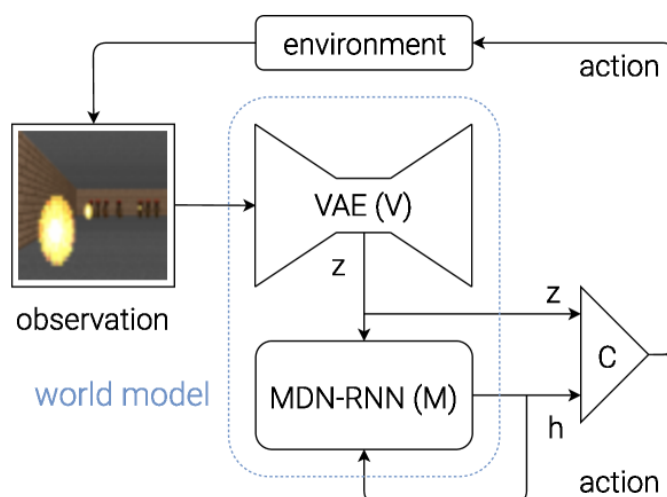


4、基于主动推理及半符号的形式。对象动力学等属性基于主动推理进行建模，混合了半象征方法。比方，Karl Friston 等，AXIOM，场景表示作对象的合成(Compositions)，基于逐像素线性投影建模，以获得对象间关联。

5、神经象征世界模型(Neuro-symbolic World Models)。

6、VAE + MDN-RNN + Control Model + Reinforcement Learning . Jürgen Schmidhuber.

基于变分自编码器(VAE)及混合密度循环网络(MDN-RNN, Mixture Density Network-Recurrent Neural Network)的世界模型框架。



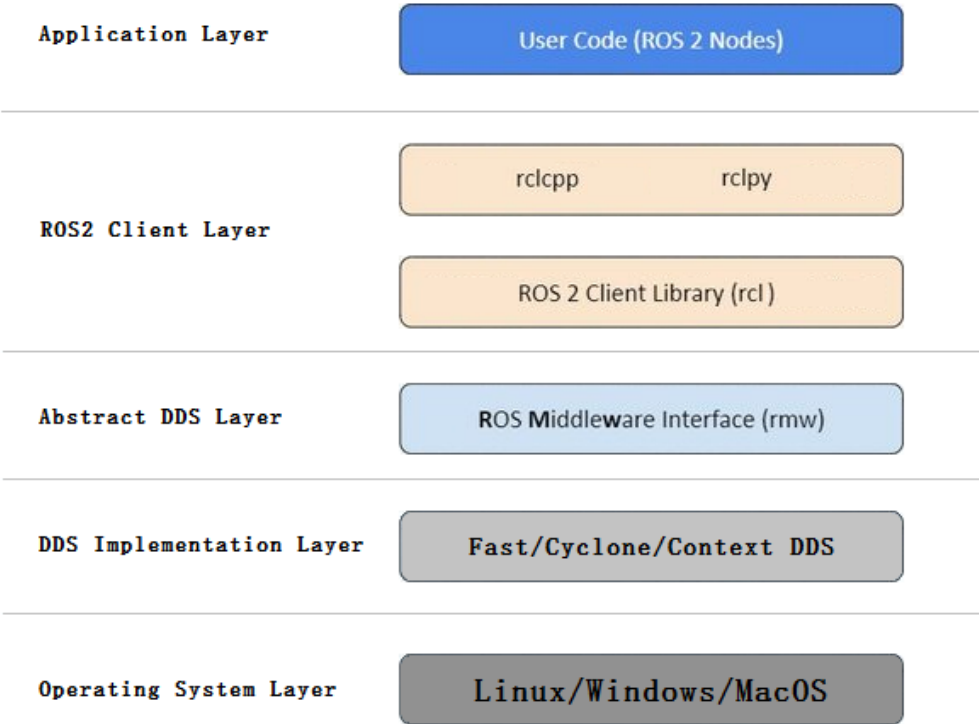
框架图，基于 VAE 提炼当前的视觉图像隐特征；基于 MDN-RNN，比方，高斯混合模型循环



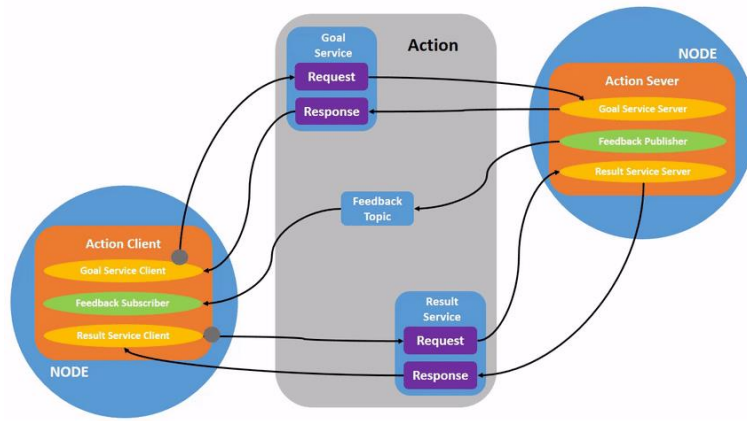
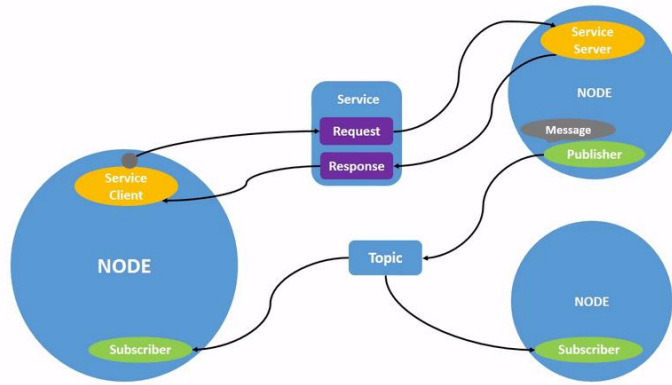
网络，利用历史信息预测下一个隐变量的概率密度分布(pdf)；控制模型基于深度强化学习，利用 VAE 得到的隐变量及基于 MDN-RNN 预测的概率分布采样的隐变量生成动作，使奖励回报最大。

# 机器人操作系统篇-ROS2

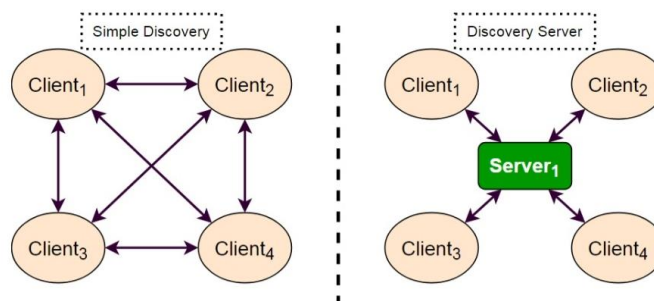
基于 DDS 通信机制的应用层机器人实时操作系统。

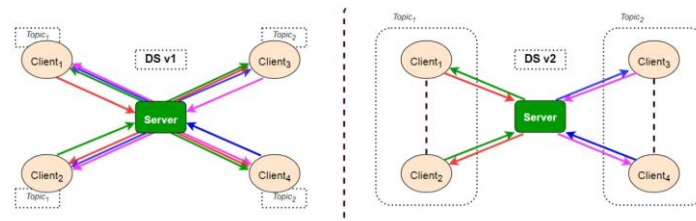


ROS2 每个节点具备独立的功能；节点之间通过 Topics，Services，Actions，Parameters 通信。

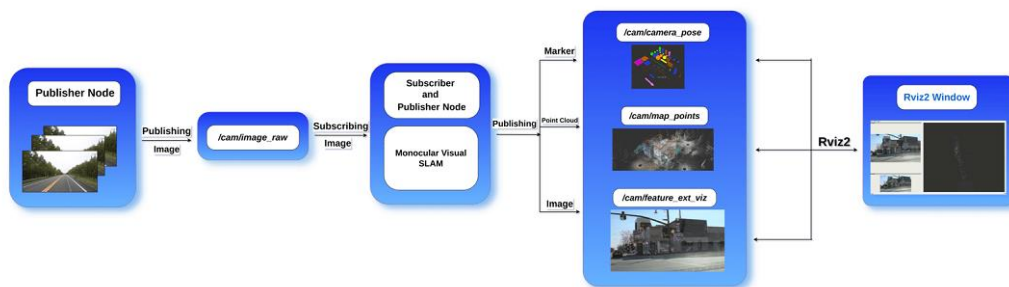


节点发现基于组播去中心化或集中化。





基于 ROS2 的单视角 SLAM 工作管程。



## 参考文献

Soumyadip. Introduction to Robotics: A Comprehensive Guide to Robotics for Beginners. 2024.

Sciencepedia. The LQR Cost Function.

Luis S. etc. Whole-Body Control of Humanoid Robots. 2018.

A. Okamura. Robot dynamics and simulation. 2019.

Russ T. Multi-Body Dynamics. 2024.

Jessy W. etc. Virtual Constraints and Hybrid Zero Dynamics for Realizing Underactuated Bipedal Locomotion. 2017.

Y. Y. Immerse Yourself in Reinforcement Learning and Robotics with MuJoCo. 2025.

Gymnasium. Training using REINFORCE for Mujoco.

Michael Kaup. etc. A Review of Nine Physics Engines for Reinforcement Learning Research. 2024.

Tim M. Monte-Carlo Tree Search (MCTS). 2023.

David S. etc. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm. 2017.

David S. etc. Mastering the Game of Go without Human Knowledge. 2017.

GR00T N1: An Open Foundation Model for Generalist Humanoid Robots.

FLOW MATCHING FOR GENERATIVE MODELING.

$\pi 0$ : A Vision-Language-Action Flow Model for General Robot Control.

LATENT ACTION PRETRAINING FROM VIDEOS.

NSVQ: Noise Substitution in Vector Quantization for Machine Learning.

Neural Discrete Representation Learning.

FLARE, Robot Learning with Implicit World Modeling.

REPRESENTATION ALIGNMENT FOR GENERATION , TRAINING DIFFUSION TRANSFORMERS I S  
EASIER THAN YOU THINK.

DREAMGEN, Unlocking Generalization in Robot Learning through Video World Models.

WAN, OPEN AND ADVANCED LARGE-SCALE VIDEO GENERATIVE MODELS.

DeXtreme: Transfer of Agile In-hand Manipulation from Simulation to Reality.

SOLVING RUBIK’S CUBE WITH A ROBOT HAND.

Ben Mildenhall. etc. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis.  
2020.

BERNHARD KERBL. etc. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. 2023.

Soumyadip. Understanding Visual SLAM for Robotics Perception: Building Monocular SLAM from Scratch in Python OpenCV. 2024.

Antoni Rosinol. etc. NeRF-SLAM: Real-Time Dense Monocular SLAM with Neural Radiance Fields. 2022.

Erik Sandstrom. etc. Splat-SLAM: Globally Optimized RGB-only SLAM with 3D Gaussians. 2024.

Greg Welch. etc. An Introduction to the Kalman Filter. 2006.

Peter S. Maybeck. Stochastic Models, Estimation, and Control Volume 1. 1979.

Giorgio Grisetti. etc. A Tutorial on Graph-Based SLAM.

Cyrill Stachniss. Graph-Based SLAM and Sparsity. 2016.

Cyrill Stachniss. Graph-Based SLAM with Landmarks. 2013.

Zachary Teed. etc. RAFT: Recurrent All-Pairs Field Transforms for Optical Flow. 2020.

Zachary Teed. etc. DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras. 2022.

Carlos Campos. etc. ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM. 2021.

Hidenobu Matsuki. etc. Gaussian Splatting SLAM. 2024.

Karnik Ram. Lie Groups and Lie Algebra. 2017.

Marina Meila. etc. Manifold learning: what, how, and why. 2023.

数学百科. <http://www.shuxueji.com/w/48475>.

Lars Mescheder. etc. Occupancy Networks: Learning 3D Reconstruction in Function Space. 2019.

仿真

MuJoCo Playground.

DeepMind Control Suite.

An Extensible, Data-Oriented Architecture for High-Performance, Many-World Simulation.

MuJoCo: A physics engine for model-based control.

Brax - A Differentiable Physics Engine for Large Scale Rigid Body Simulation.

WARP: An Efficient Engine for Multi-Vector Retrieval.

Reinforcement Learning with Augmented Data.

Mastering Visual Continuous Control: Improved Data-Augmented Reinforcement Learning.

Mastering Diverse Domains through World Models.

TD-MPC2: Scalable, Robust World Models for Continuous Control.

Lucas M. etc. LeWorldModel: Stable End-to-End Joint-Embedding Predictive Architecture from Pixels. 2026.